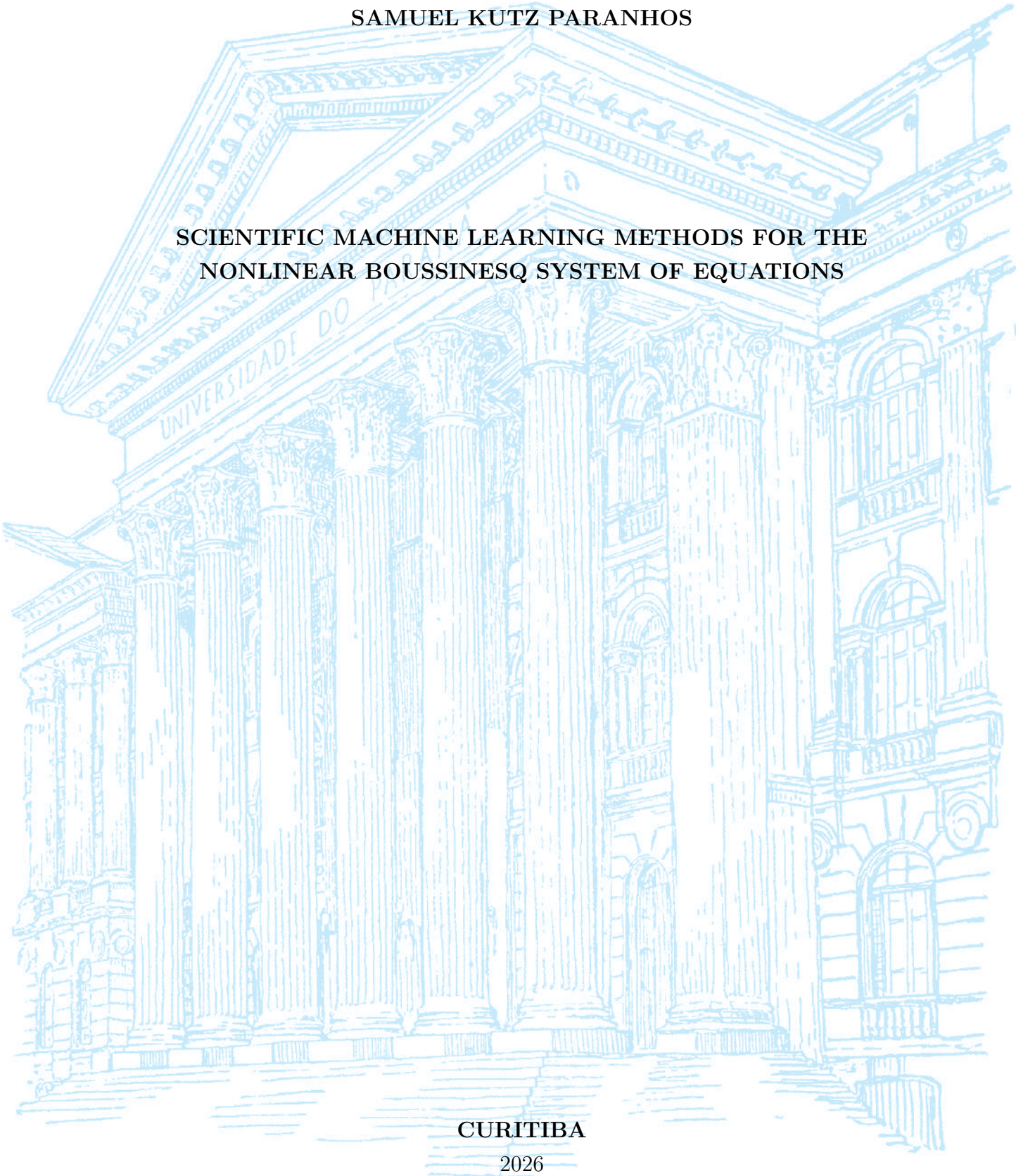
UNIVERSIDADE FEDERAL DO PARANÁ

SAMUEL KUTZ PARANHOS

SCIENTIFIC MACHINE LEARNING METHODS FOR THE NONLINEAR BOUSSINESQ SYSTEM OF EQUATIONS



CURITIBA

2026

SAMUEL KUTZ PARANHOS

TRABALHO DE CONCLUSÃO DE CURSO

SCIENTIFIC MACHINE LEARNING METHODS FOR THE NONLINEAR BOUSSINESQ SYSTEM OF EQUATIONS

Trabalho apresentado como requisito parcial à obtenção do grau de Bacharel em Matemática Industrial no Departamento de Matemática da Universidade Federal do Paraná.

Área de concentração: Matemática Industrial.

Orientador: Prof. Roberto Ribeiro.

Título do Projeto: Scientific Machine Learning Methods for the Nonlinear Boussinesq System of Equations.

CURITIBA

2026

RESUMO

Este trabalho investiga a eficácia de métodos de Scientific Machine Learning (SciML) na resolução e aproximação do sistema não linear de Boussinesq, um sistema de Equações Diferenciais Parciais (EDPs) utilizado para modelar a dinâmica de ondas aquáticas. Neste estudo, o sistema de Boussinesq será tratado como uma EDP dependente de dois parâmetros: os parâmetros de não linearidade e de dispersão. O objetivo é analisar o uso do ecossistema contemporâneo de ferramentas desenvolvido em torno do recente crescimento de Machine Learning como complemento aos métodos numéricos clássicos empregados na solução do sistema de Boussinesq. Foi utilizado o método pseudoespectral para gerar um conjunto de dados consistindo de soluções de referência do sistema de Boussinesq para diversos parâmetros e é então empregado como base de treinamento para três arquiteturas de SciML investigadas neste trabalho: Redes Neurais Informadas pela Física (PINNs), treinadas a um único parâmetro tanto com restrições de dados quanto de forma puramente física; Operadores Neurais de Fourier (FNO), operando em regime puramente supervisionado a vários parâmetros com uso exclusivo de dados sem nenhuma informação do operador diferencial da EDP; e Operadores Neurais Informados pela Física (PINO), que consiste em uma FNO com a inclusão do operador diferencial da EDP na função de perda da arquitetura. Como métricas de eficiência dessas arquiteturas, utilizamos a evolução do erro espectral e do erro relativo ao longo do espaço-tempo com relação a solução de referência. Como resultados, percebe-se que as PINNs tradicionais, especialmente quando treinadas sem o auxílio de dados, sofrem de um fenômeno conhecido como viés espectral, apresentando tendência a aprender primeiramente as componentes de baixa frequência da solução e grandes dificuldades em capturar detalhes essenciais de alta frequência à medida que a não linearidade aumenta. Em contrapartida, por aprenderem diretamente no espaço das frequências, os modelos de aprendizado de operadores, como o FNO e o PINO treinado com dados, atenuam substancialmente essa limitação nas baixas e médias frequências, embora nenhum método a elimine por completo: um erro residual de alta frequência persiste em todas as arquiteturas, e o viés reaparece de forma mais acentuada quando o PINO é treinado de maneira puramente física. Observa-se ainda que a inclusão do resíduo físico confere estabilidade temporal, suprimindo o artefato de Gibbs que degrada o FNO no instante final. Assim, concluímos que a inclusão de dados na função de perda constitui a estratégia mais eficaz para mitigar — ainda que não anular — o viés espectral comumente presente nessas arquiteturas.

Palavras-chave: Scientific Machine Learning; Equações Diferenciais Parciais; Sistema de Boussinesq; Redes Neurais Informadas pela Física.

ABSTRACT

This work investigates the effectiveness of Scientific Machine Learning (SciML) methods in solving and approximating the nonlinear Boussinesq system, a system of Partial Differential Equations (PDEs) used to model water wave dynamics. In this study, the Boussinesq system will be treated as a PDE dependent on two parameters: the nonlinearity and dispersion parameters. The objective is to analyze the use of the contemporary ecosystem of tools developed around the recent growth of Machine Learning as a complement to the classical numerical methods employed in solving the Boussinesq system. The pseudospectral method was used to generate a dataset consisting of reference solutions of the Boussinesq system for various parameters and is then employed as the training basis for three SciML architectures investigated in this work: Physics-Informed Neural Networks (PINNs), trained at a single parameter both with data constraints and purely physics-informed; Fourier Neural Operators (FNOs), operating in a purely supervised regime at multiple parameters using only data without any differential operator information from the PDE; and Physics-Informed Neural Operators (PINOs), which consist of an FNO with the inclusion of the PDE differential operator in the architecture loss function. As efficiency metrics for these architectures, we use the evolution of spectral error and relative error over space-time with respect to the reference solution. As results, it is observed that traditional PINNs, especially when trained without the aid of data, suffer from a phenomenon known as spectral bias, showing a tendency to learn the low-frequency components of the solution first and great difficulty capturing essential high-frequency details as nonlinearity increases. In contrast, by learning directly in the frequency space, operator learning models such as FNO and PINO trained with data attenuate this limitation substantially in the low and mid frequencies, although no method removes it entirely: a residual high-frequency error persists across all architectures, and the bias re-emerges more strongly when PINO is trained in a purely physics-informed manner. We further observe that the physics residual confers temporal stability, suppressing the Gibbs artifact that degrades the FNO at the final time. Thus, we conclude that including data in the loss function is the most effective strategy to mitigate — though not to eliminate — the spectral bias commonly present in these architectures.

Keywords: Scientific Machine Learning; Partial Differential Equations; Boussinesq System; Physics-Informed Neural Networks.

ACKNOWLEDGEMENTS

I would like to thank CNPq, the LabFluid laboratory, and the Federal University of Paraná (UFPR) for their support, infrastructure, and guidance during this research.

This work was supported by CNPq and developed with the collaboration of the LabFluid research group at UFPR.

CONTENTS

LIST OF FIGURES

LIST OF ABBREVIATIONS

SciML	Scientific Machine Learning
ML	Machine Learning
PDE	Partial Differential Equation
PINN	Physics-Informed Neural Network
FNO	Fourier Neural Operator
PINO	Physics-Informed Neural Operator
MLP	Multilayer Perceptron
RK4	Runge–Kutta 4th-order method
DFT	Discrete Fourier Transform
FFT	Fast Fourier Transform
NTK	Neural Tangent Kernel
MSE	Mean Squared Error
SGD	Stochastic Gradient Descent
AD	Automatic Differentiation

LIST OF SYMBOLS

Domain and general operators

x	spatial coordinate
t	temporal coordinate
$u(x,t)$	PDE solution (generic)
$\mathcal{D}$	differential operator of the PDE
$\mathcal{L}$	linear differential operator; also a loss functional
∇	gradient (nabla) operator
N_x	number of spatial grid points
N_t	number of time steps
Δx, Δt	spatial and temporal grid spacings

Boussinesq system and spectral quantities

$\eta(x,t)$	surface elevation field
$u(x,t)$	depth-averaged horizontal velocity field
α	nonlinearity parameter of the Boussinesq system
β	dispersion parameter of the Boussinesq system
A	initial wave amplitude
L	domain half-length, $\Omega = [-L, L]$
k	wavenumber (integer mode index)
ξ_k	wavenumber $\pi k/L$ on the domain $[-L, L]$
$\omega(k)$	angular frequency / dispersion relation
$\hat{f}(k)$	Fourier coefficient of f at mode k
$\hat{f}(k,t)$	time-evolving Fourier mode of f at wavenumber k

Neural networks and Neural Tangent Kernel

θ	trainable parameter vector of a neural network
u_θ	neural network approximation of the PDE solution
$h^{(l)}$	hidden-layer activation vector at layer l
σ	activation function
$W^{(l)}$, $b^{(l)}$	weight matrix and bias vector at layer l
N_{MLP}	number of neurons in a hidden layer
L_{MLP}	number of layers in the neural network
$K(\theta)$	empirical Neural Tangent Kernel, $K = JJ^{\mathsf{T}}$
λ_k	NTK eigenvalue associated with mode k
$\hat{e}(k,t)$	Fourier coefficient of the training error at mode k

Neural operators

$G^\dagger$, G_θ	target solution operator and its neural approximation
$\mathcal{K}_\ell$	integral operator of the ℓ-th FNO spectral layer
κ_ℓ	kernel function of the FNO integral operator
P	lifting operator of the FNO
Q	projection operator of the FNO
d_c	channel (width) dimension of the FNO
$R_{\ell,k}$	FNO spectral weight (discretized kernel) at layer ℓ, mode k
$k_{\max}$	FNO mode-truncation threshold

1 INTRODUCTION

Since the start of the scientific thinking, mathematics has been one of the key languages in which we can understand the world, and Partial Differential Equations (PDEs) are one of the main dialects of the mathematical language we use to model and to simplify nature's complexity.

When modelling these equations, we cannot always afford them to have simple closed-form solutions (that is, when they have solution at all), this exposes the need of obtaining approximated solutions coming from numerical methods for these PDEs, also giving the option to circumvent analytical complexity. This allows the extraction of predictions, comparisons with data, build simulations and so on. These classical numerical methods, widely studied for all types of differential equations, usually rely on discretizing the PDE's domain into grids and performing temporal steps.

With the recent ascension of Machine Learning (ML) methods and a whole ecosystem of tools built around it, a natural path is to bridge these two worlds and let ML augment the numerical solution of PDEs, taking advantage of the many software frameworks developed. This is one facet of Scientific Machine Learning (SciML), the broader effort to fuse physics-based, mechanistic models with data-driven methods. Within SciML, this work focuses on the numerical solution of PDEs and, given the vast literature around the topic, restricts its review of recent works to hydrodynamics equations.

1.1 Literature Overview on Scientific Machine Learning for PDEs

The neural network building blocks underlying all of these approaches trace a line beginning with the formal neuron of McCulloch et al. (1943) and Rosenblatt's perceptron (ROSENBLATT, 1958), the first trainable single-layer model. The key algorithmic advance that unlocked multi-layer training was the backpropagation algorithm (RUMELHART et al., 1986), making gradient-based optimization practical for deep architectures. Shortly after, the universal approximation theorem (CYBENKO, 1989; HORNIK et al., 1989) provided the theoretical guarantee for a multilayer perceptron (MLP) with a single hidden layer of sufficient width to be able to approximate any continuous function on a compact domain to arbitrary precision. This combination of practical and universal

approximation established the MLP as the backbone of modern deep learning. The term neural network itself reflects the biological metaphor at the heart of these models, in which McCulloch et al. (1943) drew an explicit analogy to the firing of biological neurons, and this vocabulary propagated through the field, causing MLPs to be routinely and interchangeably referred to as artificial neural networks (ANNs) or feedforward neural networks. The idea of "deep" in deep learning, popularized following the breakthrough results of LeCun et al. (2015), simply denotes an MLP with many hidden layers, a regime in which representational capacity grows substantially and where the backpropagation-based training paradigm proves to be exceptionally efficient.

These ML foundations become the basis of a scientific methodology once the methods are coupled to the physical laws that govern a problem of interest. This is the premise of Scientific Machine Learning, a term consolidated by Baker et al. (2019) to name the discipline at the intersection of scientific computing and machine learning, and surveyed in breadth by Karniadakis et al. (2021). Its unifying goal is to combine the interpretability, generalization, and inductive biases of physics-based models with the flexibility and data-adaptivity of ML, rather than treating the network as a pure black box. The methods span several families. Some recover governing equations directly from data, as in the Sparse Identification of Nonlinear Dynamics (SINDy) (BRUNTON et al., 2016). Others embed differential-equation structure into the network itself, as in Neural Ordinary Differential Equations (neural ODEs) (CHEN, R. T. Q. et al., 2018) and Universal Differential Equations (UDEs), in which a trainable network stands in for the unknown terms of an otherwise mechanistic model (RACKAUCKAS et al., 2020). These tools have been carried across a wide range of domains, from global weather and climate forecasting (BONEV et al., 2023) to molecular dynamics, materials design, and turbulence modelling (KARNIADAKIS et al., 2021). Among all these directions, the one that concerns us here is the use of SciML to obtain numerical solutions of PDEs and, more specifically, of the hydrodynamics equations that govern water waves. Given the vastness of the literature, the remainder of this review is devoted to that subarea.

The contemporary landscape of SciML for PDEs was reshaped by one idea in particular, the Physics-Informed Neural Networks (PINNs) (RAISSI et al., 2019), that is, the embedding of the governing PDE residual directly into the training loss of a Neural Network, working as regularization term for not only fitting data, but also respecting the underlying physics. In hydrodynamics, a growing body of work has applied PINNs to water flow (DE PINA et al., 2021, 2023), shallow-water dynamics on the sphere (BIHLO et al., 2022), and inverse problems in strongly nonlinear wave systems (JAGTAP et al., 2022). The consistent result that we can address from reviews (CUOMO et al., 2023) is that PINNs are most effective when observational data are sparse and the task is

parameter identification, namely inverse problems, which are known for usually being not well-posed problems. But, for a reliable precise forward simulation at scale, they do not yet rival classical solvers. In our vision, these approaches serve best to complement them.

As a natural extension from neural networks, there is a recent development in learning maps between infinite-dimensional function spaces, in which the theoretical foundation was popularized by T. Chen et al. (1995), who extended the universal approximation theorem to nonlinear operator, in other words, a shallow network can approximate any continuous operator between Banach spaces to arbitrary precision. Bringing this idea into practical terms, Lu et al. (2021) introduced DeepONet, the first deep architecture expressly designed for learning operators coming from differential equations, demonstrating it across a range of ODEs and PDEs. Building on this line of work, operator learning puts aside the single-instance limitation of PINNs by training a model to approximate the full mapping from parameters and initial data to the solution, so that at inference time a new solution costs only a forward pass for the network. Parallel to DeepONet, Fourier Neural Operator (FNO) (LI et al., 2021) is one of the most widely adopted instance of this idea, it parametrizes an integral kernel in Fourier space, achieving speed-ups over other neural operators and is discretization-independent, the so-called zero-shot superesolution. Subsequent works explored alternative bases like wavelets (GUPTA et al., 2021), but FNO was validated on many types of equations such as dispersive and soliton wave equations (PEI et al., 2022; ZHONG et al., 2023), was also caled to geophysical problems spanning regional ocean modeling, shallow-water emulation, and global weather forecasting (BONEV et al., 2023; LKHAGVASUREN et al., 2024; UCHIDA et al., 2025; PATEL et al., 2025; YU et al., 2025).

Following the idea for PINNs, The Physics-Informed Neural Operator (PINO) (LI et al., 2023a) combines operator learning with PDE-residual constraints, cutting the labelled-data requirement while preserving physical consistency. Applications to shallow-water systems (ROSOFSKY et al., 2023) confirmed that this hybrid regime inherits the generalization of FNO without its data hunger. Broad surveys of the field (TODO, 2023) identify these physics-data approaches working together as the most promising SciML direction, although highlighting turbulence modeling, scalability, and noisy-data robustness as open challenges.

A known limitation that cuts across all SciML architectures is the so-called spectral bias, the tendency of gradient-based optimizers to learn the low-frequency components of a solution faster than the high-frequency ones (RAHAMAN et al., 2019; XU et al., 2020). The theoretical account of this behavior came with the Neural Tangent Kernel (NTK) (JACOT et al., 2018), which showed that a sufficiently wide network trains, to leading order, as a linear model governed by a kernel that stays essentially fixed dur-

ing optimization, the so-called lazy-training regime. In this picture each mode of the solution is learned at a rate set by its associated NTK eigenvalue, so the wide disparity between the largest and smallest eigenvalues translates directly into the frequency imbalance seen during training (ARORA et al., 2019; CAO et al., 2021; BORDELON et al., 2020). Wang, Yu, et al. (2022) carried this analysis to PINNs, showing that the eigenvalue disparity between the PDE-residual and initial-condition loss components is what drives their spectral bias. This pathology is especially consequential for dispersive wave problems, where the solution energy is broadly distributed across wavenumbers. A recent systematic study (KHODAKARAMI et al., 2026) showed that second-order optimizers combined with spectrum-aware loss formulations substantially mitigate it.

As far as this review reveals, SciML methods have been applied to a broad class of shallow-water and dispersive wave equations. To the best of our knowledge, however, no study has systematically evaluated FNO, PINO, or PINNs on the parametric 1D Boussinesq system, where both the nonlinearity coefficient α and the dispersion coefficient β vary simultaneously. This gap motivates our effort to implement and evaluate these models with the goal of exposing their details for the user aiming to bring together this new set of tools.

1.2 The Boussinesq System of Equations

For this work, we choose to implement SciML methods for a specific PDE originating from J. Boussinesq's 1872 effort to give a theoretical account of the solitary wave of translation observed by J. Scott Russell in a canal in 1834 (BOUSSINESQ, 1872). Russell had noticed that a hump of water, set in motion by a stopping barge, could travel for miles without changing shape, a phenomenon the linear wave theory of the time could not explain. Boussinesq showed that the stability of such a wave is the result of a precise balance between nonlinear steepening and frequency dispersion, two effects that are separately destabilizing but mutually compensating in the shallow-water, long-wave regime. The mathematical structure of this balance, and of dispersive wave theory in general, are treated in depth by Whitham (1974) and Debnath (2012).

The modern two-field form of the system, for the free-surface elevation $\eta(x,t)$ and the depth-averaged horizontal velocity $u(x,t)$, was systematically derived and classified by Bona et al. (2002), who identified a four-parameter family of equivalent Boussinesq systems obtainable from the incompressible, irrotational Euler equations by asymptotic simplifications. Earlier, Peregrine (1967) had already written down the standard form of these equations for water of variable depth, establishing the notation and scaling that remain in widespread use. What makes the Boussinesq system particularly suitable as a parametric testbed for this work is that changing the nonlinearity and dispersion co-

efficients continuously deforms the solution from the near-linear, wave-packet regime to the strongly nonlinear soliton regime, giving rise to a more interesting range of behaviors within a single mathematical framework.

The specific one-dimensional form of the system used in this work, with its parameterization of the nonlinearity coefficient α and the dispersion coefficient β, was adopted from Martins (2023) and Martins et al. (2024). In the dataset generated for this study α and β are held equal, defining a one-parameter family of problems whose difficulty grows. Namely, for small $\alpha = \beta$ the dynamics are nearly linear and the solution is well-captured by low-frequency modes, while for large $\alpha = \beta$ the strong coupling between nonlinearity and dispersion produces sharper, more energetically distributed wave profiles that are fundamentally harder to learn. This controlled variation is precisely what makes the setting useful for comparing SciML architectures, with the goal of observing how each method learns the target solution's frequency.

Classical numerical methods for the Boussinesq system rely on pseudospectral and finite-difference discretizations that exploit the periodic or nearly periodic spatial structure of the waves. Pseudospectral (TREFETHEN, 2000) schemes achieve spectral accuracy on the spatial derivatives and are therefore well suited to the dispersive character of the equations (STEINMOELLER et al., 2012); this is the approach used to generate the reference dataset in this work. On the application side, operational coastal-engineering codes such as FUNWAVE (TODO, 2010) implement Boussinesq-type solvers for harbor and nearshore wave propagation, demonstrating the practical relevance of the model beyond the academic setting.

SciML methods have recently only been applied to the Boussinesq equation, not the specific Boussinesq System. Liu et al. (2023) applied parameter-integrated PINNs with phase-domain decomposition to reconstruct two-dimensional Boussinesq dynamics including phase transitions and rogue-wave formation, identifying spectral bias as a key obstacle at high nonlinearity. Wang et al. (2024) trained a physics-informed network to infer velocity and pressure fields from surface-elevation data alone on a Boussinesq model, demonstrating the inverse-problem capabilities of the approach. Complementary studies on related nonlinear wave equations (ZHANG et al., 2024; KUMAR et al., 2025) have confirmed that purely data-driven and purely physics-driven networks each display characteristic failure modes at strong nonlinearity. Taken together, these results suggest that a systematic, parametric evaluation of FNO/PINO/PINN on the 1D Boussinesq system, which is the focus of this work, can be fruitful to understand better the interactions between the method's ease of training and nonlinearities and dispersiveness.

1.3 Research Goals

This work implements and evaluates three SciML methods on the parametric 1D Boussinesq system: Physics-Informed Neural Networks (PINNs), Fourier Neural Operators (FNOs), and Physics-Informed Neural Operators (PINOs). All results are measured against a pseudospectral reference solver, which sets a high-accuracy baseline. The study is guided by the following questions.

- Accuracy relative to the pseudospectral baseline: how closely can each SciML method reproduce the reference solutions as nonlinearity and dispersion grow? Where does each method start to fail, and how does its error scale with the difficulty of the regime?
- Data-driven, physics-informed, and hybrid training: how does the presence or absence of supervised reference data affect the accuracy and training behavior of each method? Does adding a physics residual term improve generalization when data are scarce, or does it introduce new difficulties?
- Spectral bias across training regimes: how does spectral bias manifest in each architecture when the error spectrum is tracked band by band during training, and to what extent does the inclusion of data or physics constraints alter the frequency at which errors concentrate?
- The mechanism behind spectral bias: does Neural Tangent Kernel (NTK) theory genuinely predict the frequency imbalance seen on the Boussinesq system, or only describe it after the fact? We put the theory to two quantitative tests on the PINN residual, across a range of network widths. The first targets the frozen-kernel (lazy-training) assumption the analysis depends on: we measure how far the network parameters and the empirical kernel drift over training, and whether the kernel's eigenvalue spectrum is essentially unchanged from initialization to convergence, as lazy training requires. The second targets the predicted driver of the bias: an eigenvalue spectrum that decays across many orders of magnitude, so that the modes carrying the high-frequency detail are learned far more slowly than the smooth, low-frequency ones. Confirming both would tie the frequency-band errors seen during training to a concrete mechanism rather than a loose analogy.

Together, these questions aim to provide a clear picture of the strengths and limitations of each SciML approach for dispersive, nonlinear wave problems, and to offer practical guidance for choosing among them.

1.4 Work Structure

This work is written with the goal of sharing SciML for senior undergraduates in mathematics, physics, or engineering. We recommend the reader to have some knowledge of scientific computing and linear algebra. The remaining chapters are organized as follows.

- Chapter 2 develops the mathematical background. It covers the Boussinesq system and its analytical properties, the pseudospectral solver, and the theoretical foundations of MLP, PINN, FNO, and PINO, including a discussion of spectral bias.
- Chapter 3 describes the experimental methodology. It details the dataset generation procedure, the model architectures, the training setup, and the evaluation metrics used throughout.
- Chapter 4 presents the numerical results and discusses the performance of each method across parameter regimes and spatial resolutions.
- Chapter 5 summarizes the main findings and outlines directions for future work.

2 MATHEMATICAL BACKGROUND

This chapter introduces the mathematical foundations of the methods studied in this work. Section 2.1 presents the Boussinesq system and the limiting wave-equation regime. Section 2.2 develops the pseudospectral numerical method used to generate reference solutions. Sections 2.3 and 2.4 introduce, respectively, the neural network and neural operator architectures studied in this work.

2.1 The Boussinesq System

The Boussinesq system is a model for long nonlinear dispersive waves in shallow water, governing the coupled evolution of two scalar fields: the free surface elevation $\eta(x,t)$ and the depth-averaged horizontal velocity $u(x,t)$, both depending on position x and time t. In the one-dimensional, spatially periodic setting studied in this work, the full system reads:

$$\begin{cases} \eta_t + u_x + \alpha\,(\eta u)_x = 0, \\ u_t - \dfrac{\beta}{3}\,u_{xxt} + \eta_x + \alpha\,u u_x = 0, \\ \eta(x,0) = A\ \mathrm{sech}^2(x), \quad u(x,0) = 0, \quad x \in [-L, L], \end{cases} \tag{2.1}$$

where $\alpha \geq 0$ is the nonlinearity parameter, $\beta > 0$ is the dispersion parameter, $A > 0$ is the initial amplitude, $L > 0$ is the domain half-length, and $\mathrm{sech}(z) = 2/(e^z + e^{-z})$ is the hyperbolic secant. The initial condition places a solitary-wave-like crest at the center $x = 0$ with zero initial velocity. This nondimensional form follows equation (3.1) of Martins (2023) (see also Martins et al. (2024)).

We note that $uu_x = \frac{1}{2}(u^2)_x$, so the momentum equation in (2.1) can equivalently be written with the conservative flux $\frac{\alpha}{2}(u^2)_x$ in place of $\alpha\,uu_x$. Both forms appear in the implementations: the PINN residual uses uu_x directly via automatic differentiation, while the pseudospectral solver exploits $\frac{1}{2}(u^2)_x$ for the physical-space product, as detailed in Section 2.2.

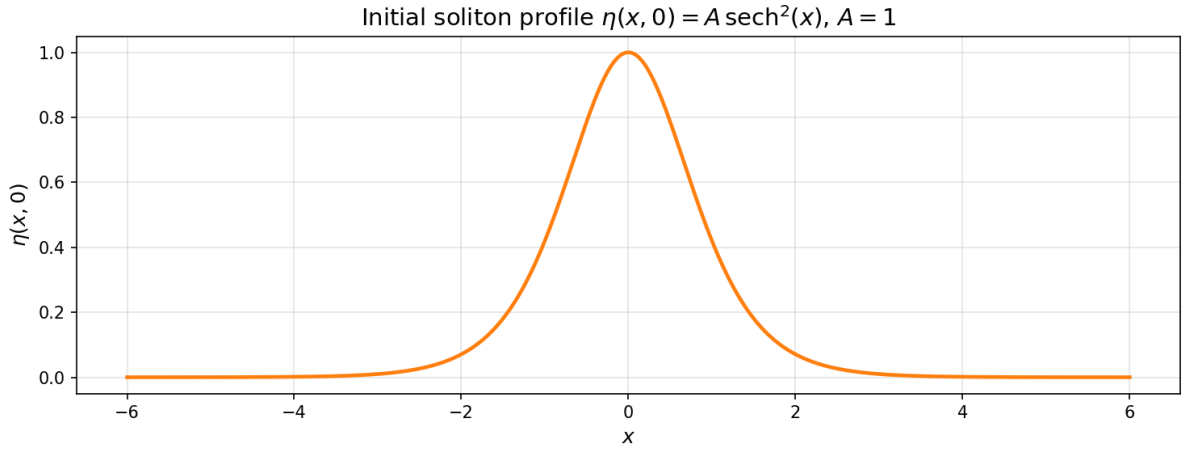


Figure 2.1 – Initial condition $\eta(x,0) = A\,\mathrm{sech}^2(x)$ for $A = 1$.

Source: The author (2026).

2.1.1 Recovering the Wave Equation

In the doubly linear limit $\alpha = \beta = 0$, all nonlinear and dispersive terms vanish and the Boussinesq system reduces to the symmetric hyperbolic system:

$$\eta_t + u_x = 0, \tag{2.2}$$

$$u_t + \eta_x = 0. \tag{2.3}$$

To obtain a single scalar equation, differentiate (2.2) with respect to t:

$$\eta_{tt} + u_{xt} = 0. \tag{2.4}$$

Differentiate (2.3) with respect to x:

$$u_{tx} + \eta_{xx} = 0. \tag{2.5}$$

Since $u_{xt} = u_{tx}$, subtracting the second from the first yields the wave equation for η:

$$\eta_{tt} = \eta_{xx}, \tag{2.6}$$

and the reverse procedure gives the same equation for u.

D'Alembert's formula (DEBNATH, 2012) provides the general solution to (2.6). Given initial data

$$\eta(x,0) = f(x), \qquad \eta_t(x,0) = g(x), \tag{2.7}$$

the solution is:

$$\eta(x,t) = \frac{f(x+t) + f(x-t)}{2} + \frac{1}{2}\int_{x-t}^{x+t} g(s)\,ds. \tag{2.8}$$

We now apply this to the initial conditions (2.1). From (2.2), $\eta_t(x,0) = -u_x(x,0) = 0$, so:

$$f(x) = A\,\mathrm{sech}^2(x), \qquad g(x) = 0. \tag{2.9}$$

Since $g \equiv 0$, the integral in (2.8) vanishes and the surface elevation evolves as:

$$\eta(x,t) = \frac{A}{2}\big[\mathrm{sech}^2(x+t) + \mathrm{sech}^2(x-t)\big]. \tag{2.10}$$

For u, from (2.3) we have $u_t(x,0) = -\eta_x(x,0)$, giving:

$$f(x) = 0, \qquad g(x) = -\eta_x(x,0). \tag{2.11}$$

The $f = 0$ term vanishes in (2.8), leaving:

$$u(x,t) = \frac{1}{2}\int_{x-t}^{x+t} \big(-\eta_x(s,0)\big)\,ds = -\frac{1}{2}\big[\eta(x+t,0) - \eta(x-t,0)\big]. \tag{2.12}$$

Substituting $\eta(x,0) = A\,\mathrm{sech}^2(x)$:

$$u(x,t) = \frac{A}{2}\big[\mathrm{sech}^2(x-t) - \mathrm{sech}^2(x+t)\big]. \tag{2.13}$$

The initial bump of amplitude A splits into two counter-propagating pulses of amplitude $A/2$, traveling at speed ± 1 without changing shape. Any deviation from this wave-equation baseline in the full system (2.1) is attributable either to dispersion ($\beta > 0$) or to nonlinearity ($\alpha > 0$).

2.2 Pseudospectral Method

The pseudospectral method exploits two complementary representations of the solution: Fourier space, where derivatives become exact algebraic multiplications, and physical space, where pointwise nonlinear operations are inexpensive. By alternating between the two representations as needed, the method attains spectral accuracy in space while handling the nonlinear coupling efficiently.

2.2.1 Spatial Discretization and the Discrete Fourier Transform

The spatial domain $\Omega = [-L, L]$ of total length $2L$ is discretized into N_x equispaced points:

$$x_j = -L + j\,\Delta x, \qquad \Delta x = \frac{2L}{N_x}, \qquad j = 0, \ldots, N_x - 1. \tag{2.14}$$

The Discrete Fourier Transform (DFT) of a function f sampled at these points is (TREFETHEN, 2000):

$$\hat{f}(k) = \sum_{j=0}^{N_x-1} f(x_j)\, e^{-2\pi i k j / N_x}, \qquad k = -\tfrac{N_x}{2}, \ldots, \tfrac{N_x}{2} - 1, \tag{2.15}$$

with its inverse recovering the physical values:

$$f(x_j) = \frac{1}{N_x} \sum_{k} \hat{f}(k)\, e^{2\pi i k j / N_x}. \tag{2.16}$$

The fundamental identity of the DFT with respect to differentiation is that, for a periodic function, the spatial derivative transforms into a pointwise multiplication (TREFETHEN, 2000; STEINMOELLER et al., 2012):

$$\widehat{(f_x)}(k) = i\frac{\pi k}{L}\,\hat{f}(k), \quad \widehat{(f_{xx})}(k) = -\frac{\pi^2 k^2}{L^2}\,\hat{f}(k). \tag{2.17}$$

This is in contrast to finite-difference methods, where derivatives are approximated with truncation error; in the DFT, equation (2.17) is exact for periodic functions.

We define the wavenumber of mode k on $[-L, L]$ as:

$$\xi_k \;:=\; \frac{\pi k}{L}. \tag{2.18}$$

With this definition, the derivative identities read:

$$\widehat{(f_x)}(k) = i\,\xi_k\,\hat{f}(k), \qquad \widehat{(f_{xx})}(k) = -\xi_k^2\,\hat{f}(k). \tag{2.19}$$

2.2.2 Spectral Formulation of the Boussinesq System

To bring the Boussinesq system (2.1) into the Fourier domain (MARTINS, 2023), we apply the DFT to each equation and use the derivative identities (2.19). The first equation, treating $\widehat{(\eta u)}(k,t)$ as a given nonlinear term, becomes:

$$\hat{\eta}_t(k,t) = -i\,\xi_k\,\hat{u}(k,t) - \alpha\, i\,\xi_k\,\widehat{(\eta u)}(k,t). \tag{2.20}$$

The second equation requires more care because of the u_{xxt} term. Since spatial derivatives commute with the DFT:

$$\widehat{(u_{xxt})}(k,t) = -\xi_k^2\,\hat{u}_t(k,t). \tag{2.21}$$

Substituting into the DFT of the momentum equation in (2.1):

$$\hat{u}_t(k,t) + \frac{\beta}{3}\,\xi_k^2\,\hat{u}_t(k,t) + i\,\xi_k\,\hat{\eta}(k,t) + \alpha\,i\,\xi_k\widehat{\left(\tfrac{1}{2}u^2\right)}(k,t) = 0. \tag{2.22}$$

Collecting the $\hat{u}_t(k,t)$ terms on the left and solving algebraically:

$$\hat{u}_t(k,t) = \frac{-i\,\xi_k\,\hat{\eta}(k,t) - \alpha\,i\,\xi_k\widehat{\left(\tfrac{1}{2}u^2\right)}(k,t)}{1 + \dfrac{\beta\,\xi_k^2}{3}}. \tag{2.23}$$

The operator $1-\frac{\beta}{3}\frac{\partial^2}{\partial x^2}$, which requires a linear solve in physical space, becomes the diagonal scalar $1 + \frac{\beta\,\xi_k^2}{3}$ in Fourier space, reducing its inversion to a single division per mode.

Equations (2.20) and (2.23) can be written jointly as a first-order ODE system in time for each fixed wavenumber $k \in \mathbb{Z}$:

$$\begin{cases} \hat{\eta}_t(k,t) = -i\,\xi_k\,\hat{u}(k,t) - \alpha\,i\,\xi_k\,\widehat{(\eta u)}(k,t), \\[2ex] \hat{u}_t(k,t) = \dfrac{-i\,\xi_k\,\hat{\eta}(k,t) - \alpha\,i\,\xi_k\widehat{\left(\tfrac{1}{2}u^2\right)}(k,t)}{1 + \dfrac{\beta\,\xi_k^2}{3}}. \end{cases} \tag{2.24}$$

The PDE system (2.1), two equations in (x,t), is thus equivalent to an infinite family of two-dimensional ODE systems (2.24), one for each $k \in \mathbb{Z}$, coupled through the nonlinear terms.

The nonlinear terms $\widehat{(\eta u)}(k,t)$ and $\widehat{(\frac{1}{2}u^2)}(k,t)$ cannot be computed directly in Fourier space without a convolution sum, which costs $O(N_x^2)$. The pseudospectral approach avoids this by computing products in physical space. For $\widehat{(\eta u)}(k,t)$, the procedure is:

1. recover $\eta(x_j)$ and $u(x_j)$ by applying the inverse DFT (2.16);
2. compute the product $(\eta u)_j = \eta(x_j) \cdot u(x_j)$ pointwise on the grid;
3. apply the DFT (2.15) to obtain $\widehat{(\eta u)}(k,t)$.

Similarly, $\widehat{(\frac{1}{2}u^2)}(k,t)$ is obtained by the same three-step procedure with $u^2(x_j) = u(x_j)^2$, where $u(x_j) = \mathrm{IDFT}(\hat{u})(x_j)$; and $(\eta u)(x_j) = \mathrm{IDFT}(\hat{\eta})(x_j) \cdot \mathrm{IDFT}(\hat{u})(x_j)$. Since each DFT/IDFT costs $O(N_x \log N_x)$ via the Fast Fourier Transform (FFT), this pseudospectral strategy assembles each nonlinear term in $O(N_x \log N_x)$ rather than $O(N_x^2)$.

2.2.3 Time Evolution via Runge-Kutta 4

We define the pseudospectral right-hand side operator F as the map

$$F(\hat{\eta}, \hat{u})(k) = \begin{pmatrix} -i\,\xi_k\,\hat{u}(k) - \alpha\, i\,\xi_k\,\widehat{(\eta u)}(k) \\ \dfrac{-i\,\xi_k\,\hat{\eta}(k) - \alpha\, i\,\xi_k\big(\widehat{\frac{1}{2}u^2}\big)(k)}{1 + \dfrac{\beta\,\xi_k^2}{3}} \end{pmatrix}, \tag{2.25}$$

so that $F(\hat{\eta}, \hat{u}) = \big(\hat{\eta}_t,\, \hat{u}_t\big)$ as given by equations (2.20)–(2.23), with nonlinear terms assembled by the pseudospectral procedure above. Time evolution uses the classical fourth-order Runge-Kutta (RK4) scheme (SÜLI et al., 2003; BURDEN et al., 2011). Given the spectral state $y_n = (\hat{\eta}^n, \hat{u}^n)$ at time t_n, the four stage evaluations $\kappa_1, \kappa_2, \kappa_3, \kappa_4$ and the update to $t_{n+1} = t_n + \Delta t$ are:

$$\begin{aligned} \kappa_1 &= F(y_n), \\ \kappa_2 &= F\big(y_n + \tfrac{\Delta t}{2}\,\kappa_1\big)\,, \\ \kappa_3 &= F\big(y_n + \tfrac{\Delta t}{2}\,\kappa_2\big)\,, \\ \kappa_4 &= F(y_n + \Delta t\,\kappa_3)\,, \\ y_{n+1} &= y_n + \frac{\Delta t}{6}\,(\kappa_1 + 2\,\kappa_2 + 2\,\kappa_3 + \kappa_4). \end{aligned} \tag{2.26}$$

At each stage, the intermediate state is decoded to physical space to assemble the nonlinear products, then encoded back to Fourier space to evaluate the spectral derivatives. This split between physical-space products and Fourier-space derivatives is the defining operation of the pseudospectral method.

2.2.4 Training Dataset

The numerical experiments in this work vary only the PDE parameters (α, β) while keeping the initial condition fixed at (2.1). The models are therefore trained and evaluated on the dataset

$$\mathcal{S} = \big\{\big((\alpha_i, \beta_i),\, (\eta_i, u_i)\big)\big\}_{i=1}^{M}, \tag{2.27}$$

where (α_i, β_i) are the nonlinearity and dispersion parameters for the i-th sample, and (η_i, u_i) is the corresponding space-time solution obtained from the pseudospectral solver of Section 2.2 with those parameters and the shared initial condition (2.1).

2.3 Neural Networks

This section introduces the neural network architectures central to this work together with the theory that explains their training behavior. We begin with the Multilayer Perceptron as the building block, develop the Neural Tangent Kernel framework and the spectral bias it predicts for a network fitting data on a regular grid, and then turn to Physics-Informed Neural Networks, showing how the same kernel analysis extends to the physics-informed setting where spectral bias is a well-documented obstacle.

2.3.1 Multilayer Perceptrons

In this work, we focus on Multilayer Perceptrons (MLPs), popularized by Rumelhart et al. (1986). These networks are compositions of affine transformations parametrized by tunable weights and biases, separated by nonlinear activations, where an optimization process fits the network to a dataset.

Given a labeled dataset $\{(x_i, u_i)\}_{i=1}^{N}$, the objective is to fit the function $u_\theta(x)$ by minimizing a loss function, here the Mean Squared Error (MSE). With $\|\cdot\|$ denoting the Euclidean norm, the unconstrained optimization problem is:

$$\mathcal{L}_{\text{MLP}}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \|u_\theta(x_i) - u_i\|^2. \tag{2.28}$$

The MLP $u_\theta : \mathbb{R}^{d_{\text{in}}} \to \mathbb{R}^{d_{\text{out}}}$ is defined by the composition:

$$h^{(0)} = x \in \mathbb{R}^{d_{\text{in}}}, \tag{2.29}$$

$$h^{(l)} = \sigma\left(W^{(l)} h^{(l-1)} + b^{(l)}\right) \in \mathbb{R}^{N_{\text{MLP}}}, \quad l = 1, \ldots, L_{\text{MLP}} - 1, \tag{2.30}$$

$$u_\theta(x) = W^{(L_{\text{MLP}})} h^{(L_{\text{MLP}}-1)} + b^{(L_{\text{MLP}})} \in \mathbb{R}^{d_{\text{out}}}. \tag{2.31}$$

where:

- $L_{\text{MLP}} \in \mathbb{N}$ is the number of layers and $N_{\text{MLP}} \in \mathbb{N}$ the number of neurons per hidden layer;
- $W^{(l)} \in \mathbb{R}^{N_{\text{MLP}} \times N_{\text{MLP}}}$ and $b^{(l)} \in \mathbb{R}^{N_{\text{MLP}}}$ are the weight matrix and bias vector at layer l, with $W^{(1)} \in \mathbb{R}^{d_{\text{in}} \times N_{\text{MLP}}}$ and $W^{(L_{\text{MLP}})} \in \mathbb{R}^{N_{\text{MLP}} \times d_{\text{out}}}$ adapted at the boundaries;
- $\sigma : \mathbb{R} \to \mathbb{R}$ is a sigmoidal activation function applied elementwise, satisfying $\sigma(t) \to 1$ as $t \to +\infty$ and $\sigma(t) \to 0$ as $t \to -\infty$;
- $\theta = \{W^{(l)}, b^{(l)}\}_{l=1}^{L_{\text{MLP}}}$ denotes all trainable parameters.

If the activation function is sigmoidal, MLPs are universal approximators: they can approximate any continuous function on a compact domain to arbitrary precision (CYBENKO, 1989).

In Figure 2.2, we show a common depiction of the neural network. Here the input vector is the pair $(x, t) \in \mathbb{R}^2$ and the output is $(\eta_\theta(x, t), u_\theta(x, t)) \in \mathbb{R}^2$.

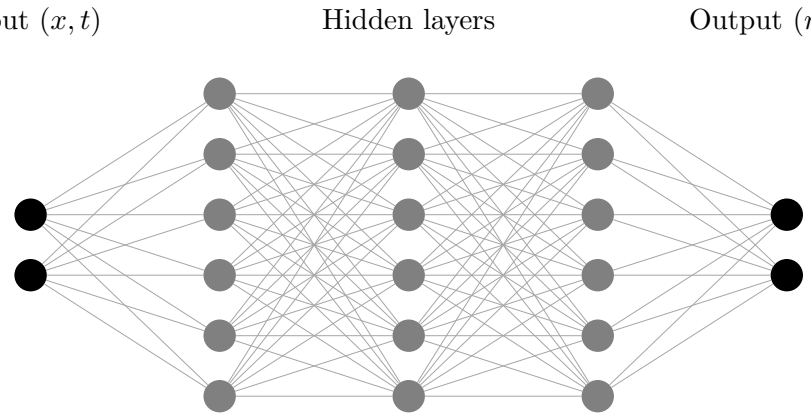


Figure 2.2 – Example of MLP architecture of input (x, t), with $L_{\text{MLP}} = 3$ hidden layers and $N_{\text{MLP}} = 6$ neurons per layer.

Source: The author (2026).

2.3.2 Gradient Flow

To understand how an MLP minimizes its data-fitting loss (2.28), consider gradient descent with learning rate $\mu > 0$. Starting from an initialization $\theta^{(0)}$, each iteration updates the parameters by

$$\theta^{(k+1)} = \theta^{(k)} - \mu \nabla_\theta \mathcal{L}(\theta^{(k)}), \qquad k = 0, 1, 2, \ldots, \tag{2.32}$$

where $\theta^{(k)}$ denotes the parameter vector after k steps. Rearranging (2.32) and dividing by the step size μ isolates the per-step increment as a difference quotient:

$$\frac{\theta^{(k+1)} - \theta^{(k)}}{\mu} = -\nabla_\theta \mathcal{L}(\theta^{(k)}). \tag{2.33}$$

To pass from this discrete recursion to a continuous trajectory, assign to each iterate a training-time stamp

$$\tau^{(k)} = k\mu, \qquad \text{so that} \qquad \tau^{(k+1)} - \tau^{(k)} = \mu, \tag{2.34}$$

spacing successive iterates by μ along a time axis τ. We then regard the discrete sequence $\{\theta^{(k)}\}_{k=0}^{\infty}$ as samples of a smooth curve $\theta(\tau)$ evaluated at these instants:

$$\theta^{(k)} = \theta(\tau^{(k)}), \qquad \theta^{(k+1)} = \theta(\tau^{(k+1)}) = \theta(\tau^{(k)} + \mu). \tag{2.35}$$

Substituting (2.35) into the left-hand side of (2.33) recasts it as a forward finite-difference quotient of θ in τ:

$$\frac{\theta(\tau^{(k)} + \mu) - \theta(\tau^{(k)})}{\mu} = -\nabla_\theta \mathcal{L}\big(\theta(\tau^{(k)})\big). \tag{2.36}$$

As the step size $\mu \to 0$, the spacing in (2.34) shrinks to zero, the discrete stamps $\tau^{(k)}$ fill out a continuous variable τ, and the left-hand side of (2.36) converges to the derivative $d\theta/d\tau$ by definition of the derivative. The recursion thus becomes the gradient flow ODE:

$$\frac{d\theta}{d\tau} = -\nabla_\theta \mathcal{L}(\theta), \qquad \theta(0) = \theta^{(0)}. \tag{2.37}$$

The gradient flow (2.37) is a central object in optimization, and it is a key insight that many standard first-order methods can be recovered as different time-discretization schemes of this same continuous trajectory. The discrete update (2.32) is precisely the forward (explicit) Euler discretization of (2.37) with step size μ, so gradient descent is just one numerical integrator for the flow; applying instead the backward (implicit) Euler scheme recovers the proximal gradient method (PARIKH et al., 2013). Reading optimizers as discretizations of (2.37) thus organizes them into a single family, and it is the continuous flow that we analyze in what follows.

2.3.3 Neural Tangent Kernel

Consider the MLP fit to the dataset $\{(x_i, u_i)\}_{i=1}^{N}$ of Section 2.3.1. Group the pointwise fitting errors into a single error vector:

$$e(\theta) = \begin{bmatrix} u_\theta(x_1) - u_1 \\ \vdots \\ u_\theta(x_N) - u_N \end{bmatrix} \in \mathbb{R}^N. \tag{2.38}$$

Up to the constant factor $1/N$, the loss (2.28) is the squared norm of this vector, which we use in the form

$$\mathcal{L}(\theta) = \frac{1}{2}\,\|e(\theta)\|^2 = \frac{1}{2}\sum_{i=1}^{N} e_i(\theta)^2. \tag{2.39}$$

Define the Jacobian of the error vector with respect to the parameters:

$$J(\theta) = \frac{\partial e}{\partial \theta} \in \mathbb{R}^{N \times N_\theta}, \qquad J_{ij} = \frac{\partial e_i}{\partial \theta_j}, \quad i = 1, \ldots, N,\ j = 1, \ldots, N_\theta, \tag{2.40}$$

where N_θ is the total number of trainable parameters. Each row $J_i = (J_{i1}, \ldots, J_{iN_\theta}) \in \mathbb{R}^{N_\theta}$ is the parameter-gradient of the i-th residual. Since $\mathcal{L} = \frac{1}{2}\sum_i e_i^2$, the j-th component of the loss gradient is:

$$\frac{\partial \mathcal{L}}{\partial \theta_j} = \sum_{i=1}^{N} e_i(\theta)\, \frac{\partial e_i}{\partial \theta_j} = \sum_{i=1}^{N} J_{ij}\, e_i(\theta) = \big(J(\theta)^T e(\theta)\big)_j. \tag{2.41}$$

Collecting all $j = 1, \ldots, N_\theta$ into a column vector:

$$\nabla_\theta \mathcal{L} = J(\theta)^T\, e(\theta) \in \mathbb{R}^{N_\theta}. \tag{2.42}$$

Substituting (2.42) into the gradient flow (2.37):

$$\frac{d\theta}{d\tau} = -J(\theta)^T\, e(\tau). \tag{2.43}$$

To find how the error vector itself evolves, differentiate each component $e_i(\theta(\tau))$ with respect to τ by the chain rule:

$$\frac{de_i}{d\tau} = \sum_{j=1}^{N_\theta} \frac{\partial e_i}{\partial \theta_j}\, \frac{d\theta_j}{d\tau} = J_i(\theta) \cdot \frac{d\theta}{d\tau}. \tag{2.44}$$

Stacking this identity for all $i = 1, \ldots, N$ into a matrix equation:

$$\frac{de}{d\tau} = J(\theta)\, \frac{d\theta}{d\tau}. \tag{2.45}$$

Substituting (2.43):

$$\frac{de}{d\tau} = J(\theta)\big(-J(\theta)^T e(\tau)\big) = -\underbrace{J(\theta)\, J(\theta)^T}_{K(\theta)\, \in\, \mathbb{R}^{N \times N}}\, e(\tau). \tag{2.46}$$

The matrix $K(\theta) = J(\theta)J(\theta)^T$ is the empirical Neural Tangent Kernel (NTK) (JACOT et al., 2018). It is symmetric positive semi-definite by construction: for any $v \in \mathbb{R}^N$,

$$v^T K(\theta)\, v = v^T J(\theta)\, J(\theta)^T v = \big\| J(\theta)^T v \big\|^2 \geq 0. \tag{2.47}$$

The (i, j) entry of $K(\theta)$ follows directly from expanding the matrix product $J(\theta)J(\theta)^T$:

$$K_{ij}(\theta) = \sum_{l=1}^{N_\theta} J_{il}(\theta)\, J_{jl}(\theta) = \sum_{l=1}^{N_\theta} \frac{\partial e_i}{\partial \theta_l}(\theta)\, \frac{\partial e_j}{\partial \theta_l}(\theta) = \nabla_\theta e_i(\theta)^T\, \nabla_\theta e_j(\theta). \tag{2.48}$$

This is the inner product in $\mathbb{R}^{N_\theta}$ of the parameter gradients of the network output at the i-th and j-th training points, where $e_i = u_\theta(x_i) - u_i$. Hence $K(\theta) \in \mathbb{R}^{N\times N}$ is the Gram matrix of the network's sensitivities across the whole dataset, and equation (2.46) shows that it alone governs how the fitting error evolves under gradient flow.

2.3.4 Spectral Bias

The spectral bias (or frequency principle) is the empirical observation that neural networks trained by gradient-based methods reduce the low-frequency components of the error faster than the high-frequency ones (RAHAMAN et al., 2019). This section derives that phenomenon directly from the NTK dynamics established above. When the training points lie on a regular grid and the target function is periodic, the error admits a discrete Fourier representation, and one can ask how its frequency content is approximated along the training iterations. We first establish the decay in the eigenbasis of the NTK and then, by transforming to Fourier space, show that it is precisely the low frequencies that are resolved first.

For a network with a sufficiently large number of parameters, the Jacobian $J(\theta)$ changes negligibly during training (JACOT et al., 2018; WANG; YU, et al., 2022). Writing $J_0 = J(\theta(0))$ and $K_0 = J_0 J_0^T$, the approximation $K(\theta(\tau)) \approx K_0$ turns equation (2.46) into a linear ODE with constant coefficients:

$$\frac{de}{d\tau} = -K_0\, e(\tau), \qquad e(0) = e_0. \tag{2.49}$$

Written component-wise, the i-th row of this system is

$$\frac{de_i}{d\tau} = -\sum_{j=1}^{N} K_0^{ij}\, e_j(\tau), \qquad i = 1, \ldots, N, \tag{2.50}$$

where $K_0^{ij} = \nabla_{\theta_0} e_i^T\, \nabla_{\theta_0} e_j$ is the inner product, at initialization, of the parameter gradients of the network output at training points x_i and x_j. The rate of change of the error at point i is therefore a weighted sum of all current errors, with weights given by these pairwise kernel values. This coupling is what makes the geometry of the sample grid and the architecture of the network jointly determine the convergence of every component.

Since $K_0 \in \mathbb{R}^{N\times N}$ is symmetric and positive semi-definite, the spectral theorem

guarantees a decomposition

$$K_0 = V\,\Lambda\,V^T, \tag{2.51}$$

where $V = [v_1 \mid \cdots \mid v_N]$ is an orthogonal matrix ($V^TV = I$) and

$$\Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_N), \qquad \lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_N \geq 0. \tag{2.52}$$

Introduce the change of coordinates

$$c(\tau) = V^T e(\tau), \tag{2.53}$$

so that, since V is orthogonal, $e(\tau) = Vc(\tau)$. Differentiating (2.53) with respect to τ and substituting the ODE (2.49):

$$\frac{dc}{d\tau} = V^T \frac{de}{d\tau} = -V^T K_0\, e(\tau). \tag{2.54}$$

Inserting $K_0 = V\Lambda V^T$ and $e(\tau) = Vc(\tau)$:

$$\frac{dc}{d\tau} = -V^T\,(V\Lambda V^T)\,(Vc(\tau)) = -(V^TV)\,\Lambda\,(V^TV)\,c(\tau) = -\Lambda\,c(\tau), \tag{2.55}$$

using $V^TV = I$ at each step. Since Λ is diagonal, this decouples into N independent scalar equations:

$$\frac{dc_k}{d\tau} = -\lambda_k\,c_k(\tau), \qquad k = 1, \ldots, N. \tag{2.56}$$

Each equation in (2.56) is a first-order linear ODE with constant coefficient (STROGATZ, 2018). Separating the variables and integrating both sides from the initial state at $\tau = 0$ up to time τ, with s and r as integration variables for the amplitude and the time,

$$\int_{c_k(0)}^{c_k(\tau)} \frac{ds}{s} = -\lambda_k \int_0^{\tau} dr \qquad \Longrightarrow \qquad \ln\frac{|c_k(\tau)|}{|c_k(0)|} = -\lambda_k\,\tau. \tag{2.57}$$

Exponentiating both sides yields the solution

$$c_k(\tau) = c_k(0)\,e^{-\lambda_k\tau}, \tag{2.58}$$

where $c_k(0) = v_k^T e_0$ is the projection of the initial error e_0 onto the k-th eigenvector of K_0. For $\lambda_k = 0$ the equation gives $c_k(\tau) = c_k(0)$ for all τ: that eigendirection never decays.

Returning to the original coordinates via $e(\tau) = Vc(\tau)$:

$$e(\tau) = \sum_{k=1}^{N} \left(v_k^T e_0\right) e^{-\lambda_k\tau}\, v_k. \tag{2.59}$$

The error decomposes into N independent eigendirections. Each eigenvector v_k carries initial amplitude $v_k^T e_0$ and decays exponentially at rate λ_k: eigenvectors with large λ_k are suppressed quickly, while those with small λ_k persist throughout training. This per-eigendirection decay, each at the rate set by its eigenvalue, is the rigorous statement of spectral bias established by Cao et al. (2021); the same spectral ordering governs generalization as the training set grows (BORDELON et al., 2020).

Equation (2.59) is the continuous flow; gradient descent takes finite steps, and its error evolution inherits the same eigenstructure in discrete form. Linearizing the update (2.32) about the frozen Jacobian J_0, a single step moves the parameters by $\theta^{(n+1)} - \theta^{(n)} = -\mu\, J_0^T e^{(n)}$, and to first order the error responds as $e^{(n+1)} - e^{(n)} = J_0\,(\theta^{(n+1)} - \theta^{(n)})$. Combining the two,

$$e^{(n+1)} = e^{(n)} - \mu\, J_0 J_0^T e^{(n)} = (I - \mu K_0)\, e^{(n)}. \tag{2.60}$$

Iterating from $e^{(0)} = e_0$ and diagonalizing with $K_0 = V \Lambda V^T$,

$$e^{(n)} = (I - \mu K_0)^n e_0 = \sum_{k=1}^{N} \big(v_k^T e_0\big)\,(1 - \mu\lambda_k)^n\, v_k. \tag{2.61}$$

This is the forward-Euler counterpart of (2.59): each eigendirection is multiplied by $(1 - \mu\lambda_k)^n$ rather than $e^{-\lambda_k \tau}$, and the two coincide as $\mu \to 0$ with $\tau = n\mu$, since $(1 - \mu\lambda_k)^n \to e^{-\lambda_k \tau}$. The discrete mode contracts only when $|1 - \mu\lambda_k| < 1$, that is $\mu < 2/\lambda_k$, so the largest eigenvalue caps the stable learning rate at $\mu < 2/\lambda_{\max}$.

From (2.58), the amplitude of the k-th eigenvector at time τ satisfies $|c_k(\tau)| = |v_k^T e_0|\, e^{-\lambda_k \tau}$. To reduce this amplitude below a target precision $\varepsilon > 0$:

$$|v_k^T e_0|\, e^{-\lambda_k \tau} < \varepsilon \implies e^{-\lambda_k \tau} < \frac{\varepsilon}{|v_k^T e_0|} \implies -\lambda_k\, \tau < \ln \frac{\varepsilon}{|v_k^T e_0|} \implies \tau > \frac{1}{\lambda_k} \ln \frac{|v_k^T e_0|}{\varepsilon}. \tag{2.62}$$

The minimum training time to resolve the k-th eigenvector to precision ε is therefore

$$\tau_k = \frac{1}{\lambda_k} \ln \frac{|v_k^T e_0|}{\varepsilon}. \tag{2.63}$$

Since $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_N \geq 0$, equation (2.63) gives $\tau_1 \leq \tau_2 \leq \cdots \leq \tau_N$: higher-indexed eigenvectors require more training time. Arora et al. (2019) reached similar conclusions from a discrete-time analysis of gradient descent, showing that the convergence speed of each eigenvector is governed by its eigenvalue and by the projection of the target onto that eigenvector.

The decomposition (2.59) is expressed in the eigenbasis $\{v_k\}_{k=1}^N$ of the NTK, which

need not coincide with any physically meaningful basis. When the N training points lie on a regular grid and the target is periodic, however, the natural basis is the Fourier one, and recasting the dynamics there shows that the decay is genuinely organized by frequency. Let $F \in \mathbb{C}^{N\times N}$ be the Fourier matrix, that is, the discrete Fourier transform (DFT) matrix with entries

$$F_{jl} = \frac{1}{\sqrt{N}}\,\omega^{jl}, \qquad \omega = e^{-2\pi \mathrm{i}/N}, \qquad j,l = 0,\dots,N-1. \tag{2.64}$$

With this normalization F is unitary, $F^*F = I$, so that $F^{-1} = F^*$, where F^* is the conjugate transpose (GOLUB et al., 2013; STRANG, 2006). Applying F to the error vector yields its spectrum, with inverse transform F^{-1}:

$$\hat{e}(\tau) = F\,e(\tau), \qquad e(\tau) = F^{-1}\hat{e}(\tau) = F^*\hat{e}(\tau). \tag{2.65}$$

In particular $\hat{e}_0 = Fe_0$ is the spectrum of the initial error, the difference between the spectra of the initial network output and the target. Since F is constant in τ, applying it to the linearized dynamics (2.49) carries the error evolution into the frequency domain:

$$\frac{d\hat{e}}{d\tau} = F\,\frac{de}{d\tau} = -FK_0\,e(\tau) = -FK_0F^{-1}\,\hat{e}(\tau). \tag{2.66}$$

Inserting the eigendecomposition $K_0 = V\Lambda V^T$ from (2.51),

$$FK_0F^{-1} = FV\,\Lambda\,V^TF^{-1} = (FV)\,\Lambda\,(FV)^* = (FV)\,\Lambda\,(FV)^{-1}, \tag{2.67}$$

where $V^TF^{-1} = V^*F^* = (FV)^*$ uses that V is real orthogonal, and $(FV)^* = (FV)^{-1}$ uses that a product of unitary matrices is unitary. The columns of FV, namely the Fourier transforms $\{Fv_k\}_{k=1}^N$ of the NTK eigenvectors, therefore diagonalize the frequency-domain dynamics. Defining the modal amplitudes $\tilde{c}(\tau) = (FV)^{-1}\hat{e}(\tau)$ and repeating the steps (2.53)–(2.58) gives

$$\frac{d\tilde{c}_k}{d\tau} = -\lambda_k\,\tilde{c}_k(\tau) \qquad \Longrightarrow \qquad \tilde{c}_k(\tau) = \tilde{c}_k(0)\,e^{-\lambda_k\tau}. \tag{2.68}$$

These are the same amplitudes as before: $\tilde{c} = (FV)^{-1}Fe = V^{-1}e = V^Te = c$, the projection of the error onto the NTK eigenvectors, now read in Fourier space. Transforming back through (2.65) yields the evolution of the error spectrum,

$$\hat{e}(\tau) = \sum_{k=1}^{N}\big[(Fv_k)^*\hat{e}_0\big]\,e^{-\lambda_k\tau}\,(Fv_k). \tag{2.69}$$

Equation (2.69) is the spectral-bias statement in the literal sense of the frequency spectrum: the error spectrum is a superposition of the transformed eigenvectors Fv_k, each damped at its own rate λ_k. The frequencies resolved first are those carried by the eigenvectors with the largest eigenvalues; empirically these eigenvectors are smooth and concentrated in the low Fourier modes (RAHAMAN et al., 2019; XU et al., 2020), so the low-frequency content of the error decays first while the high-frequency content persists, exactly the behaviour observed in practice.

To close, note that when the network is initialized at small scale its output u_{θ_0} is negligible relative to the target, a common simplification in spectral-bias analyses. The initial error is then the target with opposite sign, and so is its spectrum:

$$\hat{e}_0 = F\big(u_{\theta_0} - u\big) = \hat{u}_{\theta_0} - \hat{u} \approx -\,\hat{u} = -\begin{bmatrix} \hat{u}(0) \\ \vdots \\ \hat{u}(N-1) \end{bmatrix}. \tag{2.70}$$

Substituting (2.70) into (2.69) casts the dynamics as a reconstruction of the target frequency by frequency: each Fourier mode of u is acquired at a rate set by the NTK eigenvalues, the smooth low-frequency content emerging first and the fine high-frequency detail last. This is exactly the regime anticipated at the start of the section — a periodic target on a regular grid — in which the quality of the approximation can be tracked mode by mode along the training iterations.

This last observation can be made quantitative. Taking the modulus of the modal amplitude (2.68) isolates the magnitude of the k-th component along training,

$$|\tilde{c}_k(\tau)| = |\tilde{c}_k(0)|\, e^{-\lambda_k \tau}, \qquad |\tilde{c}_k(0)| = |(Fv_k)^* \hat{e}_0| \approx |(Fv_k)^* \hat{u}|, \tag{2.71}$$

where the last estimate uses the small-scale initialization (2.70), so that $|(Fv_k)^*\hat{u}|$ measures how strongly the target overlaps with the k-th eigen-mode. Training, however, advances in discrete steps: by the construction of the gradient flow (2.34), the continuous time and the iteration count n are related by $\tau = n\mu$, with μ the learning rate. Substituting $\tau = n\mu$ into (2.71) and requiring the amplitude to fall below a target precision $\varepsilon > 0$,

$$|(Fv_k)^*\hat{u}|\, e^{-\lambda_k n\mu} < \varepsilon \implies n > \frac{1}{\mu\,\lambda_k}\, \ln \frac{|(Fv_k)^*\hat{u}|}{\varepsilon}, \tag{2.72}$$

gives an estimate of the number of gradient-descent iterations needed to resolve the k-th frequency:

$$n_k = \left\lceil \frac{1}{\mu\,\lambda_k}\, \ln \frac{|(Fv_k)^*\hat{u}|}{\varepsilon} \right\rceil. \tag{2.73}$$

2.3.5 Physics-Informed Neural Networks

Introduced by Raissi et al. (2019), Physics-Informed Neural Networks (PINNs) insert the PDE directly into the optimization process, forcing the neural network to satisfy the model equations and approximating the PDE solution.

Consider the general PDE Initial Value Problem:

$$\begin{cases} \mathcal{D}[u](x,t) = 0, \quad \forall (x,t) \in \Omega \times (0,T], \\ \quad u(x,0) = u_0(x), \quad x \in \Omega, \end{cases} \tag{2.74}$$

where $\mathcal{D}$ is a differential operator on u, $\Omega \times (0,T]$ is the spatio-temporal domain, and u_0 is the initial condition.

The PDE residual loss penalizes violation of the governing equation at collocation points $\{(x_i,t_i)\}_{i=1}^{N_\mathcal{D}} \subset \Omega \times (0,T]$:

$$\mathcal{L}_{\text{pde}}(\theta) = \frac{1}{N_\mathcal{D}} \sum_{i=1}^{N_\mathcal{D}} \left\| \mathcal{D}[u_\theta](x_i,t_i) \right\|^2 . \tag{2.75}$$

The initial condition is enforced at collocation points $\{x_i\}_{i=1}^{N_0} \subset \Omega$:

$$\mathcal{L}_{\text{ic}}(\theta) = \frac{1}{N_0} \sum_{i=1}^{N_0} \left\| u_\theta(x_i,0) - u_0(x_i) \right\|^2 . \tag{2.76}$$

The combined PINN loss is:

$$\mathcal{L}_{\text{PINN}}(\theta) = \mathcal{L}_{\text{pde}}(\theta) + \mathcal{L}_{\text{ic}}(\theta). \tag{2.77}$$

Due to the compositional structure of MLPs, PINN implementations evaluate PDE residuals via Automatic Differentiation (AD) (LINNAINMAA, 1970; PASZKE et al., 2019), using ADAM (KINGMA et al., 2014) or L-BFGS (NOCEDAL, 1980) as the optimizer.

Application to the Boussinesq Dataset

For the Boussinesq system, a separate PINN is trained for each sample $(\alpha_i, \beta_i) \in \mathcal{S}$ (2.27). The differential operator $\mathcal{D}$ in the generic problem (2.74) is instantiated as the pair of Boussinesq residuals with fixed parameters (α_i, β_i):

$$\mathcal{D}_1[u_\theta] = (\eta_\theta)_t + (u_\theta)_x + \alpha_i \left((\eta_\theta \, u_\theta)_x \right), \tag{2.78}$$

$$\mathcal{D}_2[u_\theta] = (u_\theta)_t - \tfrac{\beta_i}{3} (u_\theta)_{xxt} + (\eta_\theta)_x + \alpha_i \, (u_\theta (u_\theta)_x). \tag{2.79}$$

The combined loss (2.77) for sample i then reads:

$$\begin{aligned}\mathcal{L}^i_{\mathrm{PINN}}(\theta) = \frac{1}{N_{\mathcal{D}}}\sum_{l=1}^{N_{\mathcal{D}}}\Big[\mathcal{D}_1[u_\theta](x_l,t_l)^2 + \mathcal{D}_2[u_\theta](x_l,t_l)^2\Big] \\ + \frac{1}{N_0}\sum_{l=1}^{N_0}\Big[(\eta_\theta(x_l,0)-\eta_0(x_l))^2 + (u_\theta(x_l,0)-u_0(x_l))^2\Big],\end{aligned} \tag{2.80}$$

where η_0 and u_0 are given by the shared initial condition (2.1). Because the parameters (α_i, β_i) enter the residuals explicitly, each PINN encodes knowledge of a single sample and must be retrained for every new parameter pair.

2.3.6 Spectral Bias in Physics-Informed Neural Networks

The Neural Tangent Kernel analysis of Sections 2.3.3–2.3.4 was carried out for an MLP fitting data on a uniform grid, the setting in which the kernel is approximately circulant and its eigenvectors are genuine Fourier modes. A PINN does not fit data directly: its loss (2.77) combines an initial-condition term with a PDE-residual term. The same machinery nevertheless applies once both constraints are stacked into a single error vector, now indexed over the N_r residual points in place of the N data points of the MLP,

$$e(\theta) = \begin{bmatrix} u_\theta(x_1,0)-u_0(x_1) \\ \vdots \\ u_\theta(x_{N_0},0)-u_0(x_{N_0}) \\ \mathcal{D}[u_\theta](x_1,t_1) \\ \vdots \\ \mathcal{D}[u_\theta](x_{N_{\mathcal{D}}},t_{N_{\mathcal{D}}}) \end{bmatrix} \in \mathbb{R}^{N_r}, \qquad N_r = N_0 + N_{\mathcal{D}}, \tag{2.81}$$

so that, up to normalization, $\mathcal{L}_{\mathrm{PINN}}$ reduces to the same quadratic form $\frac{1}{2}\|e\|^2$ analyzed for the MLP. With this definition the gradient-flow derivation leading to (2.46) carries over verbatim, with the Jacobian now also containing the parameter gradients of the PDE residuals, and the error still obeys the linear dynamics $\frac{de}{d\tau} = -K_0\, e$, with convergence of each eigenvector governed by its eigenvalue exactly as in (2.63) (WANG; YU, et al., 2022).

This prediction is borne out empirically. Spectral bias is among the most consistently reported obstacles in PINN training: networks fit smooth, low-frequency solutions readily but stall on sharp gradients and high-frequency content (RAHAMAN et al., 2019; XU et al., 2020; WANG; TENG, et al., 2021), and the same pathology underlies several documented PINN failure modes (KRISHNAPRIYAN et al., 2021). Wang, Yu, et al. (2022) traced the effect directly to the NTK, showing that its spectrum is concentrated at

low eigenvalues, so that high-frequency PDE residuals are the last to be minimized; for the Boussinesq system specifically, Wang et al. (2024) report the same difficulty in resolving the finer wave structure. A range of remedies target precisely this imbalance, including adaptive loss weighting (WANG; TENG, et al., 2021), Fourier feature embeddings, and domain decomposition. Understanding the spectral bias is therefore essential background for the neural-operator methods studied in the remainder of this work, whose ability to capture the high-frequency content of the Boussinesq solutions is examined directly in the numerical results of Chapter 4.

2.4 Neural Operators

Neural operators generalize neural networks from learning finite-dimensional mappings to learning operators between infinite-dimensional function spaces. This section introduces the general framework, then the Fourier Neural Operator (FNO) as the principal architecture, and finally the Physics-Informed Neural Operator (PINO) as the physics-constrained extension.

2.4.1 General Definition

Let $D \subset \mathbb{R}^d$ be a bounded open domain and let $\mathcal{A}(D;\mathbb{R}^{d_a})$ and $\mathcal{U}(D;\mathbb{R}^{d_u})$ be Banach spaces of functions (KREYSZIG, 1978). The target operator is a (potentially nonlinear) map

$$G^\dagger : \mathcal{A} \to \mathcal{U}, \qquad a \mapsto u = G^\dagger(a), \tag{2.82}$$

where both the input a and the output u are functions defined on D.

In this work, $G^\dagger$ is the solution operator of the Boussinesq system: given a parameter encoding (c, u_0) containing the PDE coefficients (α, β) and the initial condition u_0, the operator returns the full space-time solution (η, u).

A neural operator G_θ approximates $G^\dagger$ by composing linear integral operators with pointwise nonlinearities. Each linear layer applies an operator of the form:

$$(\mathcal{K}v)(x) = \int_D \kappa(x,y)\, v(y)\, dy, \tag{2.83}$$

where $\kappa : D \times D \to \mathbb{R}^{d_u \times d_u}$ is a learned kernel. The full neural operator architecture stacks these integral layers into the composition:

$$G_\theta = Q \circ \sigma\Big(W_L + \mathcal{K}_L\Big) \circ \cdots \circ \sigma\Big(W_1 + \mathcal{K}_1\Big) \circ P, \tag{2.84}$$

where:

- $P : \mathbb{R}^{d_a} \to \mathbb{R}^{d_c}$ is the lifting operator, mapping the input $a(x)$ pointwise to a higher-dimensional channel representation with $d_c \gg d_a$;
- $\sigma\Big(W_\ell + \mathcal{K}_\ell\Big)$, for $\ell = 1, \ldots, L$, are the operator layers, each combining a local linear map W_ℓ with a nonlocal integral operator $\mathcal{K}_\ell$, followed by a pointwise activation σ;
- $Q : \mathbb{R}^{d_c} \to \mathbb{R}^{d_u}$ is the projection operator mapping the final channel representation back to the output dimension.

Different choices of kernel parametrization for each $\mathcal{K}_\ell$ give rise to different neural operator architectures.

2.4.2 Fourier Neural Operators

Presented by Li et al. (2020), the Fourier Neural Operator (FNO) is the neural operator (2.84) in which each $\mathcal{K}_\ell$ uses a shift-invariant kernel: $\kappa_\ell(x, y) = \kappa_\ell(x - y)$. This assumption turns each integral operator into a convolution, which by the convolution theorem can be evaluated efficiently in the frequency domain.

Each spectral layer updates the channel representation via:

$$v_{\ell+1}(x) = \sigma\Big((W_\ell\, v_\ell)(x) + (\mathcal{K}_\ell v_\ell)(x)\Big). \tag{2.85}$$

The operator $\mathcal{K}_\ell$ acts via convolution with the shift-invariant kernel κ_ℓ:

$$(\mathcal{K}_\ell v_\ell)(x) = \int_D \kappa_\ell(x - y)\, v_\ell(y)\, dy. \tag{2.86}$$

By the convolution theorem, this equals:

$$(\mathcal{K}_\ell v_\ell)(x) = \mathcal{F}^{-1}\Big(\mathcal{F}(\kappa_\ell) \cdot \mathcal{F}(v_\ell)\Big)(x). \tag{2.87}$$

To see how this is parametrized, recall that for a periodic function f on $[0, 2\pi]$:

$$f(x) = \sum_{k\in\mathbb{Z}} \hat{f}(k)\, e^{ikx}, \qquad \hat{f}(k) = \frac{1}{2\pi}\int_0^{2\pi} f(x) e^{-ikx}\, dx. \tag{2.88}$$

The convolution $(\kappa * v)$ simplifies to:

$$(\kappa * v)(x) = \int_0^{2\pi} \kappa(x - y)\, v(y)\, dy = \sum_{k\in\mathbb{Z}} \big(2\pi\, \hat{\kappa}(k)\, \hat{v}(k)\big)\, e^{ikx}. \tag{2.89}$$

Instead of learning κ_ℓ as a function in physical space, the FNO directly parametrizes its Fourier coefficients as learnable complex-valued matrices $R_{\ell,k} \approx 2\pi\, \hat{\kappa}_\ell(k)$. Only the

$|k| \leq k_{\max}$ low-frequency modes are retained, with all others set to zero; $k_{\max}$ is a hyperparameter.

Training Objective

The FNO is trained as a pure supervised learning problem. Given the dataset $\mathcal{D}$ (2.27) of parameter-solution pairs, the training objective minimizes the mean squared error between the predicted and reference space-time fields:

$$\mathcal{L}_{\text{FNO}}(\theta) = \frac{1}{M}\sum_{i=1}^{M}\frac{1}{N_x N_t}\sum_{j=0}^{N_x-1}\sum_{n=0}^{N_t}\Big[(\eta_\theta^i(x_j,t_n)-\eta_i(x_j,t_n))^2+(u_\theta^i(x_j,t_n)-u_i(x_j,t_n))^2\Big], \tag{2.90}$$

where $(\eta_\theta^i, u_\theta^i) \coloneqq G_\theta(\alpha_i, \beta_i)$ is the network prediction for sample i.

Application to Non-Time-Periodic Problems

When the solution is also assumed to be periodic in time with period T (so that $D = [0, 2\pi] \times [0, T]$), shift-invariance extends to the time direction as well: $\kappa_\ell(x, y, t, \tau) = \kappa_\ell(x - y,\, t - \tau)$. The convolution then generalizes to:

$$(\mathcal{K}_\ell v_\ell)(x,t) = \int_D \kappa_\ell(x-y,\, t-\tau)\, v_\ell(y,\tau)\, dy\, d\tau = \mathcal{F}^{-1}\Big(\mathcal{F}(\kappa_\ell)\cdot\mathcal{F}(v_\ell)\Big)(x,t), \tag{2.91}$$

and the FNO parametrizes the two-dimensional Fourier modes (k_x, k_t) with $|k_x| \leq k_{x,\max}$ and $|k_t| \leq k_{t,\max}$, both hyperparameters.

For problems that are not time-periodic, however, applying the Fourier transform along the temporal axis implicitly extends the signal periodically, potentially introducing a jump discontinuity at the temporal boundary. This can trigger the Gibbs phenomenon (GIBBS, 1899), producing oscillatory artifacts near the boundary that persist regardless of how many temporal modes are retained. Standard signal-processing techniques such as zero-padding and window functions (e.g. Hann, Hamming windows) can partially mitigate these effects, at the cost of altering the temporal structure of the signal. Li et al. (2020) acknowledge this limitation and provide an alternative formulation in which the Fourier layers operate only along the spatial axis while the model advances forward in time, avoiding the periodicity assumption in the temporal direction entirely. In this work we deliberately retain the two-dimensional space-time formulation, in which the Fourier layers act on both the spatial and the temporal axes, so that a single operator emits the entire trajectory in one forward pass. This keeps the architecture simple and uniform across space and time, but, as just noted, it reintroduces the temporal-periodicity assumption for a Boussinesq solution that is not time-periodic, and therefore admits a Gibbs artifact

at the temporal boundary $t = T$. Rather than suppress this artifact with windowing, we retain the plain formulation and examine the boundary behaviour directly in Chapter 4, where it turns out to distinguish the purely data-driven operator from its physics-informed variant.

2.4.3 Physics-Informed Neural Operators

Analogously to how PINNs add a physics-informed residual loss to the MLP training objective, the Physics-Informed Neural Operator (PINO) (LI et al., 2023a) incorporates PDE constraints into the FNO training. The FNO backbone remains unchanged; what differs is the loss function, which now combines a data term with residuals evaluated on the predicted space-time fields.

Unlike a PINN, which represents the solution as a single continuous function and differentiates it via automatic differentiation, the FNO outputs (η_θ, u_θ) as a discrete $N_x \times N_t$ array in a single forward pass over the full space-time domain. Evaluating the PDE residuals therefore reduces to numerically differentiating this output array.

Spatial derivatives are computed pseudospectrally using ξ_k (2.18): the DFT of η_θ along the spatial axis at each time t_n yields $\hat{\eta}_\theta(k, t_n)$, from which

$$(\eta_x)_j = \mathcal{F}^{-1}(i\,\xi_k\,\hat{\eta}_\theta(k,t_n))_j\,, \qquad (\eta_{xx})_j = \mathcal{F}^{-1}\big(-\xi_k^2\,\hat{\eta}_\theta(k,t_n)\big)_j\,, \tag{2.92}$$

and identically for u_θ. Time derivatives are computed by finite differences: second-order central in the interior and first-order one-sided at the boundaries,

$$(\eta_\theta)_t^n = \begin{cases} \dfrac{\eta_\theta^{n+1} - \eta_\theta^n}{\Delta t}, & n = 0, \\ \dfrac{\eta_\theta^{n+1} - \eta_\theta^{n-1}}{2\,\Delta t}, & 1 \le n \le N_t - 1, \\ \dfrac{\eta_\theta^n - \eta_\theta^{n-1}}{\Delta t}, & n = N_t, \end{cases} \tag{2.93}$$

and identically for $(u_\theta)_t^n$. The mixed term $(u_\theta)_{xxt}$ is obtained by first computing $(u_\theta)_{xx}$ spectrally and then applying the time-difference stencil to the result. This combination of spectral spatial derivatives and finite-difference temporal derivatives follows the reference implementation of Li et al. (2023a) (LI et al., 2023b).

With all derivatives available, the Boussinesq equations (2.1) define two differential operators acting on the predicted fields:

$$\mathcal{D}_1(\eta_\theta, u_\theta) \coloneqq (\eta_\theta)_t + (u_\theta)_x + \alpha\,(\eta_\theta\,u_\theta)_x, \tag{2.94}$$

$$\mathcal{D}_2(\eta_\theta, u_\theta) \coloneqq (u_\theta)_t - \tfrac{\beta}{3}(u_\theta)_{xxt} + (\eta_\theta)_x + \alpha\,(u_\theta(u_\theta)_x). \tag{2.95}$$

Here $(\eta_\theta^i, u_\theta^i) \coloneqq G_\theta(\alpha_i, \beta_i)$ denotes the network output for sample i, and the subscript $_{j,n}$ indicates evaluation at the grid point (x_j, t_n). The PINO loss is a weighted sum of three terms:

$$\mathcal{L}_{\text{PINO}}(\theta) = \lambda_{\text{pde}}\,\mathcal{L}_{\text{pde}}(\theta) + \lambda_{\text{ic}}\,\mathcal{L}_{\text{ic}}(\theta) + \lambda_{\text{data}}\,\mathcal{L}_{\text{data}}(\theta), \tag{2.96}$$

where:

- $\mathcal{L}_{\text{pde}}(\theta) = \dfrac{1}{M}\displaystyle\sum_{i=1}^{M}\frac{1}{N_x N_t}\sum_{j=0}^{N_x-1}\sum_{n=0}^{N_t}\Big[\mathcal{D}_1(\eta_\theta^i, u_\theta^i)_{j,n}^2 + \mathcal{D}_2(\eta_\theta^i, u_\theta^i)_{j,n}^2\Big]$ penalizes violation of the Boussinesq equations at each grid point, with residuals evaluated using the parameters (α_i, β_i) of each sample;
- $\mathcal{L}_{\text{ic}}(\theta) = \dfrac{1}{M}\displaystyle\sum_{i=1}^{M}\frac{1}{N_x}\sum_{j=0}^{N_x-1}\Big[(\eta_\theta(x_j,0) - \eta_0(x_j))^2 + (u_\theta(x_j,0) - u_0(x_j))^2\Big]$ enforces the shared initial condition (2.1);
- $\mathcal{L}_{\text{data}}(\theta) = \dfrac{1}{M}\displaystyle\sum_{i=1}^{M}\frac{1}{N_x N_t}\sum_{j=0}^{N_x-1}\sum_{n=0}^{N_t}\Big[(\eta_\theta^i(x_j,t_n) - \eta_i(x_j,t_n))^2 + (u_\theta^i(x_j,t_n) - u_i(x_j,t_n))^2\Big]$ measures deviation from the reference solutions $(\eta_i, u_i) \in \mathcal{D}$ (2.27);
- λ_{pde}, λ_{ic}, $\lambda_{\text{data}} > 0$ are weighting hyperparameters.

The PINO therefore generalizes the FNO by adding physics supervision: it can be trained with fewer labeled samples by relying on the PDE residual for additional guidance.

3 METHODOLOGY

This chapter aims to show how the experiments were built, on what data, and under which rules, in enough detail to enable reproduction. Three things are common to every experiment, so we settle them first with the computational setup and the reproducibility rules (Section 3.1), then the pseudospectral dataset that act both as training targets and as ground truth (Section 3.2), and the error metrics that score every model on one scale (Section 3.3). Section 3.4 then defines the six trained models. They are built as a nested sequence: each model keeps the previous one and adds a single ingredient, so a change in behaviour can be traced to that ingredient and not to a tangle of them. Section 3.5 sets up the probe of spectral bias, the behaviour they all share, run on the same Boussinesq solution and the same network as the rest of the work, and Section 3.6 collects the hyperparameters in one table.

3.1 Computational Setup and Reproducibility

The software behind these experiments was written for this work. It runs on Python 3.11 with PyTorch 2.12.0 (PASZKE et al., 2019) built against CUDA 13.0, on a single NVIDIA GeForce GTX 1660 SUPER GPU. Figures were make with Matplotlib and Plotly. We used no pre-trained weights, no third-party operator-learning library, and no external dataset, although we took much inspiration from existing work. The pseudospectral solver, the multilayer perceptron, PINNs, and each optimization loop are our own code, which is what let us instrument them for the spectral analysis and the custom visualization used in upcoming chapters. The full implementation is available at `https://github.com/samuelkutz/TCC`.

Each learned architecture stays close to its source. The Fourier Neural Operator follows Li et al. (2021) original implementation, the Physics-Informed Neural Operator follows Li et al. (2023a) too, together with its released code (LI et al., 2023b), and the Physics-Informed Neural Network follows Raissi et al. (2019), but implemented from scratch with PyTorch. Where an implementation departs from its reference, we point that out.

Reproducibility rests on two conventions. A single global seed initializes the

Python, NumPy, and PyTorch at the start of the pipeline, so a rerun reproduces the reported results.

We set the experimental spatial and temporal domains of the Boussinesq System 2.1 to be:

$$x \in [-L, L], \quad L = 60, \qquad t \in [0, T], \quad T = 30. \tag{3.1}$$

Every experiment uses the initial condition of Equation (2.1) at unit amplitude $A = 1$, so that $\eta(x, 0) = \operatorname{sech}^2(x)$ and $u(x, 0) = 0$. A unit crest sits at the centre of a domain 120 units wide. That width is deliberate. It keeps the two counter-propagating pulses of the wave-equation limit (2.10)–(2.13) away from the periodic boundary for the whole simulated horizon, so no artificial wrap-around contaminates the reference. Table 3.1 lists the constants that hold everywhere. Each of them is restated below at the point where it first matters.

Table 3.1 – Fixed simulation, dataset, and evaluation constants shared by every experiment. Unless a section says otherwise, these values hold throughout.

Parameter	**Value**
Domain and initial condition	
Spatial half-length L $(\Omega = [-L, L])$	60
Final time T $(t \in [0, T])$	30
Initial amplitude A	1
Initial condition	$\eta(x, 0) = \operatorname{sech}^2(x),\ u(x, 0) = 0$
Dataset solve	
Spatial grid points N_x	128
Time levels (127 RK4 steps)	128
Spatial spacing $\Delta x = 2L/N_x$	0.9375
Temporal spacing $\Delta t = T/127$	≈ 0.2362
Parameter sampling	
Coupled coefficients	$\alpha = \beta$
Training samples M	20
Sampling grid	linspace(0.1, 4.0, 20)
Evaluation and reproducibility	
Evaluation values $\alpha = \beta$	{0.1, 3.21, 4.2}
Evaluation resolutions N_x	{128, 256, 512}
Spectral snapshot resolution	256
Global seed	37

Source: The author (2026).

3.2 The Parametric Dataset

The reference solutions do double duty. They are the supervised targets that the data-driven models learn from, and they are the ground truth against which every model is

scored, working as a high-fidelity baseline in which the pseudospectral Runge–Kutta solver of Section 2.2 supplies it well, since we take domains big enough to consider the Boussinesq system 2.1 a periodic domain. Its spectral spatial accuracy is exactly what a dispersive wave problem needs for a good approximation(TREFETHEN, 2000; STEINMOELLER et al., 2012).

We keep the parameter family one-dimensional on purpose. Following Section 2.2.4, the initial condition is fixed and only the PDE coefficients vary, tied together as $\alpha = \beta$ so that the two-parameter Boussinesq family collapses onto a single dimesion of difficulty. For small values of the parameters, the dynamics stay close to linear. For larger values, nonlinearities bring the role of high frequencies into play. We sample parameter-solutinon pairs uniformly at $M = 20$ points,

$$\alpha_i = \beta_i \in \text{linspace}(0.1,\ 4.0,\ 20), \qquad i = 1, \ldots, 20, \tag{3.2}$$

running from the near-linear value 0.1 to a more nonlinear regime at 4.0.

Each value in (3.2) is handed to the solver on $N_x = 128$ spatial points and marched through 127 Runge–Kutta steps over $[0, T]$, giving 128 time levels. The grid spacings are

$$\Delta x = \frac{2L}{N_x} = \frac{120}{128} = 0.9375, \qquad \Delta t = \frac{T}{127} = \frac{30}{127} \approx 0.2362. \tag{3.3}$$

A solve returns the two space-time fields $\eta_i(x_j, t_n)$ and $u_i(x_j, t_n)$ on the 128×128 grid, and we collect them into the dataset $\mathcal{S}$ of Equation (2.27). The fields are stored as four channels in and two out, meaning we gather, for each sample, the iniital conditions $\eta_i(x_j, 0)$ and $u_i(x_j, 0)$ for each spatial point x_j, and the two coefficientes α_i and β_i, all in the spacetime grid:

$$X \in \mathbb{R}^{M \times 4 \times N_x \times N_t}, \qquad Y \in \mathbb{R}^{M \times 2 \times N_x \times N_t}. \tag{3.4}$$

The two output channels carry η_i and u_i.

From now on the panels report only the surface elevation η, since it carries the wave structure we care about and the velocity field behaves the same way.

Following standart practices of deep learning, we therefore map every tensor into the unit interval with a single affine, channel-wise min–max transform, computed once over the whole dataset,

$$\tilde{f} = \frac{f - f_{\min}}{f_{\max} - f_{\min} + \varepsilon}, \qquad \varepsilon = 10^{-12}, \tag{3.5}$$

where $f_{\min}$ and $f_{\max}$ are taken per channel across the sample, spatial, and temporal axes. The operators train and predict in this normalized space, and every prediction is sent

back to physical units by inverting (3.5) before any metric or residual is formed. One consequence matters for the physics-only regimes of Section 3.4.4. Because $f_{\min}$ and $f_{\max}$ come from the reference solutions, a model trained with no data-fitting term still inherits the output scale of the dataset through (3.5).

In this work "no data" means no supervised residual on the interior fields. It does not mean full independence from the reference.

3.3 Error Metrics

A comparison is only fair if every model is measured against the same reference and scales. Each method is scored against a freshly recomputed pseudospectral reference using the three parameter values for 2.1,

$$\alpha = \beta \in \{0.1,\ 3.21,\ 4.2\}, \tag{3.6}$$

one near-linear and seen in the dataset, one intermediatly nonlinear and not seen in the dataset, one strongly nonlinear and outside the dataset. The middle value 3.21 is our default whenever a single representative parameter is wanted, e.g MLPs and PINNs.

To test the discretization invariance that operators are supposed to enjoy, each trained operator is queried without retraining on grids of resolution

$$N_x \in \{128,\ 256,\ 512\}, \tag{3.7}$$

the training resolution and its two and four times refinements, chosen to also allow FFT (TREFETHEN, 2000) implementations. For each query we resample the input channels of Section 3.2 onto the finer grid and recompute the reference at the matching resolution.

Let η_{true} and η_{pred} be the reference and predicted elevation on a shared space-time grid. We report three complementary numbers.

Time-resolved relative error

At each time level t_n we take the spatial L^2 error and divide it by the norm of the reference at that instant,

$$\mathbb{E}(t_n) = \frac{\left\|\eta_{\text{pred}}(\cdot, t_n) - \eta_{\text{true}}(\cdot, t_n)\right\|_2}{\left\|\eta_{\text{true}}(\cdot, t_n)\right\|_2 + \varepsilon}, \qquad \varepsilon = 10^{-12}. \tag{3.8}$$

Read against time, $\mathbb{E}(t_n)$ shows where along the trajectory the error piles up, and its spread over all time levels is summarized by a box plot. This is the quantity behind the parameter and resolution panels of Chapter 4.

Relative spectral error

To see which spatial frequencies are wrong, we take the spatial real DFT at a fixed time and compare amplitudes mode by mode. Writing $\hat{\eta}(k, t_n)$ for the amplitude at wavenumber index k, the per-mode error is

$$\mathbb{E}_{\text{spec}}(k, t_n) = \frac{\big|\, |\hat{\eta}_{\text{pred}}(k, t_n)| - |\hat{\eta}_{\text{true}}(k, t_n)| \,\big|}{\max\big(|\hat{\eta}_{\text{true}}(k, t_n)|,\, \delta\big)}, \tag{3.9}$$

with the floor $\delta = \max(\delta_r \max_k |\hat{\eta}_{\text{true}}|,\, \delta_m)$, $\delta_r = 10^{-2}$ and $\delta_m = 10^{-3}$, which keeps the ratio finite at modes whose reference amplitude sits near zero. Averaging over modes gives one spectral number per time level, and the spectral panels show both the mode-resolved curves at $t = 0$, $t = T/2$, and $t = T$ and the time evolution of that average.

Frequency-band error during training

To watch spectral bias form as the optimizer runs, we split the spatial spectrum of the error into three contiguous bands and log each band error every 500 iterations. This ratio differs slightly from (3.9). Here we apply the spatial real DFT to the error field $\eta_{\text{pred}} - \eta_{\text{true}}$ and to the reference, average each amplitude over the time levels, and divide,

$$\mathbb{E}_{\text{band}}(k) = \frac{\big\langle \big|\widehat{(\eta_{\text{pred}} - \eta_{\text{true}})}(k, t_n)\big| \big\rangle_{t_n}}{\big\langle |\hat{\eta}_{\text{true}}(k, t_n)| \big\rangle_{t_n} + \varepsilon}, \qquad \varepsilon = 10^{-12}. \tag{3.10}$$

Transforming the error before taking magnitudes makes this metric sensitive to the phase of the error rather than to magnitudes alone. With $N_k = \lfloor N_x/2 \rfloor + 1$ retained real-DFT modes, the low, mid, and high bands cut the index range into quarters,

$$k < \tfrac{N_k}{4}, \qquad \tfrac{N_k}{4} \le k < \tfrac{N_k}{2}, \qquad k \ge \tfrac{N_k}{2}, \tag{3.11}$$

and each band error is the average of (3.10) over its modes. These curves feed the band-tracking comparison of Chapter 4. A logging detail shapes how they are read. Each operator is tracked on the first dataset sample, $\alpha = \beta = 0.1$, whereas the single-parameter MLP and PINN are tracked on their own training solution at $\alpha = \beta = 3.21$. We keep that difference of reference in mind when comparing the bands across models.

3.4 The Six Models

Four architectures give rise to six trained models. The reason the count is six and not four is that two of the architectures, the PINN and the PINO, are each run in two

regimes, with and without a supervised data term. Read as a ladder, the models add one ingredient at a time. The first is a plain regressor that knows only the data. The second wraps that same network in the governing equation. The third lifts the problem from one solution to the whole family. The fourth adds the equation to the family-wide operator. Taking them in this order isolates what the physics term contributes, what operator learning contributes, and what the two contribute together.

3.4.1 MLP: A Data-Only Regressor

The first model is a Multilayer Perceptron that fits one solution from data alone. It is the fully connected network of Section 2.3.1, mapping a space-time point to an elevation, $(x, t) \mapsto \eta_\theta(x, t)$, with two input units, one output unit, four hidden layers of 256 neurons, and hyperbolic-tangent activations. That comes to 198,401 trainable parameters. The PINN of the next model reuses this backbone unchanged, so any difference between the two is a difference of training signal and not of architecture.

Training is supervised regression, nothing more. The objective is the mean-squared error (2.28) between the network output and the pseudospectral reference at $\alpha = \beta = 3.21$, on the 128-point spatial grid, and inputs are mapped to $[-1, 1]^2$ before the forward pass. We optimize with Adam (KINGMA et al., 2014) at learning rate 10^{-3} for 15,000 iterations, taking one full-batch gradient step over the whole reference field each iteration. The budget matches the PINN exactly.

What makes this model useful is what its objective leaves out. There is no PDE-residual term and no initial-condition term, so the equation (2.1) never enters and the network sees only data. That makes it the simplest testbed for spectral bias in this work. With nothing competing against the data fit, the order in which the network resolves the spatial frequencies of the target is set by the fitting dynamics alone. We read that order off the frequency-band error (3.10)–(3.11), tracked on the network's own training solution at $\alpha = \beta = 3.21$, and tie it back to the eigenvalue mechanism through the probes of Section 3.5.

3.4.2 PINN: Adding the Equation

The second model keeps the feed-forward backbone and adds the Boussinesq residual to the loss. The PINN of Section 2.3.5 represents the solution as a single continuous map $(x, t) \mapsto (\eta_\theta, u_\theta)$, so it is tied to one parameter pair. Each network has two input units, two output units, four hidden layers of 256 neurons, and hyperbolic-tangent activations, for 198,658 trainable parameters, and it trains with Adam (KINGMA et al., 2014) at learning rate 10^{-3} for 15,000 iterations.

The composite objective (2.80) has three parts. The residual term is evaluated at 5,000 interior collocation points, resampled uniformly at random over $[-L, L] \times [0, T]$ at every iteration, and the derivatives inside the Boussinesq operators $\mathcal{D}_1, \mathcal{D}_2$ come from automatic differentiation, so they carry no discretization error. The initial-condition term is enforced at 500 equispaced points at $t = 0$. The data term, when present, is a mean-squared error against a random subsample of at most 5,000 points from the pseudospectral reference. That data term is the one ingredient we toggle, which gives two regimes:

- PINN (no data): purely physics-informed, data weight 0; the network sees only the equation and the initial condition.
- PINN (with data): hybrid, data weight 1; the network also fits the reference at the training parameter.

Both regimes train at $\alpha = \beta = 3.21$, the intermediate difficulty of (3.6), chosen in the nonlinear range where spectral bias is expected to be more pronounced (WANG et al., 2024; LIU et al., 2023). Setting the two regimes side by side isolates the effect of the physics term while the architecture and the budget stay fixed.

3.4.3 FNO: Learning the Operator

The third model changes what is being learned. Rather than one solution, the Fourier Neural Operator learns the map from parameter to solution for the whole family (3.2) in a single training, so that answering a new $\alpha = \beta$ costs one forward pass instead of a fresh optimization. The architecture is the operator of Section 2.4.2. A pointwise lifting takes the six input features at each grid point, the four channels of (3.4) together with the two normalized grid coordinates, up to a channel width of 32. Four Fourier layers follow, each combining a spectral convolution truncated to 16 modes per axis with a local 1×1 convolution and a GELU activation. A pointwise projection then returns the two output channels. The model has 2,106,146 trainable parameters, an order of magnitude more than the pointwise models. Training minimizes the supervised mean-squared error (2.90) on the normalized fields with Adam at learning rate 10^{-3} for 15,000 iterations. With only $M = 20$ samples in a batch of 256, each iteration is effectively a full-batch step.

One implementation choice drives a result in Chapter 4. Each Fourier layer applies a two-dimensional real DFT over the last two axes of the feature array, which are the spatial and the temporal axes. The operator therefore treats time as a second periodic spectral dimension and emits the entire space-time solution in one pass. This keeps the architecture uniform, and it carries the periodicity assumption discussed in Section 2.4.2.

The Boussinesq solution is not periodic in time, so the implicit wrap-around at $t = T$ excites the Gibbs phenomenon (GIBBS, 1899). The resulting boundary artifact is measurable, and it later separates the FNO from its physics-informed successor.

3.4.4 PINO: Physics on the Operator

The fourth model puts the equation onto the operator, joining the two earlier additions. The Physics-Informed Neural Operator keeps the FNO backbone of Section 3.4.3 untouched and changes only the objective, appending the Boussinesq residual (2.96) to the data loss. Evaluating that residual means differentiating the predicted space-time array, and, following the reference implementation (LI et al., 2023b), we treat the two directions differently.

Spatial derivatives are taken pseudospectrally with the wavenumbers ξ_k of (2.18): the spatial DFT of the predicted field is multiplied by $i\xi_k$ for the first derivative and by $-\xi_k^2$ for the second, then transformed back. Temporal derivatives use finite differences on the grid, second-order central in the interior and first-order one-sided at the two temporal boundaries. The mixed term u_{xxt} is formed by taking the spatial second derivative spectrally and then applying the temporal stencil. The momentum nonlinearity is assembled in the conservative form $\frac{1}{2}(u^2)_x$, matching the flux used by the pseudospectral solver. The residual is built in physical units. The normalized output and coefficients are mapped back through the inverse of (3.5) before any derivative is formed, so the constraint enforces the true Boussinesq balance and not a rescaled surrogate.

The residual differs from the data term in one more respect. The supervised loss is evaluated on the native 128×128 training grid, but the physics residual is evaluated on a finer 256×256 grid, onto which the predicted field is spectrally interpolated before the derivatives are formed. A denser residual grid places a stricter, better-resolved constraint on the high-wavenumber content, at the price of extra transforms per iteration, and it means the physics term already sees resolution 256 during training. We return to that fact when reading the zero-shot resolution behaviour in Chapter 4. The objective is the weighted sum

$$\mathcal{L}_{\text{PINO}} = \lambda_{\text{pde}}\,\mathcal{L}_{\text{pde}} + \lambda_{\text{ic}}\,\mathcal{L}_{\text{ic}} + \lambda_{\text{data}}\,\mathcal{L}_{\text{data}}, \tag{3.12}$$

with the interior residual, the initial-condition term, and the supervised term of (2.96). We weight the residual well above the other two, $\lambda_{\text{pde}} = 100$ against $\lambda_{\text{ic}} = 1$, so that the equation leads the optimization. That emphasis was needed for the residual to leave a visible mark against the operator's strong pull toward the data. As with the PINN, the data weight is the single toggle:

- PINO (with data): $\lambda_{\text{pde}} = 100$, $\lambda_{\text{ic}} = \lambda_{\text{data}} = 1$; physics and data act together, the

residual dominating the objective and the data term anchoring the solution to the reference.

- PINO (no data): $\lambda_{\text{pde}} = 100$, $\lambda_{\text{ic}} = 1$, $\lambda_{\text{data}} = 0$; the operator is guided by the equation and the initial condition alone, subject to the normalization caveat of Section 3.2.

Optimization is again Adam at learning rate 10^{-3} for 15,000 iterations, and every other hyperparameter matches the FNO, so that any change in behaviour traces to the physics term.

3.5 Probing the Spectral-Bias Mechanism

Every model shares one behaviour, the spectral bias derived in Section 2.3.4, and its cause is worth examining directly rather than inferring from training curves. We examine it on the simplest object this work can offer, and we keep it on the Boussinesq system: a multilayer perceptron from $\mathbb{R}$ to $\mathbb{R}$ that learns one elevation profile, the pseudospectral reference at the fixed instant $t = 15$ and the intermediate parameter $\alpha = \beta = 3.21$,

$$x \longmapsto \eta_\theta(x) \approx \eta(x,\, 15), \qquad x \in [-L, L]. \tag{3.13}$$

One input, one output, one field, one instant. Nothing is introduced that the rest of the work does not already use: no auxiliary equation, no synthetic target, no second architecture. Everything the theory of Section 2.3.4 speaks of, a target sampled on a regular periodic grid and fit by gradient descent, is present, and nothing else is.

Two choices deserve a word. Fixing the time removes the temporal axis from the problem, so the Fourier analysis of the error is a plain spatial transform on the $N_x = 128$ grid and the wavenumbers of the theory are the wavenumbers of the figure, with no averaging in between. Fitting from data alone, with no residual and no initial-condition term, leaves the order in which frequencies are resolved to the fitting dynamics; adding the equation would mix the kernel of the data term with the kernel of the differential operator and blur the very quantity we want to isolate. The instant $t = 15$ sits at the middle of the horizon, late enough for the nonlinearity to have sharpened the crest and spread its energy over a broad band of wavenumbers, which is what makes the profile a demanding target rather than a smooth one.

3.5.1 The Probe Setting

We train three networks, at widths $N_{\text{MLP}} \in \{8, 64, 512\}$, each with four hidden layers, hyperbolic-tangent activations, one input unit, one output unit, and the spatial

coordinate mapped to $[-1, 1]$ before the forward pass. Every figure in this section comes from those same three runs.

Because the target lives on a single grid of $N_x = 128$ points, the Jacobian $J \in \mathbb{R}^{N_x \times N_\theta}$ is cheap enough to assemble explicitly, one backward pass per grid point, at every width including the widest. That matters, because the empirical kernel

$$K_0 = J_0 J_0^\mathsf{T} = V \Lambda V^\mathsf{T} \tag{3.14}$$

has to be eigendecomposed, not merely applied, and it is only 128×128. It is also cheap enough to rebuild *during* training, which is what lets one run answer both questions at once: whether the predictions hold, and whether the kernel that makes them stays put. Every run is carried out in double precision so the small eigenvalues survive the decomposition, and eigenvalues below $10^{-13}\lambda_{\max}$ are discarded as numerical noise rather than reported.

Training is full-batch gradient descent on the sum-of-squares loss $\frac{1}{2}\|e\|^2$, so that a single step is exactly

$$\theta \leftarrow \theta - \mu\, J^\mathsf{T} e, \tag{3.15}$$

the update behind (2.60), with $e = \eta_\theta - \eta_{\text{true}}$ measured on the grid. The learning rate is $\mu = 1/\lambda_{\max}$, with $\lambda_{\max}$ the largest eigenvalue of K_0 at that width. That value sits inside the stability bound $\mu < 2/\lambda_{\max}$ of Section 2.3.4, so under the linearized dynamics every eigendirection contracts monotonically, and it is the largest such step, which resolves as many eigendirections as the run can reach. We iterate 20,000 times.

One caveat has to be stated plainly, because it is what separates this probe from a construction that could only confirm itself. The MLP is nonlinear in its parameters, so its Jacobian is *not* constant: J drifts away from J_0 as training proceeds, and the frozen-kernel prediction $e^{(n)} = (I - \mu K_0)^n e_0$ of (2.61) is an approximation here, not an identity. Whatever agreement we measure is therefore a statement about a real network fitting a real Boussinesq profile, and any departure from it is itself a result. With that understood, the three predictions of the theory become testable one at a time:

- Error decay $e(\tau)$. We project the measured error onto each eigenvector v_k of K_0, giving $c_k^{(n)} = v_k^\mathsf{T} e^{(n)}$, and compare its amplitude against the predicted $|c_k(0)|\,|1 - \mu\lambda_k|^n$ of (2.61). Because the decay times $1/(\mu\lambda_k)$ span several decades, the tracked eigendirections are chosen to spread over the timescales the run can resolve, and the iteration axis is logarithmic.

- Spectral decay $\hat{e}(\tau)$. We take the real DFT of the error profile at each iteration, giving one amplitude per wavenumber, and compare the measured decay against

the prediction of (2.69). The wavenumbers are fixed at physical values, so the panel reads the frequency principle literally.

- Per-mode iteration count n_k. For each eigendirection we record the iteration at which $|c_k^{(n)}|$ first drops below a precision ε and compare it against the closed form $n_k = \lceil \ln(|c_k(0)|/\varepsilon)/(\mu\lambda_k) \rceil$ of (2.73). We set $\varepsilon = 0.05 \max_k |c_k(0)|$, a fraction of the largest initial coefficient rather than an absolute number, so the threshold scales with the amplitude of η instead of with the units it is measured in. The closed form carries no width, so the three networks should fall on one diagonal; both axes are logarithmic, since the counts span decades.

The first two panels are drawn at the widest network, where the frozen-kernel picture has the best claim to hold; the third collects all three widths.

3.5.2 Kernel Drift and the Eigenvalue Spectrum

The predictions above assume the kernel stays put, and we measure that during the very runs whose predictions we are testing, rather than in a separate experiment. Every 500 iterations we log three things: the relative parameter drift $\|\theta - \theta_0\|/\|\theta_0\|$, the relative kernel drift $\|K - K_0\|/\|K_0\|$, and the eigenvalue spectrum of K, recorded at initialization and again at the end of training.

The first two ask whether the kernel stays put, as lazy training assumes, and whether it does so more convincingly as the network widens, which is what the frozen-kernel approximation (2.49) predicts. Measuring them along the same trajectory that produces the decay curves means any departure seen there can be read against the drift that caused it, instead of against a drift measured under some other optimizer. The third exposes the eigenvalue disparity that (2.63) names as the cause of spectral bias. Together with the three checks of Section 3.5.1, these curves open the results of Chapter 4, where they anchor the reading of the spectral-bias curves measured on the full models.

3.6 Configuration Summary

Table 3.2 gathers the configuration of every experiment. The four learned architectures share a common budget, 15,000 iterations and, for the Adam-optimized models, learning rate 10^{-3}, so the comparison of Chapter 4 reflects the architecture and the training regime rather than the effort spent on tuning. The last row reports the measured training wall-clock on the hardware of Section 3.1, which is indicative and depends on the GPU used.

Table 3.2 – Configuration of the four training architectures and of the spectral-bias probe. The FNO and PINO share the operator backbone and its parameter count; the MLP and PINN share the feed-forward backbone and are single-parameter models. All Adam-trained models use 15,000 iterations. The probe is a separate, deliberately minimal network, $\mathbb{R} \to \mathbb{R}$ on the single profile $\eta(x, 15)$, trained once per width.

Setting	**MLP**	**PINN**	**FNO**	**PINO**	**Bias probe**
Input → output	$\mathbb{R}^2 \to \mathbb{R}$	$\mathbb{R}^2 \to \mathbb{R}^2$	operator	operator	$\mathbb{R} \to \mathbb{R}$
Trainable parameters	198,401	198,658	2,106,146	2,106,146	by width
Hidden layers	4	4	4 Fourier	4 Fourier	4
Width / channels	256	256	32	32	$8, 64, 512$
Fourier modes (per axis)	—	—	16	16	—
Optimizer	Adam	Adam	Adam	Adam	GD
Learning rate	10^{-3}	10^{-3}	10^{-3}	10^{-3}	10^{-4}
Iterations	15,000	15,000	15,000	15,000	20,000
Parameters trained	single (3.21)	single (3.21)	all 20	all 20	single (3.21)
Collocation / batch	full batch	5,000 pts	batch 256	batch 256	128 pts (full)
Loss weights pde/ic/data	$-/-/1$	$1/1/\{1, 0\}$	$-/-/1$	$100/1/\{1, 0\}$	$-/-/1$
Physics-residual grid	—	5,000 pts	—	256×256	—
Target	$\eta(x, t)$	η, u	family	family	$\eta(x, 15)$
Data regimes	data only	with / without	supervised	with / without	data only
Training wall-clock	≈ 155 s	≈ 880 s	≈ 1160 s	≈ 4200–5200 s	—

Source: The author (2026).

4 NUMERICAL RESULTS

This chapter reports what the experiments of Chapter 3 produced. Section 4.1 accounts for what each model costs to train. Section 4.2 establishes the spectral-bias mechanism on the kernel of the data-only MLP fitting the Boussinesq elevation. Section 4.3 then follows the error through the frequency bands as training proceeds. Section 4.4 fixes a common parameter and compares the final spectra, and Section 4.5 isolates the temporal artifact that the physics term corrects. Two stress tests push the operators off their training conditions in Section 4.6, and Section 4.7 gathers the findings. The pseudospectral solver is the ground truth throughout, and each model turns out to fail in its own characteristic way.

4.1 Training Cost

Before asking how well the models perform, we ask what they cost. Table 4.1 lists the size, wall-clock training time, and final training loss of the six trained models. Two quantities set the models apart. In capacity, the operators carry about 2.1 million parameters, roughly ten times the $\sim$200,000 of the pointwise MLP and PINN. In training time, the gap is wide: the MLP finishes in under three minutes, the plain FNO and the two PINNs in fifteen to twenty, and the PINO in well over an hour. The PINO is slow because it assembles its physics residual on the finer 256×256 grid (Section 3.4.4) at every iteration.

That extra hour of training is what makes the operators general. A single FNO or PINO covers the entire 20-point family (3.2) from one training, whereas each MLP and PINN is fixed to $\alpha = \beta = 3.21$ and must be retrained from scratch for any other value. Spread over the family, the operators are the cheaper route to a solution at a new parameter, and the FNO especially so, since inference there is a single forward pass.

One caution belongs up front, because the final-loss column is not a ranking. The losses do not share a scale, as each objective sums a different subset of data, residual, and initial-condition terms. The PINO with data reaches a larger final loss (8.1×10^{-5}) than the PINO without (4.7×10^{-5}) because the extra supervised term adds its own residual to the total, and the sections below show that it is the more accurate of the two. The

Table 4.1 – Model size, training time on the GTX 1660 SUPER, and final training loss. The final-loss column is not comparable across rows: each objective combines data, residual and initial-condition terms differently, so its magnitude reflects the makeup of the loss rather than solution accuracy. Accuracy is measured instead by the relative-error metrics of the following sections.

Model	Regime	Parameters	Train time (s)	Final loss
MLP	data only	198,401	155.4	3.5×10^{-5}
FNO	supervised	2,106,146	1162.6	3.6×10^{-6}
PINO	with data	2,106,146	5236.2	8.1×10^{-5}
PINO	no data	2,106,146	4221.6	4.7×10^{-5}
PINN	with data	198,658	884.4	7.9×10^{-5}
PINN	no data	198,658	861.3	1.2×10^{-4}

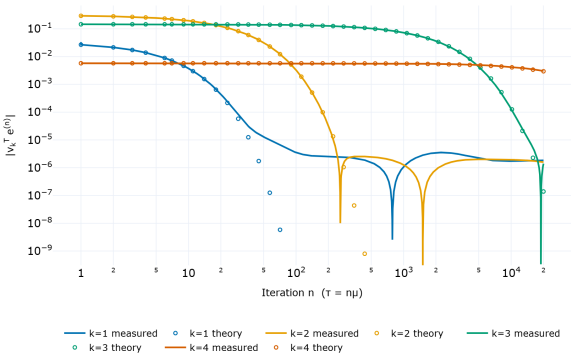
Source: The author (2026).

plain FNO carries the smallest loss in the table (3.6×10^{-6}) because its objective is a lone data term with nothing added. Training loss is a poor proxy for accuracy here. Every comparison that follows rests instead on the held-out relative-error metrics of Section 3.3.

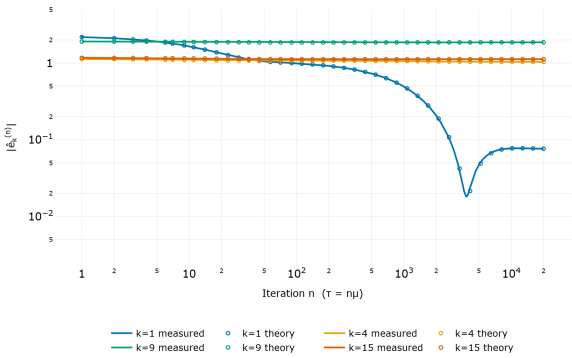
4.2 Spectral Bias, From Theory to the Network

One behaviour is common to all the models, and we establish its mechanism before comparing them. Spectral bias, derived in Section 2.3.4, predicts that gradient training resolves low frequencies before high ones at a rate set by the eigenvalues of the Neural Tangent Kernel. The probe of Section 3.5 puts that prediction on the Boussinesq system in its barest form: a network from $\mathbb{R}$ to $\mathbb{R}$ learning the single profile $\eta(x, 15)$ at $\alpha = \beta = 3.21$, at three widths, by gradient descent at the fixed step $\mu = 10^{-4}$. Because the network is nonlinear in its parameters, nothing here is true by construction. Figure 4.1 reports the four checks.

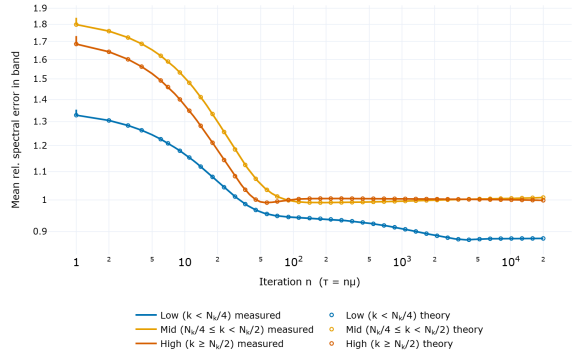
Each panel tests one prediction. In Figure 4.1a the amplitude of the error along each eigendirection follows the predicted $|1 - \mu\lambda_k|^n$ of (2.61) over four to five decades, and the ordering is unmistakable. The leading eigendirection is spent within about 70 iterations, the second within some 300, the third only in the last part of the run near 2×10^4, and the fourth has barely begun to move by the time the run ends. Each waits its turn in exact order of eigenvalue, which is the per-eigendirection ordering of (2.59) read directly off a real network. The measured curves leave the prediction only where a coefficient passes through zero and turns back up, at amplitudes near 10^{-6}, which is where the drifting Jacobian finally overtakes the frozen-kernel approximation. Figure 4.1b reads the same dynamics in Fourier space, where the frequency principle is stated as plainly as it can be: the wavenumber $k = 1$ amplitude is driven down while $k = 4$, $k = 9$ and $k = 15$



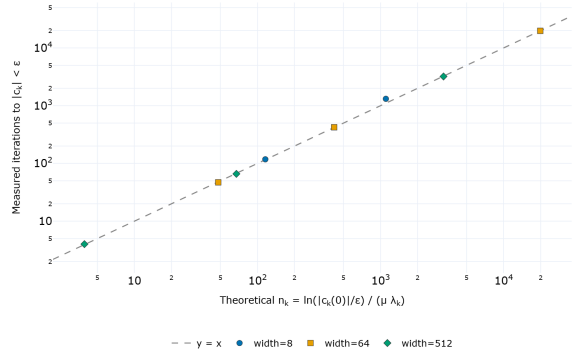
(a) Per-eigendirection error decay.



(b) Per-mode spectral decay.



(c) Band-averaged relative spectral error.



(d) Measured versus predicted iteration count n_k.

Figure 4.1 – Spectral-bias predictions measured on the Boussinesq probe of Section 3.5, an $\mathbb{R} \to \mathbb{R}$ network fitting $\eta(x, 15)$ at $\alpha = \beta = 3.21$ by full-batch gradient descent at the fixed step $\mu = 10^{-4}$. Panels (a)–(c) are the widest network, $N_{\mathrm{MLP}} = 512$; panel (d) collects all three widths. Lines are measured, open markers are the linearized prediction.

Source: The author (2026).

lie flat across the entire run, unchanged from their initial values. Figure 4.1c takes the whole spectrum rather than four sampled wavenumbers, averaging the relative error over the low, mid and high bands of (3.11). Only the low band descends and stays down, from 1.35 to 0.88. The mid and high bands fall from 1.84 and 1.73 but level off at 1.01 and 1.00 within the first hundred iterations and never move again, meaning the error at those wavenumbers remains the size of the reference itself.

Figure 4.1d makes the agreement quantitative and adds the check the other panels cannot make. The closed form $n_k = \ln(|c_k(0)|/\varepsilon)/(\mu\lambda_k)$ of (2.73) carries no width, so if it transfers, networks of 241, 12,673 and 789,505 parameters must fall on one diagonal. They do, across four decades of iteration count. In log space the measured and predicted counts correlate at 0.99999 for both width 64 and width 512, and the median ratio of measured to predicted tightens from 1.11 at width 8 to 1.00 at width 64 and 0.99 at width 512. The prediction is accurate, and it becomes more accurate as the network widens, which is what lazy training claims.

One measurement qualifies all of this, and it is the most telling result in the section. None of the three networks learns the profile. Figure 4.2 sets what they produced against what they were asked to reproduce.

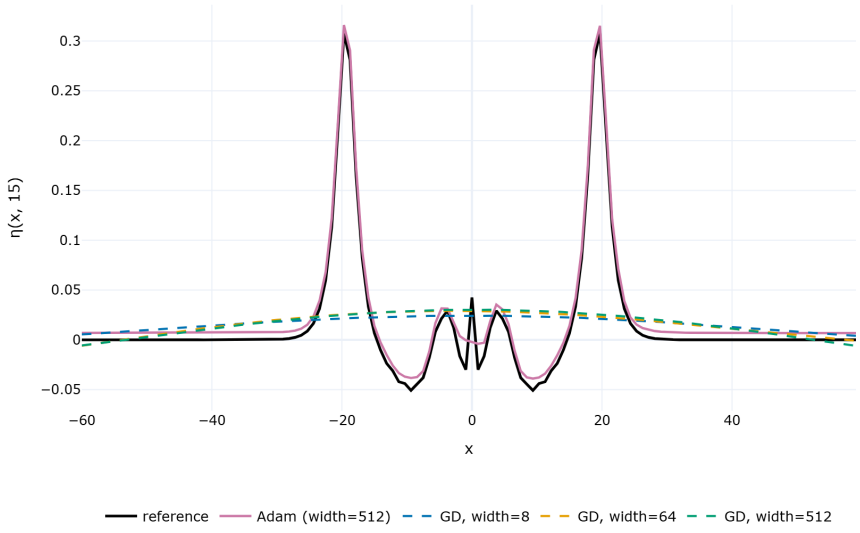


Figure 4.2 – The three gradient-descent probes against the reference profile $\eta(x, 15)$ at $\alpha = \beta = 3.21$, after 20,000 iterations, with an Adam-trained control of the same architecture and budget. Every descent run returns a nearly flat curve; the control recovers the crests. The gap between them is the practical content of spectral bias.

Source: The author (2026).

The reference carries two sharp crests near $x = \pm 20$, reaching about 0.31, with dispersive oscillations between them. All three descent runs return a nearly flat curve close to 0.025, recovering little beyond a constant offset. The error norm falls from 2.360

to 0.723 at width 8, from 0.779 to 0.721 at width 64, and from 0.798 to 0.720 at width 512, against $\|\eta(\cdot, 15)\| = 0.756$: the three runs converge to the same residual and are left with 95.7%, 95.4% and 95.3% relative error. That is also why the band curves of Figure 4.1c barely descend, and why only one of the four wavenumbers in Figure 4.1b moves.

The eigenvalues say why. At width 512 they run 1968, 444, 6.93, 0.333, 1.2×10^{-2}, 7.6×10^{-4}, 2.8×10^{-5}, and by the thirteenth mode they stop decaying altogether, flattening onto a plateau at $1.5 \times 10^{-16} \lambda_{\max}$, which is machine epsilon rather than any property of the kernel. The kernel of a smooth one-dimensional network has an effective rank of about twelve, spanning some 14 orders of magnitude, and only the first three or four of those directions are reachable in any practical number of steps. They are the smooth ones. Two independent measurements confirm that this, and not the run length, is the binding constraint. Projecting the target onto the leading eigendirections of K_0 shows that the first twelve capture only 19.4% of it and even the first forty only 28.5%, so no amount of descent inside that subspace could do much better. And repeating the run at the largest stable step, $\mu = 0.9 \times 2/\lambda_{\max}$, ten times the step used above, still ends at 95.2% relative error. The wall is set by the spectrum, not by patience.

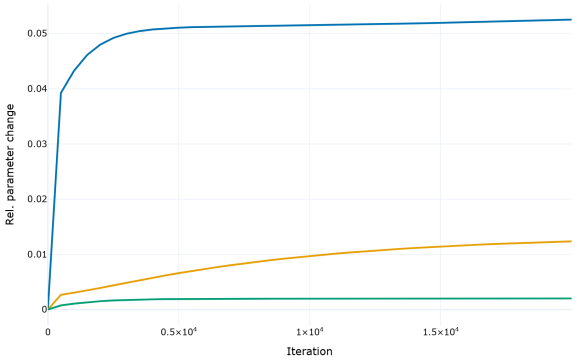
The reason is a mismatch of scales. The target is two narrow pulses in a domain 120 units wide, so once x is mapped to $[-1, 1]$ a pulse of unit width occupies a sixtieth of the input range. Its energy is correspondingly spread: the first eight wavenumbers hold only 70% of it and sixteen are needed for 92%. A dozen smooth eigenfunctions cannot carry that.

One control keeps this from being read as a statement about the architecture, and it matters enough to state outright. Trained with Adam at learning rate 10^{-3}, for the same 20,000 iterations and with the same width, hidden layers and activation, the identical network ends at 13.8% relative error and reproduces both crests, as the control curve of Figure 4.2 shows. Its error is not monotone, passing below 9.1% near the eight-thousandth iteration before settling, which is itself worth noting: the adaptive optimizer trades the orderly geometric decay of Figure 4.1a for a faster but noisier descent. The network has ample capacity for this profile. What stalls at 95% is gradient descent in the lazy regime, which is precisely the dynamics (2.61) describes and precisely the setting in which the eigenvalue disparity binds. Adam rescales each parameter direction by its own gradient statistics and is therefore not confined to the leading eigendirections at all.

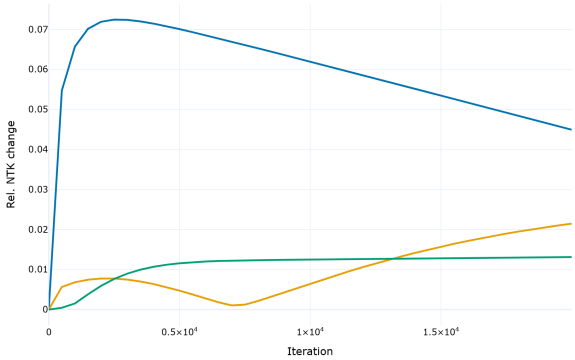
This is the sharpest statement of spectral bias in this work, and also the most carefully bounded one. The probe does not show that a multilayer perceptron cannot represent a Boussinesq profile; it shows that the optimizer the theory analyses cannot get it there, and it quantifies the gap at roughly a factor of seven in relative error. The theory predicts that gap rather than merely describing it: the same closed form that

places the crossings on the diagonal of Figure 4.1d puts the fifth eigendirection beyond 10^6 iterations and the eighth beyond 10^9. It also explains why the adaptive and spectrum-aware optimizers of Section 5.3 are the natural lever against the effect, and it sets up the question the following sections ask of the full architectures, which carry far more structure than this probe: how much of that wall does each of them move.

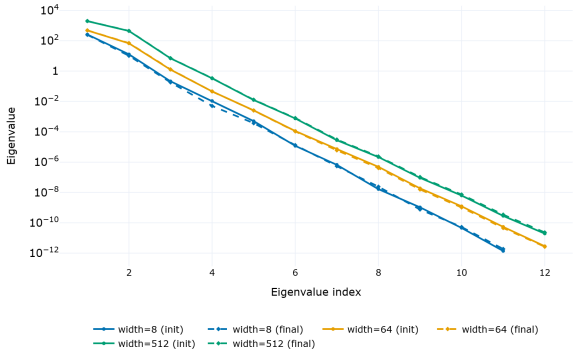
The predictions above assume the kernel stays put. Figure 4.3 reports whether it does, measured along the same three runs.



(a) Relative parameter drift $\|\theta - \theta_0\|/\|\theta_0\|$.



(b) Relative kernel drift $\|K - K_0\|/\|K_0\|$.



(c) Eigenvalue spectrum of K: initialization (solid) versus end of training (dashed), on log–log axes.

Figure 4.3 – Neural Tangent Kernel diagnostic for the Boussinesq probe at $\alpha = \beta = 3.21$, logged along the same gradient-descent runs as Figure 4.1, for widths 8, 64 and 512. Eigenvalues below $10^{-13}\lambda_{\max}$ are numerical noise and are not drawn.

Source: The author (2026).

The diagnostic supports both ingredients the theory rests on, and both shrink monotonically with width. Over the full run the parameters move by 5.3% of their initial norm at width 8, 1.2% at width 64, and 0.20% at width 512 (Figure 4.3a), a factor of twenty-six across the range. The empirical kernel follows the same ordering, drifting by 4.5%, 2.1% and 1.3% in relative norm (Figure 4.3b). Almost all of the movement in both quantities happens in the first few hundred iterations, after which the curves are

nearly flat, which is the behaviour the linearization needs. The frozen-kernel approximation (2.49) therefore sharpens with width exactly as the analysis assumes, and it does so in step with the iteration counts of Figure 4.1d, whose median ratio tightens toward unity over the same three networks. The two measurements agree on which network the theory describes best.

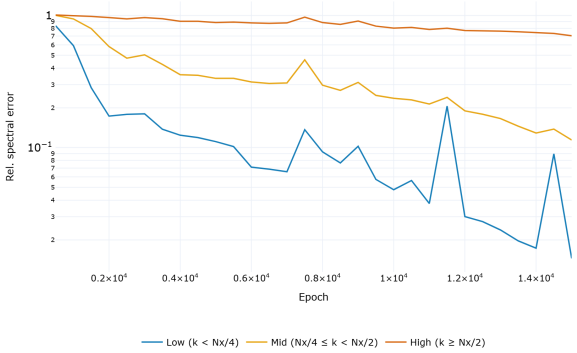
The last panel supplies the driver, and it is the one figure worth carrying into every later section. The eigenvalue spectrum falls by roughly 14 orders of magnitude within about a dozen modes at every width, from $\lambda_{\max} \approx 259$, 478 and 1968 down to the double-precision floor. The initialization and end-of-training curves lie almost on top of each other at every width, separating perceptibly only at width 8, which is the same ordering the drift panels give. By (2.63) the time to resolve an eigendirection is inversely proportional to its eigenvalue, so a spectrum this steep means that beyond the first handful of smooth directions the cost of learning becomes prohibitive, and beyond about the twelfth there is no direction left to learn at all. This is the eigenvalue disparity that Wang, Yu, et al. (2022) identify as the source of spectral bias. It sets the expectation for everything that follows: a network resolves low-frequency structure readily and stalls on high-frequency detail, and on a target as sharp as a Boussinesq pulse the stall dominates. The band-tracking curves of the next section can be read as that mechanism operating inside the full architectures, where a supervised objective, a physics residual and an operator backbone all push against it with varying success.

4.3 Learning by Frequency Band

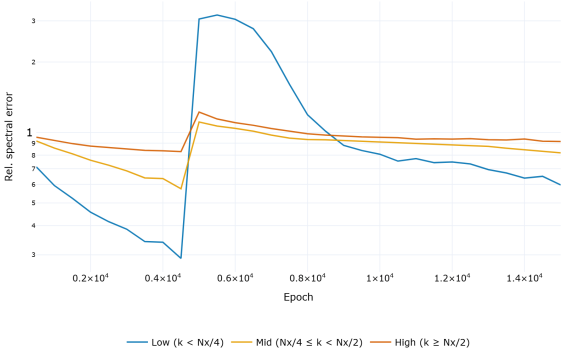
We now watch spectral bias develop as the optimizer runs. Figure 4.4 tracks the relative spectral error in the low, mid, and high bands of (3.11) across training for all six models. Within any single panel the figure tells one clean story. Across panels it carries the caveat of Section 3.3: the operators are monitored on the near-linear sample $\alpha = \beta = 0.1$, while the pointwise MLP and PINN are monitored on the harder $\alpha = \beta = 3.21$. Cross-family magnitudes are therefore not directly comparable here, and the matched comparison waits for Section 4.4, where every model is evaluated at the common parameter 3.21.

The ordering within each panel is the same in all six cases. The low band is driven lowest, the mid band trails, and the high band is resolved last and least. Every architecture and every regime obeys this ordering, which is the measured face of the NTK spectrum of Figure 4.3 and the probe of Figure 4.1. What changes from panel to panel is the size of the gap between the bands and how far each is pushed down.

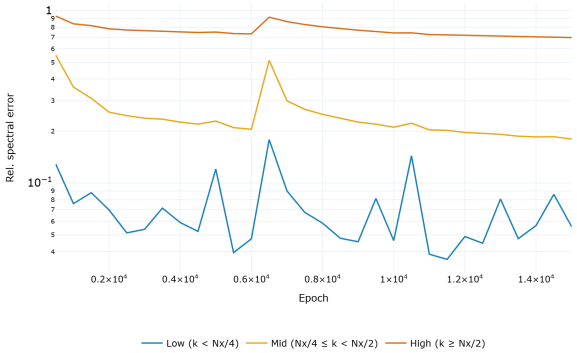
We read the MLP panel (Figure 4.4a) first, since it shows spectral bias with nothing to obscure it. With no physics term and no initial-condition term competing against the



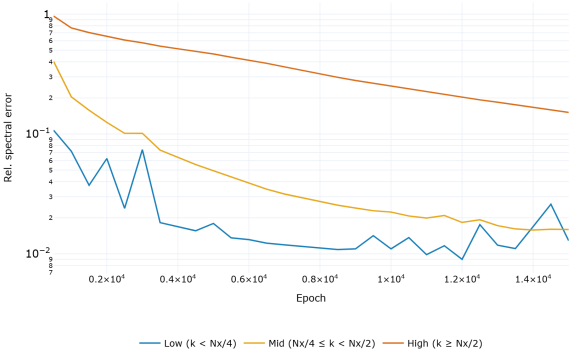
(a) MLP (data only), at $\alpha = \beta = 3.21$.



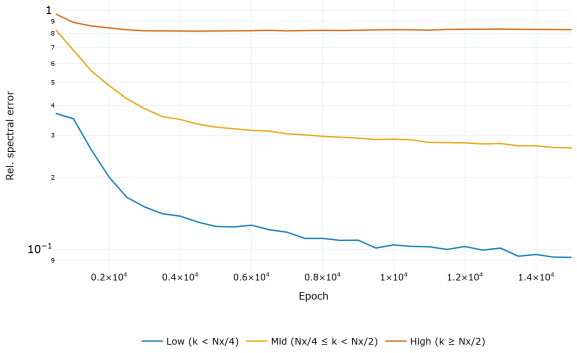
(b) PINN (no data), at $\alpha = \beta = 3.21$.



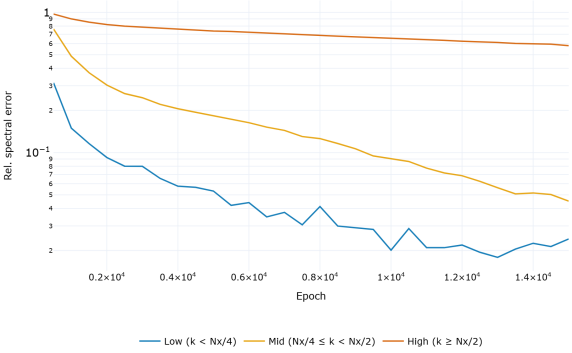
(c) PINN (with data), at $\alpha = \beta = 3.21$.



(d) FNO, at $\alpha = \beta = 0.1$.



(e) PINO (no data), at $\alpha = \beta = 0.1$.



(f) PINO (with data), at $\alpha = \beta = 0.1$.

Figure 4.4 – Relative spectral error by frequency band during training. In every panel the low band (blue) is learned first and driven lowest, the mid band (purple) trails it, and the high band (red) is learned last and least, the signature of spectral bias. Note the mixed reference samples: the pointwise MLP and PINN are tracked at $\alpha = \beta = 3.21$, the operators at $\alpha = \beta = 0.1$.

Source: The author (2026).

data fit, its ordering carries no confounders. The low band falls from 0.837 at the first logged epoch to 0.015 at the last, almost two orders of magnitude. The mid band falls from 1.004 to 0.114. The high band moves only from 1.009 to 0.704. The low frequencies are resolved and the high frequencies are left almost untouched, which is the prediction of Section 2.3.4 in its plainest empirical form. Every other panel is a variation on it, with data or physics added.

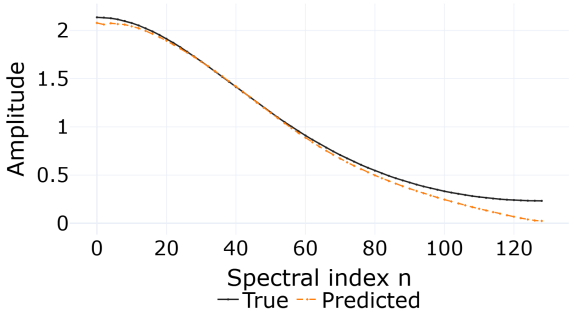
The comparisons that are valid across panels hold the architecture and the reference sample fixed and change only the data term, reading the two regimes at the same final iteration. For the PINN at 3.21, adding data lowers the terminal low-band error from 0.60 to 0.06 and the mid band from 0.82 to 0.18. The high band also improves, from 0.92 to 0.70, though it stays the worst band. For the PINO at 0.1 the effect is larger in the low and mid bands: the data term drives the low band from 0.09 to 0.02 and the mid band from 0.27 to 0.05, with a smaller gain in the high band, which falls from 0.83 to 0.58. In both families the supervised term acts mainly on the low and mid bands and little on the high band. The physics residual alone, in the two no-data panels, does not bring any band near the levels reached with data. The PINN (no data) high band barely moves over the whole run, ending at 0.92, and the PINO (no data) high band ends at 0.83.

The FNO panel, monitored at the same near-linear parameter as the PINO, reaches the lowest low- and mid-band errors in the figure (0.013 and 0.016) and the lowest high-band error too (0.151). Even so, its high band sits an order of magnitude above its low band. Read across its panels, Figure 4.4 shows that operators compress spectral bias rather than remove it. Even the lowest-error model still leaves a high-frequency residual near 15%, and every other model leaves more.

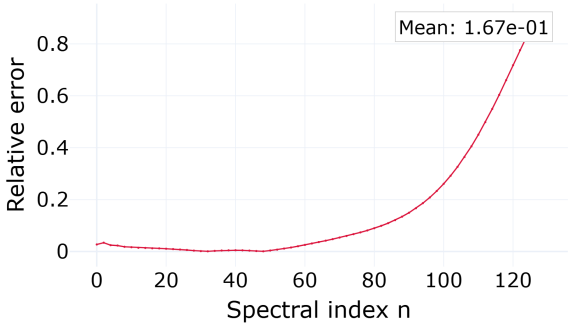
4.4 Reproducing the Spectrum

To compare the families on equal terms we fix $\alpha = \beta = 3.21$ and inspect the full spatial spectrum of the prediction at three times, $t = 0$, $t = T/2 = 15$, and $t = T = 30$. Figures 4.5–4.10 report the six models. In every panel the black curve is the reference spatial spectrum and the dashed orange curve is the prediction, with the relative-error curves and the time-averaged error at the foot of each figure.

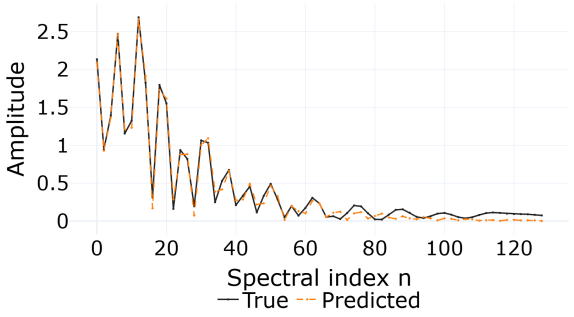
The pointwise panels show the same failure, already present in the initial state. Both PINN regimes fail to represent the high-wavenumber content of the sech^2 profile at $t = 0$, and the relative error climbs toward unity as the mode index grows. Without data (Figure 4.6) the predicted spectrum has shed most of its mid- and high-frequency amplitude by $t = 30$, and the mean spectral error reaches 0.81. Adding data (Figure 4.7) tightens the low-wavenumber crests, in step with the low-band gain of Figure 4.4, yet the aggregate mean improves only modestly ($0.17 \to 0.60 \to 0.70$). The mid and high modes



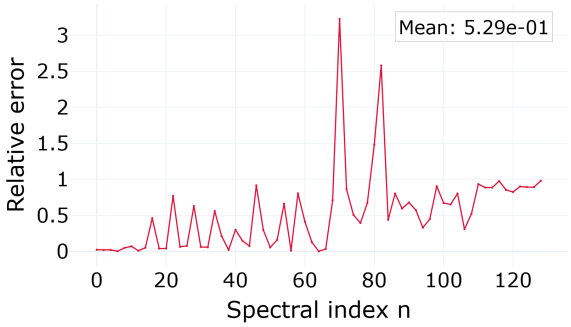
(a) Spectrum, $t = 0$.



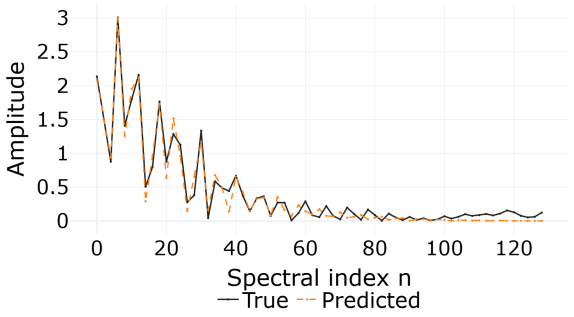
(b) Relative error, $t = 0$.



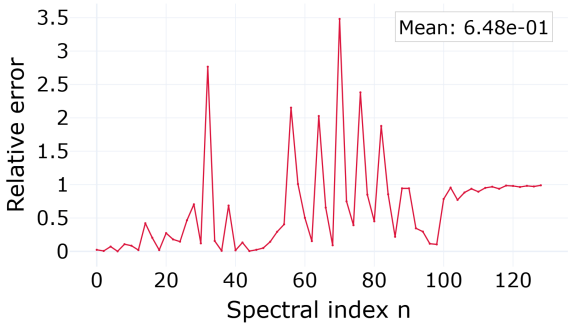
(c) Spectrum, $t = 15$.



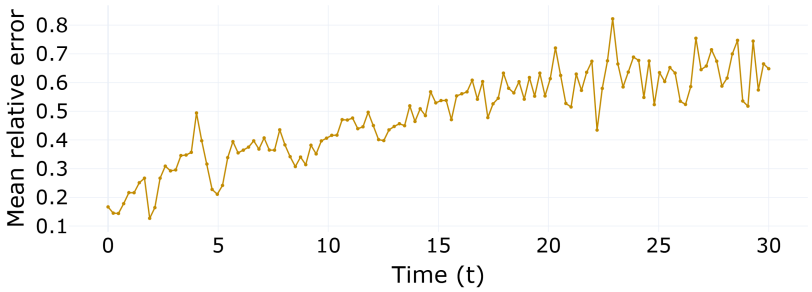
(d) Relative error, $t = 15$.



(e) Spectrum, $t = 30$.



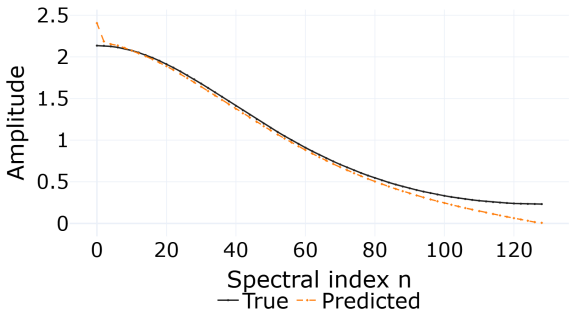
(f) Relative error, $t = 30$.



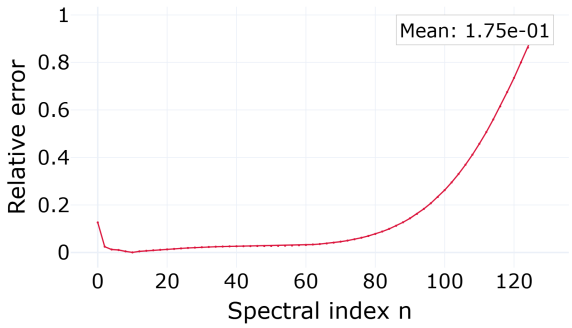
(g) Mean relative spectral error over time.

Figure 4.5 – MLP (data only) spatial spectrum at $\alpha = \beta = 3.21$. Fitting the reference with no equation and no physics term, the network reproduces the low-wavenumber crests but undershoots the tail. The relative error climbs with wavenumber, the most pronounced case of spectral bias among the six models.

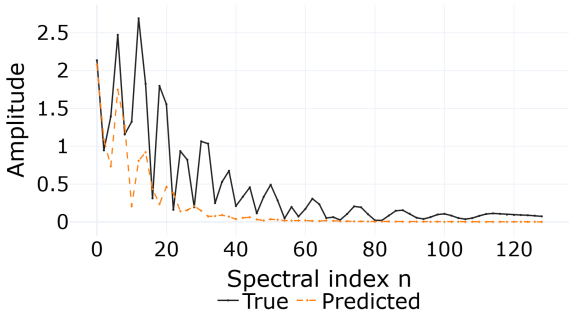
Source: The author (2026).



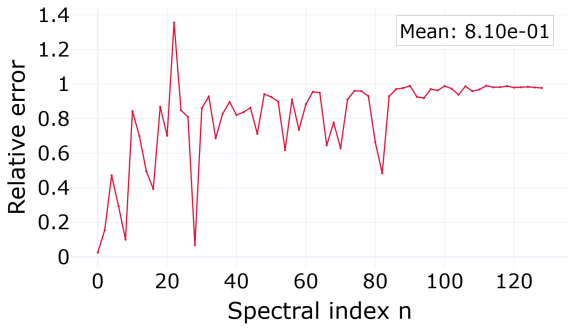
(a) Spectrum, $t = 0$.



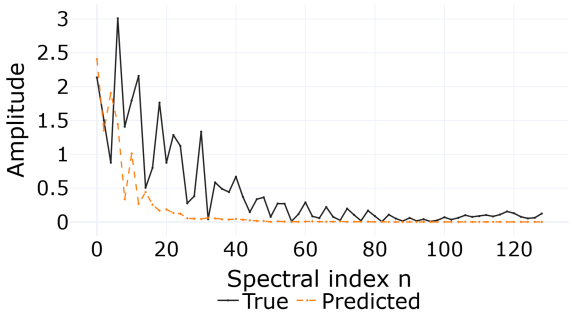
(b) Relative error, $t = 0$.



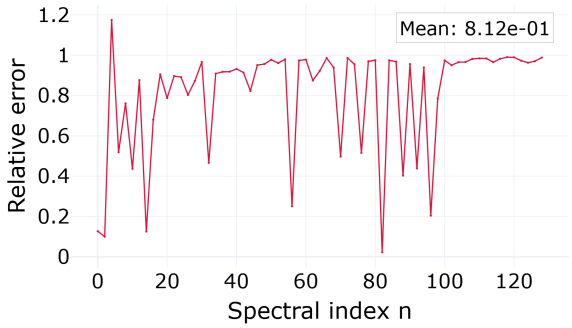
(c) Spectrum, $t = 15$.



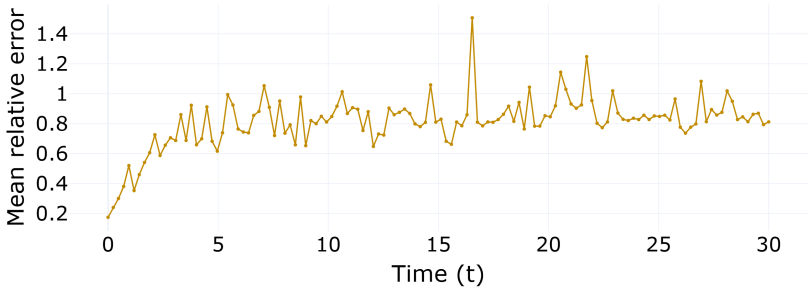
(d) Relative error, $t = 15$.



(e) Spectrum, $t = 30$.



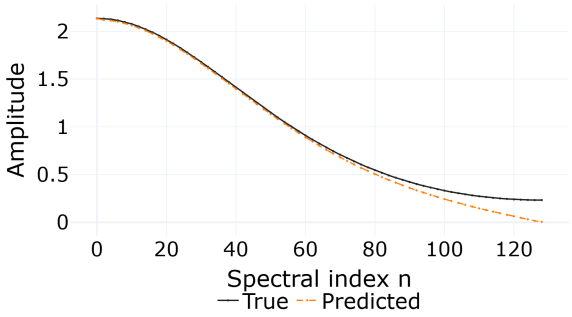
(f) Relative error, $t = 30$.



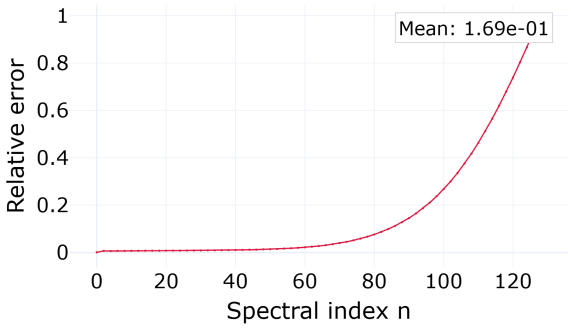
(g) Mean relative spectral error over time.

Figure 4.6 – PINN (no data) spatial spectrum at $\alpha = \beta = 3.21$. The network cannot represent the sech^2 tail even at $t = 0$, and the relative error rises monotonically with wavenumber. The mean spectral error grows from 0.18 at $t = 0$ to 0.81 at $t = 30$.

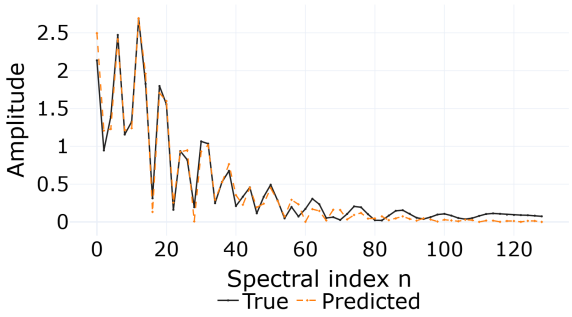
Source: The author (2026).



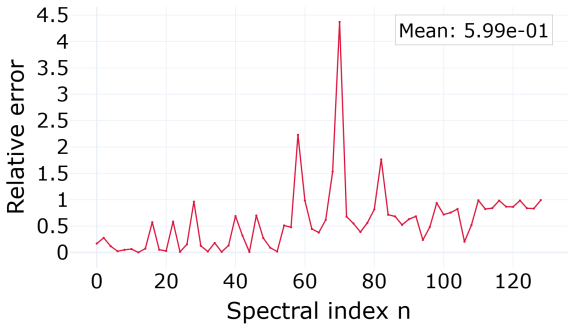
(a) Spectrum, $t = 0$.



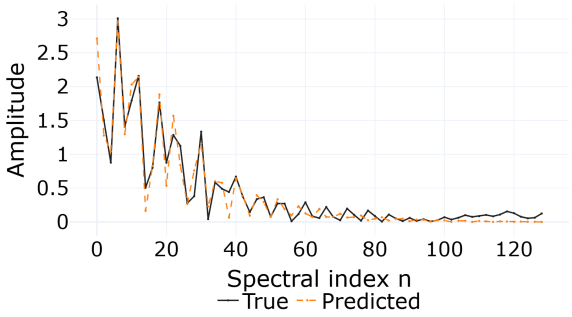
(b) Relative error, $t = 0$.



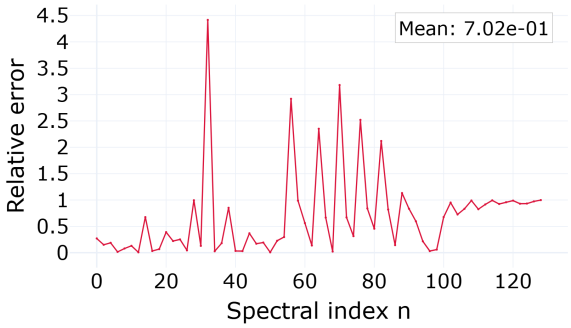
(c) Spectrum, $t = 15$.



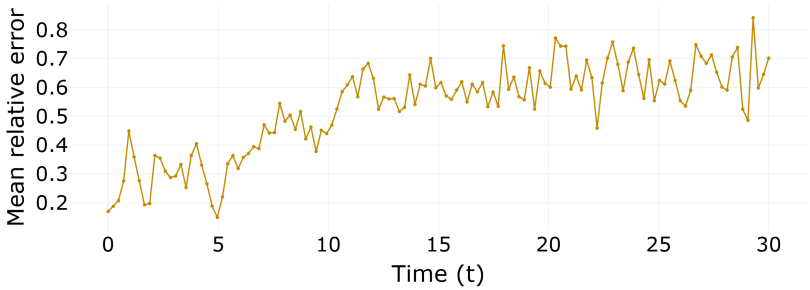
(d) Relative error, $t = 15$.



(e) Spectrum, $t = 30$.



(f) Relative error, $t = 30$.



(g) Mean relative spectral error over time.

Figure 4.7 – PINN (with data) spatial spectrum at $\alpha = \beta = 3.21$. The low-wavenumber crests are tracked more tightly than in Figure 4.6, but the high-wavenumber undershoot and the growth of the mean error in time ($0.17 \to 0.60 \to 0.70$) remain.

Source: The author (2026).

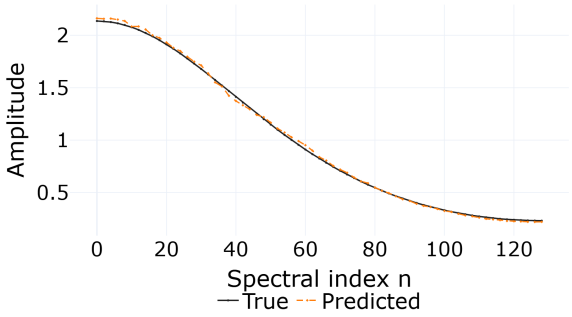
account for most of that mean, and the data term does not recover them at this hard parameter. The MLP (Figure 4.5) tells the same story with the physics stripped away. It fits the reference where the frequencies are low and undershoots where they are high, its mean spectral error rising from 0.17 at $t = 0$ to 0.53 at $t = 15$ and 0.65 at $t = 30$. Data helps the pointwise models where the kernel is favourable, but not where it is adverse.

The operators, learning in the frequency domain, invert this picture at low and mid wavenumbers. The FNO (Figure 4.8) reproduces the low and mid spectrum almost exactly at early times, a mean error of 0.022 at $t = 0$, below the pointwise models, because those modes are what its spectral layers parametrize. Its weakness is in time. The error gathers in the high modes and grows, from a mean of 0.022 at $t = 0$ to 0.36 at $t = 30$. The PINO with data (Figure 4.9) inherits the operator's spectral accuracy without this late-time blow-up. Its mean error rises to 0.20 at $t = 15$ and settles to 0.17 at $t = 30$, the smallest late-time value of the six. At $t = 30$ the PINO with data (0.17) is more accurate than the FNO (0.36), even though the FNO was ahead at $t = 0$. The physics residual trades a slightly worse start for a steadier trajectory. The PINO without data (Figure 4.10) gives up the advantage, its low- and mid-band undershoot returning and its mean error running $0.13 \rightarrow 0.23 \rightarrow 0.18$, between the operators-with-data and the PINNs. The physics constraint by itself does not replace data.

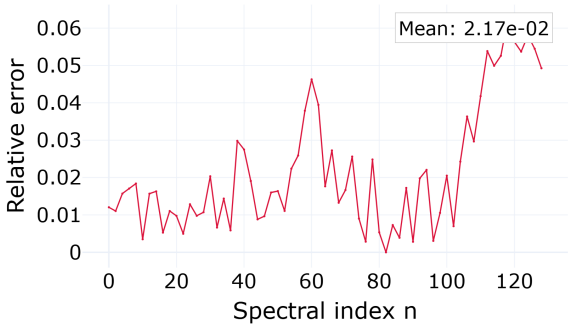
4.5 Temporal Stability from the Physics Residual

The temporal growth of the FNO error in Figure 4.8 has a specific and instructive cause, and isolating it shows where the physics term makes its most visible difference. As Section 3.4.3 explains, the operator applies the Fourier transform along the temporal axis as well as the spatial one, implicitly assuming the trajectory is periodic in time. It is not. The field at $t = T$ does not match the field at $t = 0$, so the assumption manufactures an artificial discontinuity at the temporal boundary and excites the Gibbs phenomenon there (GIBBS, 1899). Figures 4.11 and 4.12 make the effect and its correction visible.

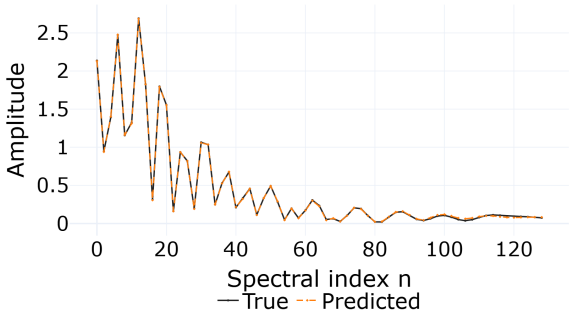
In the FNO panel (Figure 4.11) every parameter shows the same feature. After an initial transient the time-resolved error settles to a plateau and then jumps at $t = 30$ to two or three times that plateau. The spike appears at all three nonlinearities and is the dominant part of the FNO's late-time error. In the PINO panel (Figure 4.12) the predicted surfaces are visually identical, yet the spike is gone and the error rises only slightly toward the boundary. The PDE residual and the initial-condition term penalize the temporal inconsistency that a periodic transform would otherwise tolerate, damping the boundary oscillation. Suppressing the FNO's Gibbs artifact is a concrete benefit of the physics term in this study. The correction is a suppression and not a cure, since a small terminal rise remains, and it comes at the modelling cost of the residual machinery.



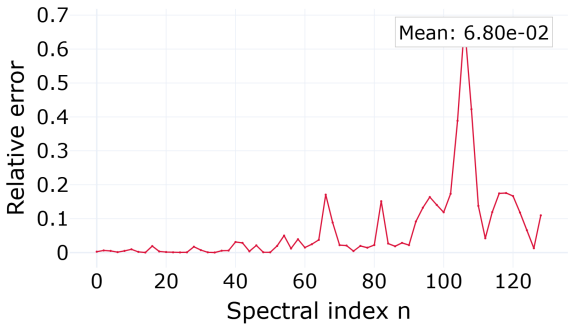
(a) Spectrum, $t = 0$.



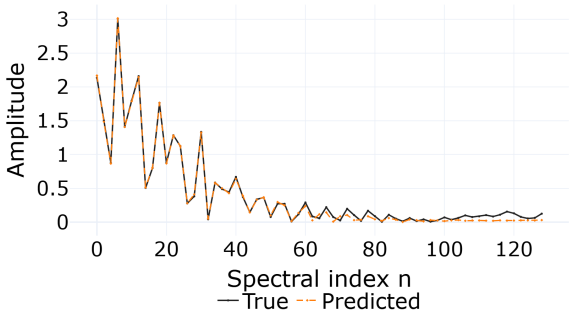
(b) Relative error, $t = 0$.



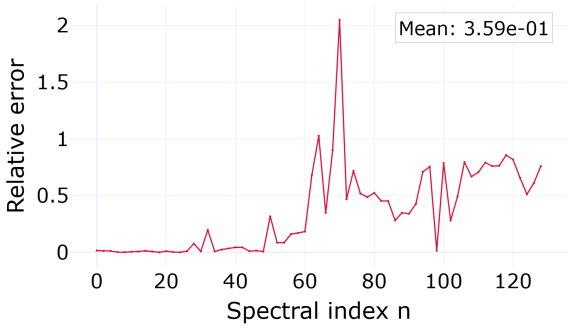
(c) Spectrum, $t = 15$.



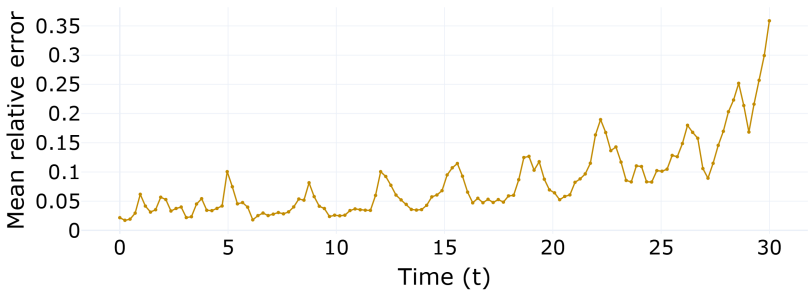
(d) Relative error, $t = 15$.



(e) Spectrum, $t = 30$.



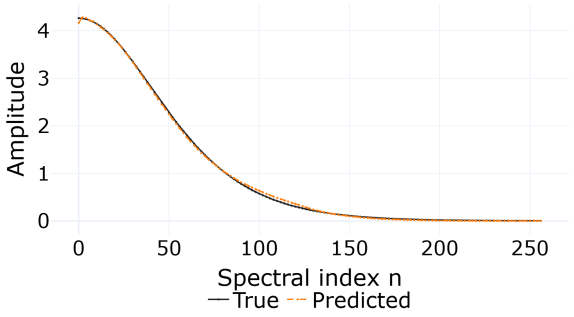
(f) Relative error, $t = 30$.



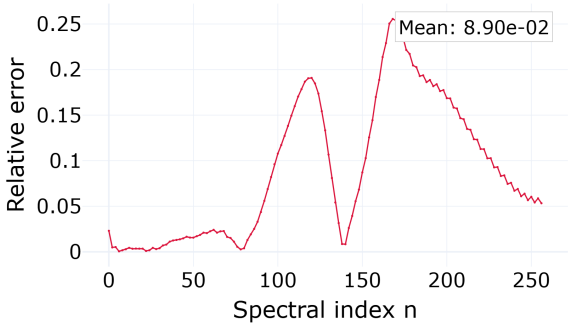
(g) Mean relative spectral error over time.

Figure 4.8 – FNO spatial spectrum at $\alpha = \beta = 3.21$. Low and mid wavenumbers are tracked almost exactly (mean error 0.022 at $t = 0$), but the error migrates to the high modes and grows in time, reaching a mean of 0.36 at $t = 30$.

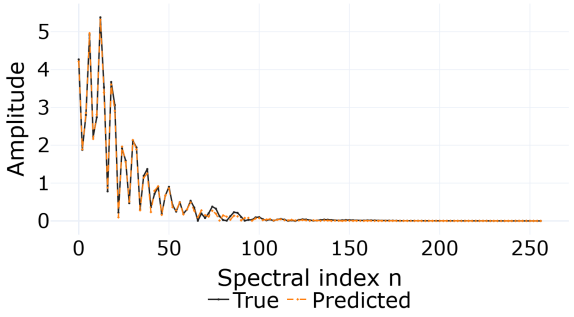
Source: The author (2026).



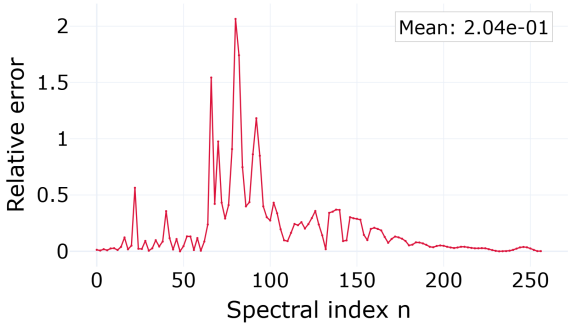
(a) Spectrum, $t = 0$.



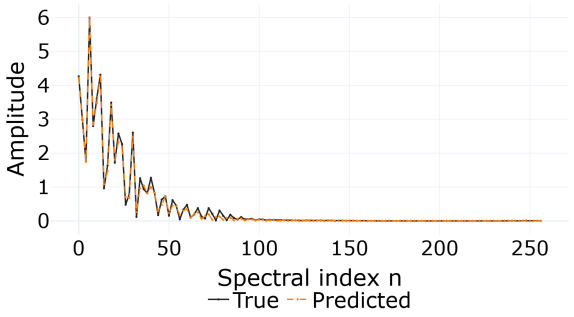
(b) Relative error, $t = 0$.



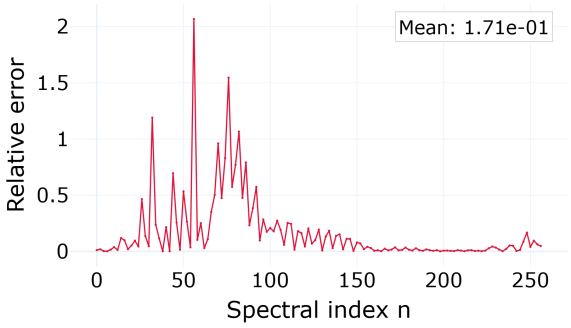
(c) Spectrum, $t = 15$.



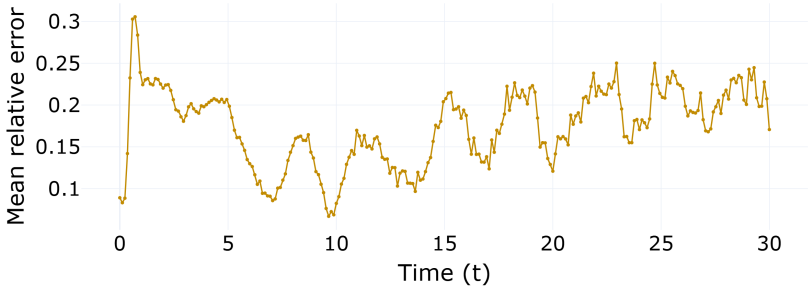
(d) Relative error, $t = 15$.



(e) Spectrum, $t = 30$.



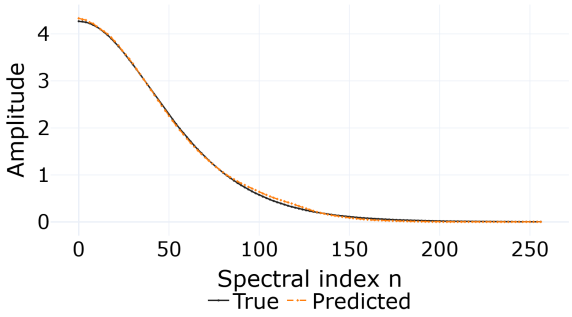
(f) Relative error, $t = 30$.



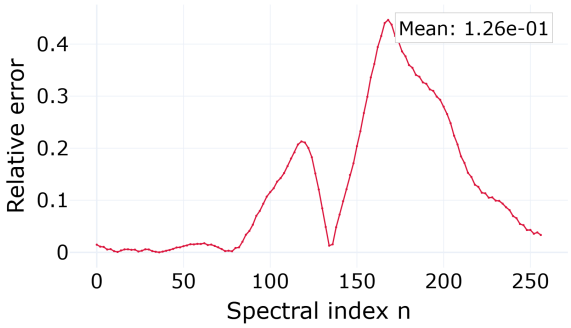
(g) Mean relative spectral error over time.

Figure 4.9 – PINO (with data) spatial spectrum at $\alpha = \beta = 3.21$. The prediction tracks the reference across the band and, unlike the FNO, the mean error does not blow up at late time. It rises to 0.20 at $t = 15$ and settles to 0.17 at $t = 30$ ($0.09 \to 0.20 \to 0.17$). The physics residual steadies the temporal evolution.

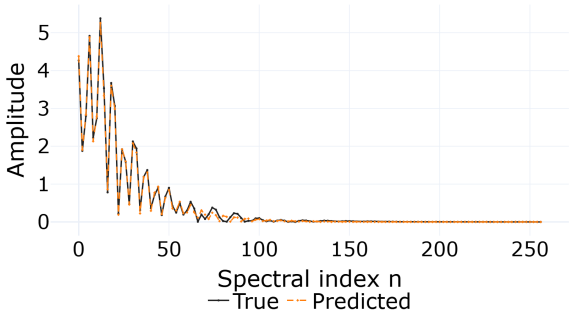
Source: The author (2026).



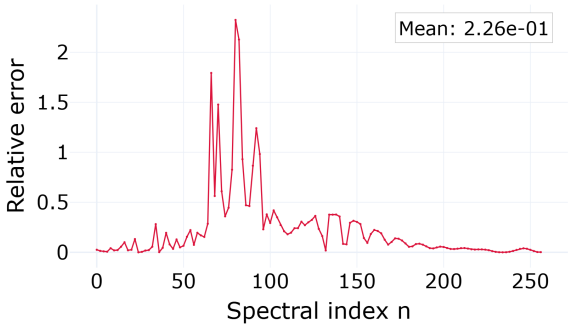
(a) Spectrum, $t = 0$.



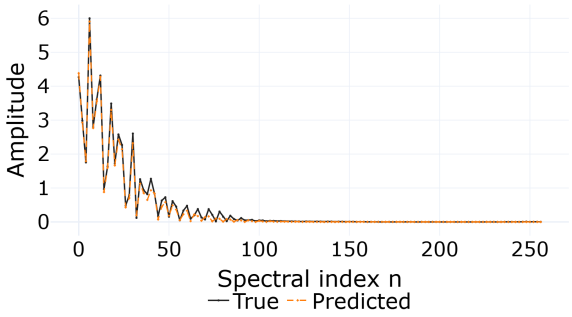
(b) Relative error, $t = 0$.



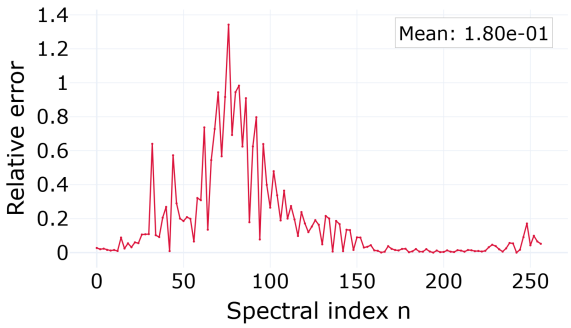
(c) Spectrum, $t = 15$.



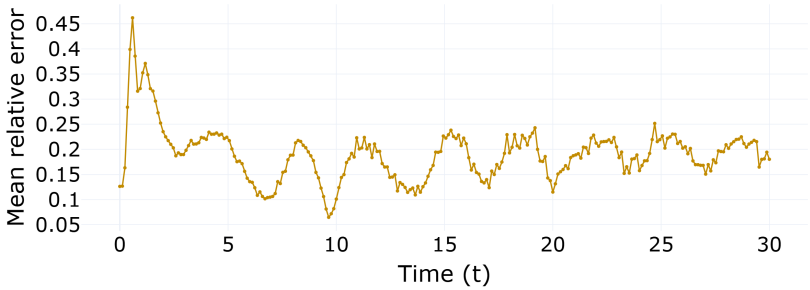
(d) Relative error, $t = 15$.



(e) Spectrum, $t = 30$.



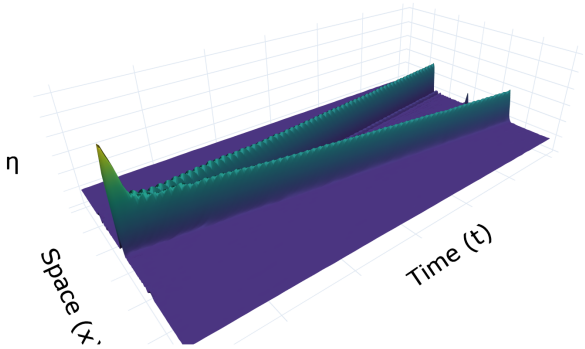
(f) Relative error, $t = 30$.



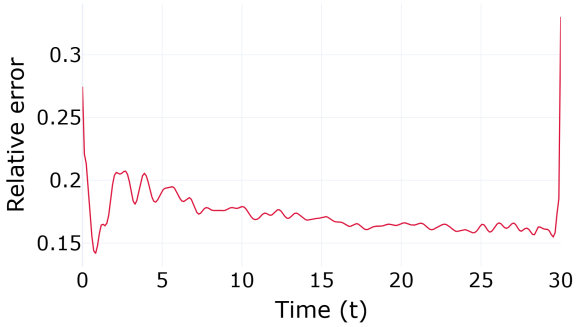
(g) Mean relative spectral error over time.

Figure 4.10 – PINO (no data) spatial spectrum at $\alpha = \beta = 3.21$. Without the supervised term the low- and mid-band undershoot returns (mean error $0.13 \rightarrow 0.23 \rightarrow 0.18$), confirming that the physics residual alone does not overcome spectral bias.

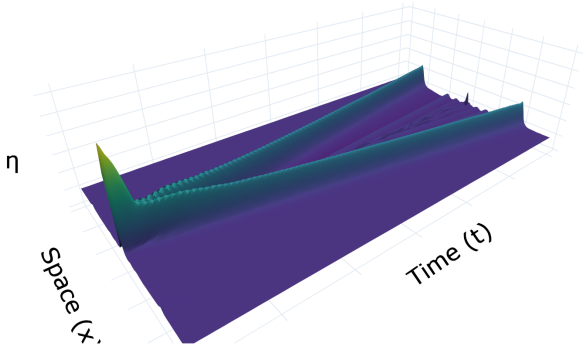
Source: The author (2026).



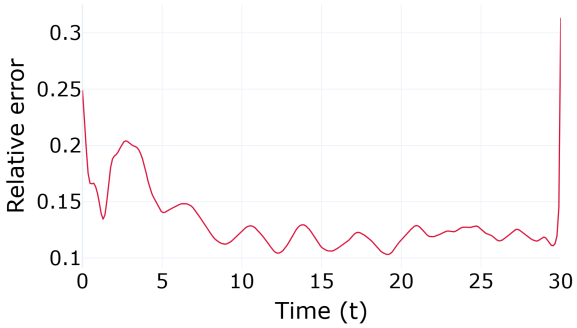
(a) Predicted $\eta(x,t)$, $\alpha = \beta = 0.1$.



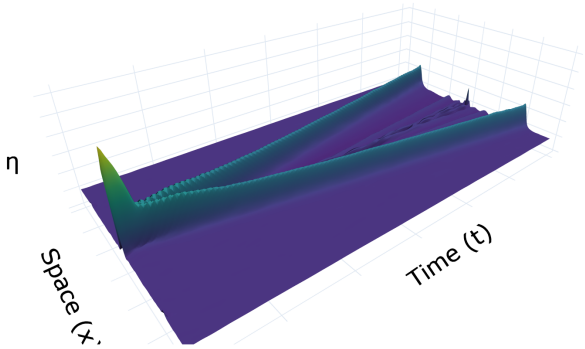
(b) Relative error, $\alpha = \beta = 0.1$.



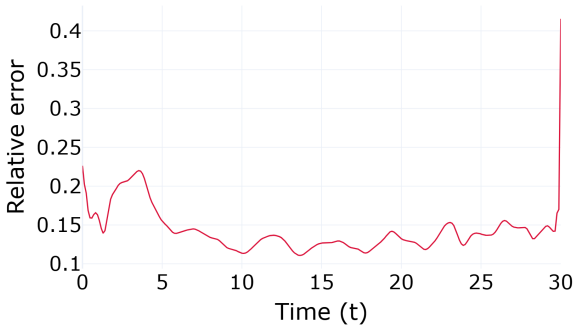
(c) Predicted $\eta(x,t)$, $\alpha = \beta = 3.21$.



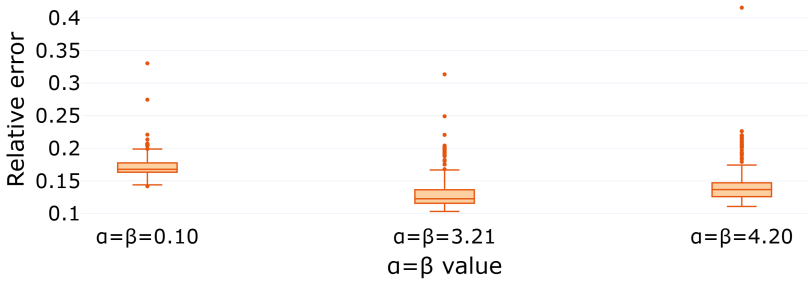
(d) Relative error, $\alpha = \beta = 3.21$.



(e) Predicted $\eta(x,t)$, $\alpha = \beta = 4.2$.



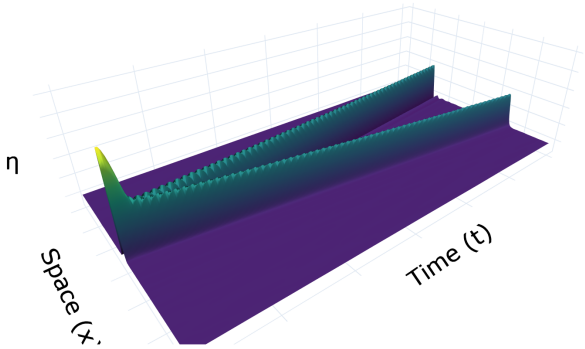
(f) Relative error, $\alpha = \beta = 4.2$.



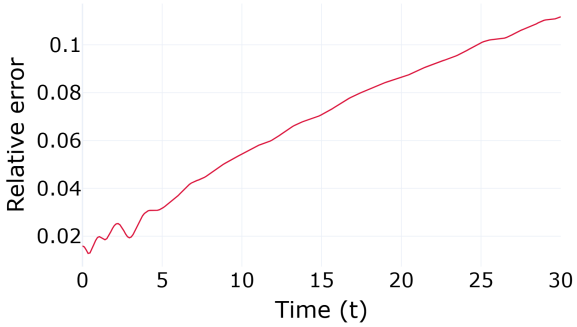
(g) Relative error distribution vs. $\alpha = \beta$.

Figure 4.11 – FNO across the nonlinearity axis ($\alpha = \beta \in \{0.1, 3.21, 4.2\}$), evaluated at resolution 256. The predicted surfaces (left) capture the two counter-propagating pulses, but the time-resolved relative error (right) spikes at the final time $t = 30$, the temporal Gibbs artifact of treating time as a periodic axis.

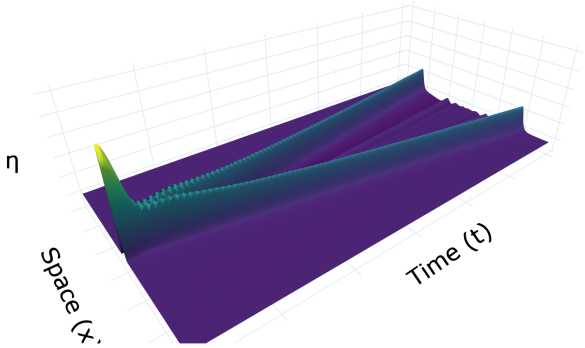
Source: The author (2026).



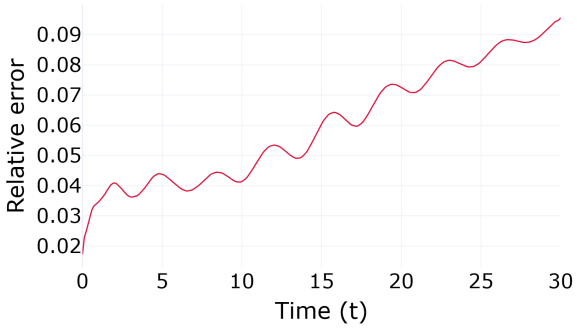
(a) Predicted $\eta(x,t)$, $\alpha = \beta = 0.1$.



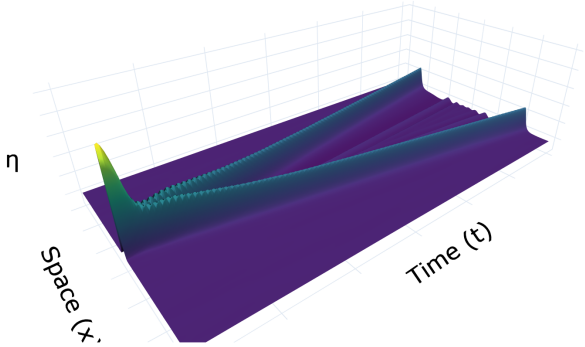
(b) Relative error, $\alpha = \beta = 0.1$.



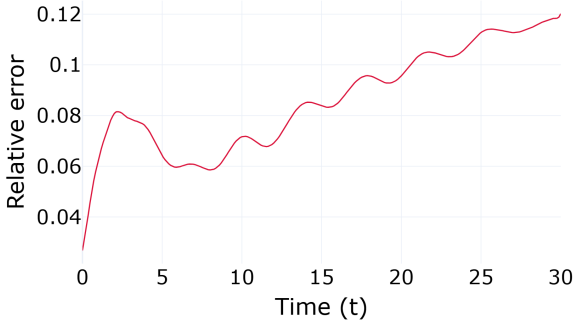
(c) Predicted $\eta(x,t)$, $\alpha = \beta = 3.21$.



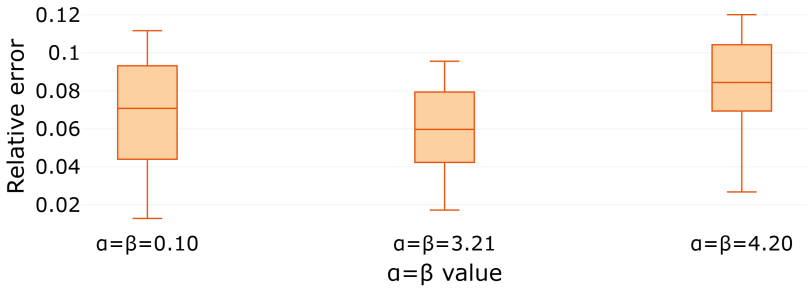
(d) Relative error, $\alpha = \beta = 3.21$.



(e) Predicted $\eta(x,t)$, $\alpha = \beta = 4.2$.



(f) Relative error, $\alpha = \beta = 4.2$.



(g) Relative error distribution vs. $\alpha = \beta$.

Figure 4.12 – PINO (with data) across the same nonlinearity axis and resolution. The predicted surfaces are visually indistinguishable from the FNO, but the terminal-time spike of Figure 4.11 is gone: the error rises only gently toward $t = 30$. The physics residual and the initial-condition term act as a temporal regularizer.

Source: The author (2026).

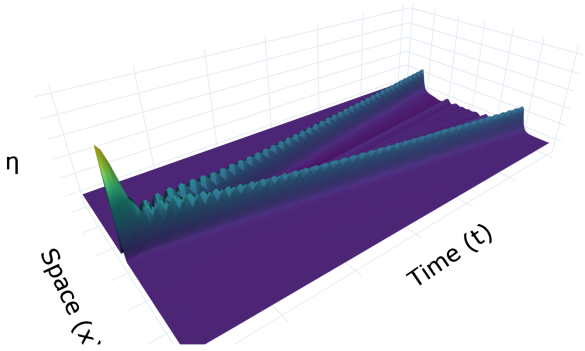
4.6 Two Stress Tests

The comparisons so far held the operators on their training conditions. Two stress tests now push them off those conditions, one along the nonlinearity axis and one along the query resolution. Both concern the operators alone, since each pointwise MLP and PINN is trained at a single parameter and a single resolution and has nothing to say about either sweep.

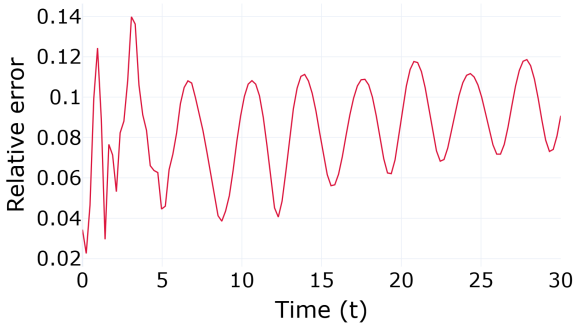
The first test sweeps the nonlinearity. The box plots at the foot of Figures 4.11 and 4.12 summarize the distribution of the time-resolved error (3.8) over the whole trajectory at each of the three parameters, and the two operators order the parameters differently. For the FNO, the smallest nonlinearity, $\alpha = \beta = 0.1$, carries the largest median error, about 0.17, against 0.12 and 0.14 at the more nonlinear 3.21 and 4.2. For the PINO with data the order reverses: the strongly nonlinear 4.2 is the hardest, with a median near 0.08, while 0.1 and 3.21 sit lower, near 0.07 and 0.06. Two effects plausibly combine to make the FNO's near-linear case its worst. First, $\alpha = \beta = 0.1$ sits at the extreme lower edge of the training grid (3.2), where the operator has the least surrounding data to interpolate from, so a boundary-of-range penalty is expected. Second, the relative metric (3.8) divides by the instantaneous norm of the reference. In the near-linear regime the solution is dominated by the two clean, slowly deforming pulses of the wave-equation limit, whose energy is concentrated and whose norm is modest, so a fixed absolute error reads as a larger relative one. In the PINO with data, the physics residual holds the low and mid modes accurate across the range, and the residual error concentrates instead at the strongly nonlinear end, where the dynamics are hardest. The two panels agree only that the relative error is non-monotone in the parameter; which end is worst depends on the model.

The second test queries the operator off its training resolution, probing the discretization invariance operators are meant to enjoy. Figure 4.13 takes the PINO with data, trained at resolution 128, and evaluates it at 128, 256, and 512 at the fixed parameter $\alpha = \beta = 3.21$ with no retraining.

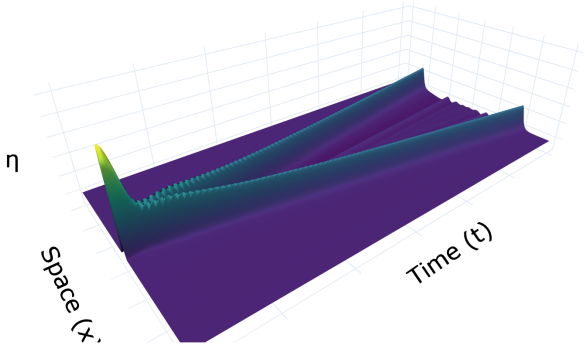
The geometric promise holds. At every resolution the operator returns a smooth, coherent surface with the correct pulse structure, with no retraining and no architectural change, since the input is simply resampled on the finer grid. The accuracy behaviour is more revealing. The box plot shows the relative error lowest at resolution 256, with a median near 0.06, and higher and comparable at 128 and 512, near 0.09 and 0.08. The minimum at 256 is not a coincidence. Recall from Section 3.4.4 that the PINO evaluates its physics residual on a 256×256 grid during training, so 256 is the resolution its physics residual actually saw. Queried at 128 the operator loses that residual-resolved detail, and queried at 512 it meets genuine high-wavenumber content that it never learned to produce,



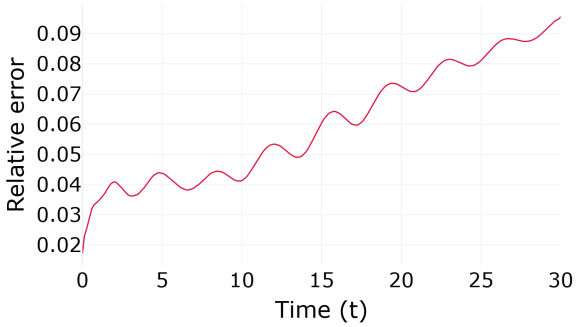
(a) Predicted $\eta(x,t)$, res. 128.



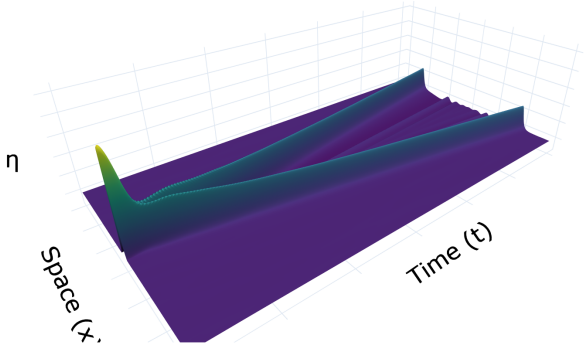
(b) Relative error, res. 128.



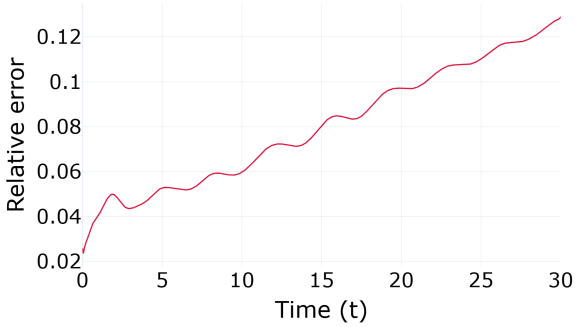
(c) Predicted $\eta(x,t)$, res. 256.



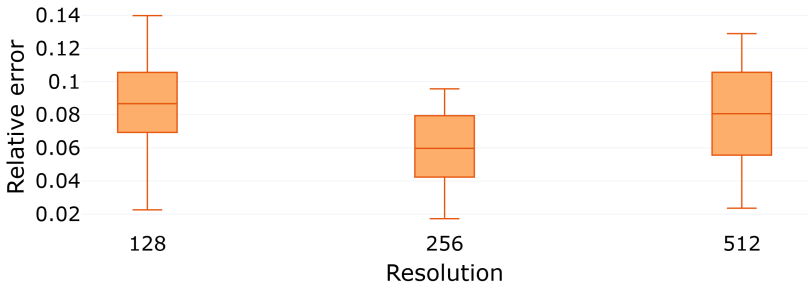
(d) Relative error, res. 256.



(e) Predicted $\eta(x,t)$, res. 512.



(f) Relative error, res. 512.



(g) Relative error distribution vs. resolution.

Figure 4.13 – PINO (with data) queried zero-shot at resolutions 128, 256 and 512 at $\alpha = \beta = 3.21$. The operator returns a coherent field at every resolution, and the relative error against the pseudospectral reference is lowest at resolution 256 (median about 0.06) and comparable at 128 and 512 (about 0.09 and 0.08): accuracy is best at the grid on which the physics residual was trained, not at the training-data grid.

Source: The author (2026).

keeping only 16 Fourier modes per axis. The plain FNO, whose only constraint is the data at resolution 128, does degrade monotonically over the same sweep, its median error climbing from about 0.02 at 128 to 0.12 at 256 and 0.17 at 512. Zero-shot superresolution is thus a geometric property, since the operator accepts and returns fields at any resolution, but not an accuracy property. Fidelity is anchored to the spectral band the training constraints resolved, and it is highest where those constraints were applied.

4.7 What the Experiments Show

Taken together, the experiments do not identify a single best method. They map where each one succeeds and where it fails.

- Spectral bias is universal, and its mechanism is measured here rather than assumed. On the Boussinesq probe of Section 4.2 the per-eigendirection decay follows the frozen-kernel prediction over some five decades, the crossing times of three networks spanning 241 to 789,505 parameters fall on one diagonal against the closed form, and the kernel spectrum drops about thirteen orders of magnitude within a dozen modes. Every architecture then obeys the same ordering, with the low wavenumbers resolved first and the high wavenumbers last (Figures 4.3, 4.1, 4.4). The probe also shows how severe the effect can be: asked for a single sharp profile, it recovers under 5% of it at every width (Figure 4.2). The data-only MLP is the plainest instance among the full models, driving its low band by nearly two orders of magnitude while its high band barely moves. No method drives the high band below roughly 0.15 relative error, and the lowest there is the plain FNO.

- Data is the main factor, and physics does not substitute for it. Within both the PINN and the PINO families, adding the supervised term reduces the low- and mid-band error, while the physics-only regimes stay close to the biased baseline (Figures 4.4, 4.9, 4.10). Physics alone neither matches data nor removes the bias.

- Operators are more accurate at low and mid wavenumbers. At the common parameter 3.21 the FNO and the PINO with data track the low and mid spectrum below either PINN or the MLP (Section 4.4), and they cover the whole family from one training, whereas each pointwise model is bound to a single parameter.

- Physics provides temporal stability. The FNO's error grows in time and spikes at the temporal boundary through the Gibbs artifact of its periodic time axis, and the PINO's residual suppresses the spike and keeps the late-time error bounded (Section 4.5). This was the main practical gain from the physics term in the study.

- The remaining limitations are real. The relative error is non-monotone in nonlinearity, with the hardest parameter differing between the two operators (Section 4.6). Zero-shot accuracy is best at the resolution the physics residual saw and worse away from it, rather than invariant (Section 4.6). And the high-frequency band is not fully resolved by any of the methods. These are consistent features of the design, not artefacts of a single run.

The picture matches the one anticipated in Chapter 1: these methods complement the pseudospectral solver that sets the accuracy ceiling here rather than replacing it. Their value is in amortizing the cost of the parameter sweep and in the temporal regularization that physics supplies, and it is least of all at the high wavenumbers where the reference solver stays unchallenged.

5 CONCLUSION

This work built a pseudospectral Boussinesq solver and three families of Scientific Machine Learning methods from first principles: Physics-Informed Neural Networks, Fourier Neural Operators, and Physics-Informed Neural Operators. It then set them against one another on a single controlled testbed. That testbed was the one-dimensional Boussinesq system with a fixed solitary initial condition and a coupled coefficient $\alpha = \beta$, swept from the near-linear to the strongly nonlinear regime, with the pseudospectral solution serving as both training reference and accuracy ceiling. One result recurs throughout: spectral bias appeared in every architecture and every regime we tried. Supervised data reduced it more than the physics residual did. The physics residual mainly provided temporal stability rather than raw accuracy, and the finest scales stayed accurate only in the classical solver. The sections below turn this finding into an assessment of each family, state what the study cannot claim, and point to the work that should come next.

5.1 Assessment of Each Method Family

The models divide into two families by how a single trained network relates to the parameter axis. Reading them family by family also answers the four questions posed in Chapter 1.

The pointwise family: MLP and PINN

These networks represent one solution as a continuous map and are fixed to a single parameter, $\alpha = \beta = 3.21$. They show spectral bias in its simplest form. The data-only MLP, with no equation and no initial-condition term competing against the data fit, drove its low band down by nearly two orders of magnitude while its high band barely moved. With nothing else in the objective, this is the bias in isolation, and it answers our question about how spectral bias manifests. Adding the equation, in the PINN, did not repair the high band. Both PINN regimes failed to represent the tail of the sech^2 spectrum even at $t = 0$, and they undershot the mid and high modes throughout. Within this family the data term mattered more than the physics residual, tightening the low-wavenumber crests, while the physics residual on its own stayed close to the biased baseline. Because

these models were run only at $\alpha = \beta = 3.21$, their behaviour as nonlinearity grows is left open, and what we can say is bounded to that one regime. Measured against the pseudospectral baseline, the pointwise family did not close the gap in the mid and high bands, and it covers only the parameter it was trained on.

The operator family: FNO and PINO

These operators learn the whole family (3.2) from a single training and answer a new parameter with one forward pass. They invert the pointwise picture at low and mid wavenumbers, tracking that part of the spectrum below either PINN or the MLP, and covering the family that the pointwise models cannot. On the role of data and physics, we can say more. Data remained the more effective intervention, since the no-data PINO gave up the low- and mid-band accuracy that the with-data PINO reached. The physics residual showed a narrower but repeatable benefit. It suppressed the temporal-boundary Gibbs spike that degrades the plain FNO at the final time, holding the late-time mean spectral error near 0.17 where the FNO climbs to 0.36. The hybrid configuration, data and physics together, gave the best accuracy in these tests, combining the operator's spectral accuracy with the residual's temporal regularization. The operators still show spectral bias at the highest wavenumbers, compressing it further than the pointwise models without removing it: no method drove the high band below roughly 0.15 relative error. One operator finding qualifies the promise of discretization invariance. Queried at finer grids, the trained PINO returned a coherent field at every resolution, so the invariance holds geometrically. Its accuracy was best at resolution 256, the grid on which its physics residual was trained, and worse both below and above it. The invariance therefore does not carry over to accuracy, because a finer grid exposes fine scales that training never resolved.

The mechanism, common to both families

The last question asked whether Neural Tangent Kernel theory genuinely predicts the frequency imbalance or only redescribes it. The probe of Chapter 4 answers that it predicts, and it does so on the Boussinesq system rather than on a model built to agree. Three networks from $\mathbb{R}$ to $\mathbb{R}$, at widths 8, 64 and 512, were fit to the single reference profile $\eta(x, 15)$ by gradient descent at a fixed step. The measured error along each kernel eigendirection followed the predicted geometric decay over four to five decades, and the eigendirections were spent in exact order of eigenvalue. The closed-form iteration count, which contains no reference to width, placed all three networks on one diagonal across four decades, correlating with the measurement at 0.99999 in log space. Both assumptions

behind that agreement held and sharpened with width: the parameters moved 5.3%, 1.2% and 0.20% of their initial norm, and the kernel 4.5%, 2.1% and 1.3%, so the widest network is the one the theory describes best, exactly as lazy training claims.

The same probe also shows how severe the consequence can be, and how precisely it can be attributed. The kernel spectrum falls about fourteen orders of magnitude within a dozen modes, leaving only three or four directions reachable in any practical number of steps, and all of them smooth. Asked for a profile made of two narrow crests, every width returned a nearly flat curve and was left with more than 95% relative error, and raising the step to the edge of stability did not change that: the wall is set by the spectrum, not by patience. What it is not is a limit of the network. The identical architecture, trained with Adam for the same budget, ends at 13.8% and recovers both crests. The failure belongs to gradient descent in the lazy regime, which is exactly the dynamics the theory analyses, and the factor of seven between those two numbers is the practical size of the effect.

That distinction is what makes the mechanism useful rather than merely descriptive. It ties the frequency-band errors of every architecture to a concrete cause rather than a loose analogy; it lets us read the compressed but persistent high-band error, in both families, as the same eigenvalue disparity at work, softened by architecture but never removed; and it identifies the lever, since an optimizer that rescales the slow eigendirections is attacking the disparity directly rather than working around it.

5.2 What This Study Cannot Claim

Several boundaries fence in these conclusions and should be weighed before carrying them further.

- One initial condition and a coupled parameter. Every experiment used a single solitary profile $\eta(x, 0) = \operatorname{sech}^2(x)$ and held $\alpha = \beta$. The operators therefore learned a one-parameter family, not the full four-parameter Boussinesq family, and the reported generalization is generalization along that single coordinate. Decoupling α and β and varying the initial data would test a substantially harder mapping.

- Pointwise models seen at one parameter. The MLP and the PINN were each trained and evaluated only at $\alpha = \beta = 3.21$. Their difficulty in the mid and high bands is a statement about that regime, not evidence of how they would behave as nonlinearity grows, which would require training them across the axis.

- Time as a spectral axis. The operator implementation applies the Fourier transform along the temporal axis, treating time as periodic, which is the direct cause of the terminal-time Gibbs artifact of Section 4.5. A time-marching or autoregressive

operator, as suggested by Li et al. (2021), would drop the periodicity assumption entirely.

- Normalization coupling in the physics-only regime. As noted in Section 3.2, the output normalization constants come from the reference dataset, so the no-data models are not strictly data-independent. They inherit the output scale of the reference. A fully data-free study would fix these constants from the initial condition alone.

- Fixed capacity and budget. Every operator kept only 16 Fourier modes per axis and trained for 15,000 iterations at a single learning rate. The persistent high-band error is consistent with this truncation, and a mode or capacity sweep would be needed to separate the architectural limit from the optimization limit.

- Relative-error normalization. The non-monotone dependence of the error on nonlinearity (Section 4.6) is partly an artefact of normalizing by the instantaneous solution norm. Absolute or energy-based metrics would round out the account of accuracy across the parameter range.

5.3 Directions Forward

The results point to several concrete continuations. The most immediate is to attack the shared failure mode directly. Spectral bias survives every regime, and its mechanism is pinned to the kernel eigenvalue disparity, so the levers that target that disparity are the ones to test on this same benchmark: adaptive loss balancing (WANG; TENG, et al., 2021) and the second-order and spectrum-aware optimizers of Khodakarami et al. (2026). A second direction is architectural. A time-marching FNO would remove the temporal Gibbs artifact by construction, isolating how much of the PINO's late-time advantage is intrinsic to the physics term rather than a repair of a periodicity assumption. A third is to broaden the problem, decoupling α and β, varying the initial data, and extending to the two-dimensional Boussinesq system, to see whether the ordering of methods found here survives a genuinely higher-dimensional solution operator. A fourth is to widen the field of comparison to DeepONet (LU et al., 2021) and to hybrid schemes that hand the high-frequency residual back to a classical corrector, which would probe whether the complementary role identified for these methods can become a solver that is at once fast and spectrally faithful.

For the nonlinear Boussinesq system, the value of these Scientific Machine Learning methods lies in amortizing a parameter sweep and, when physics is added, in gaining temporal stability. The fine scales that make dispersive waves hard remain, for now, the solver's alone.

REFERENCES

ARORA, S. et al. Fine-Grained Analysis of Optimization and Generalization for Overparameterized Two-Layer Neural Networks. In: PROCEEDINGS of the 36th International Conference on Machine Learning (ICML). [S.l.: s.n.], 2019. v. 97, p. 322–332.

BAKER, N. et al. **Workshop Report on Basic Research Needs for Scientific Machine Learning: Core Technologies for Artificial Intelligence**. Washington, DC, 2019. DOI: 10.2172/1478744.

BIHLO, A.; POPOVYCH, R. O. Physics-informed neural networks for the shallow-water equations on the sphere. **Journal of Computational Physics**, 2022. Available from: <https://arxiv.org/abs/2104.00615>.

BONA, J. L.; CHEN, M.; SAUT, J.-C. Boussinesq equations and other systems for small-amplitude long waves in nonlinear dispersive media: I. Derivation and linear theory. **Journal of Nonlinear Science**, v. 12, p. 283–318, 2002. DOI: 10.1007/s00332-002-0466-4.

BONEV, B. et al. Spherical Fourier Neural Operators: Learning Stable Dynamics on the Sphere. In: INTERNATIONAL Conference on Machine Learning (ICML). [S.l.: s.n.], 2023. Available from: <https://arxiv.org/abs/2306.03838>.

BORDELON, B.; CANATAR, A.; PEHLEVAN, C. Spectrum Dependent Learning Curves in Kernel Regression and Wide Neural Networks. In: PROCEEDINGS of the 37th International Conference on Machine Learning (ICML). [S.l.: s.n.], 2020. Available from: <https://arxiv.org/abs/2002.02561>.

BOUSSINESQ, J. Théorie des ondes et des remous qui se propagent le long d'un canal rectangulaire horizontal. **Journal de Mathématiques Pures et Appliquées**, v. 17, p. 55–108, 1872.

BRUNTON, S. L.; PROCTOR, J. L.; KUTZ, J. N. Discovering governing equations from data by sparse identification of nonlinear dynamical systems. **Proceedings of the National Academy of Sciences**, v. 113, n. 15, p. 3932–3937, 2016. DOI: 10.1073/pnas.1517384113.

BURDEN, R. L.; FAIRES, J. D. **Numerical Analysis**. 9. ed. Boston: Brooks/Cole, 2011.

CAO, Y. et al. Towards Understanding the Spectral Bias of Deep Learning. In: PROCEEDINGS of the Thirtieth International Joint Conference on Artificial Intelligence (IJCAI). [S.l.: s.n.], 2021. Available from: <https://arxiv.org/abs/1912.01198>.

CHEN, R. T. Q. et al. Neural Ordinary Differential Equations. In: ADVANCES in Neural Information Processing Systems (NeurIPS). [S.l.: s.n.], 2018. v. 31. Available from: <https://arxiv.org/abs/1806.07366>.

CHEN, T.; CHEN, H. Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems. **IEEE Transactions on Neural Networks**, v. 6, n. 4, p. 911–917, 1995.

CUOMO, S. et al. Physics-informed neural networks for data-driven simulation: Advantages, limitations, and opportunities. **Physica A: Statistical Mechanics and its Applications**, 2023. DOI: 10.1016/j.physa.2022.128415.

CYBENKO, G. Approximation by superpositions of a sigmoidal function. **Mathematics of Control, Signals and Systems**, v. 2, n. 4, p. 303–314, 1989.

DE PINA, I. et al. Investigating Steady Unconfined Groundwater Flow using Physics Informed Neural Networks. **Advances in Water Resources**, 2023. DOI: 10.1016/j.advwatres.2023.104445.

_____. Investigating Steady Unconfined Groundwater Flow Using Physics-Informed Neural Networks. **arXiv preprint arXiv:2112.13792**, 2021. Available from: <https://arxiv.org/abs/2112.13792>.

DEBNATH, L. **Nonlinear Partial Differential Equations for Scientists and Engineers**. 3. ed. New York: Birkhäuser, 2012.

GIBBS, J. Willard. Fourier's Series. **Nature**, v. 59, n. 1539, p. 606, 1899. DOI: 10.1038/059606a0.

GOLUB, G. H.; VAN LOAN, C. F. **Matrix Computations**. 4. ed. Baltimore: Johns Hopkins University Press, 2013.

GUPTA, G. et al. Multiwavelet-based Operator Learning for Differential Equations. In: ADVANCES in Neural Information Processing Systems (NeurIPS). [S.l.: s.n.], 2021. Available from: <https://arxiv.org/abs/2109.13459>.

HORNIK, K.; STINCHCOMBE, M.; WHITE, H. Multilayer feedforward networks are universal approximators. **Neural Networks**, v. 2, n. 5, p. 359–366, 1989.

JACOT, A.; GABRIEL, F.; HONGLER, C. Neural Tangent Kernel: Convergence and Generalization in Neural Networks. In: ADVANCES in Neural Information Processing Systems (NeurIPS). [S.l.: s.n.], 2018. v. 31. Available from: <https://arxiv.org/abs/1806.07572>.

JAGTAP, A. D. et al. Deep Learning of Inverse Water Waves Problems Using Multi-Fidelity Data: Application to Serre–Green–Naghdi Equations. **arXiv preprint arXiv:2202.02899**, 2022. Available from: <https://arxiv.org/abs/2202.02899>.

KARNIADAKIS, G. E. et al. Physics-informed machine learning. **Nature Reviews Physics**, v. 3, n. 6, p. 422–440, 2021. DOI: 10.1038/s42254-021-00314-5.

KHODAKARAMI, S. et al. Spectral bias in physics-informed and operator learning: analysis and mitigation guidelines. **arXiv preprint**, 2026. Available from: <https://arxiv.org/abs/2602.19265>.

KINGMA, D. P.; BA, J. Adam: A method for stochastic optimization. **arXiv preprint arXiv:1412.6980**, 2014.

KREYSZIG, E. **Introductory functional analysis with applications**. [S.l.]: John Wiley & Sons, 1978.

KRISHNAPRIYAN, A. S. et al. Characterizing Possible Failure Modes in Physics-Informed Neural Networks. In: ADVANCES in Neural Information Processing Systems (NeurIPS). [S.l.: s.n.], 2021. v. 34, p. 26548–26560.

KUMAR, S.; DHAMA, S. Physics-informed neural networks for exploring soliton collisions in the non-integrable Schamel equation in plasmas. **Physics of Fluids**, v. 37, n. 1, 2025. DOI: 10.1063/5.0301334.

LECUN, Y.; BENGIO, Y.; HINTON, G. Deep learning. **Nature**, v. 521, n. 7553, p. 436–444, 2015. DOI: 10.1038/nature14539.

LI, Z. et al. Fourier Neural Operator for parametric partial differential equations. In: INTERNATIONAL Conference on Learning Representations (ICLR). [S.l.: s.n.], 2021. Available from: <https://arxiv.org/abs/2010.08895>.

_____. Physics-Informed Neural Operator for Learning Partial Differential Equations. **ACM/IMS Transactions on Data Science**, 2023. Available from: <https://arxiv.org/abs/2111.03794>.

_____. **PINO: Physics-Informed Neural Operator – Reference Implementation**. [S.l.: s.n.], 2023. GitHub repository. Available from: <https://github.com/neuraloperator/PINO>.

LI, Z. et al. Fourier neural operator for parametric partial differential equations. **arXiv preprint arXiv:2010.08895**, 2020.

LINNAINMAA, A. Taylor expansion of the accumulated rounding error. **BIT Numerical Mathematics**, v. 10, n. 1, p. 47–55, 1970.

LIU, H. et al. Prediction of phase transition and time-varying dynamics of the (2+1)-dimensional Boussinesq equation by parameter-integrated PINNs with phase domain decomposition. **Physical Review E**, v. 108, n. 4, p. 045303, 2023. DOI: 10.1103/PhysRevE.108.045303.

LKHAGVASUREN, B.; CHOI, B.-J.; JIN, H. S. Applications of the Fourier neural operator in a regional ocean modeling and prediction. **Frontiers in Marine Science**, v. 11, 2024. DOI: 10.3389/fmars.2024.1383997.

LU, L. et al. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. **Nature Machine Intelligence**, v. 3, p. 218–229, 2021. DOI: 10.1038/s42256-021-00302-5.

MARTINS, L. G. **Estudo Numérico do Modelo Boussinesq com Topografia e Pressão Variável no Espaço e no Tempo**. 2023. MA thesis – Universidade Federal do Paraná, Curitiba.

MARTINS, L. G.; FLAMARION, M. V.; RIBEIRO-JR, R. A forced Boussinesq model with a sponge layer. **Partial Differential Equations in Applied Mathematics**, v. 10, p. 100661, 2024. DOI: 10.1016/j.padiff.2024.100661.

MCCULLOCH, W. S.; PITTS, W. A logical calculus of the ideas immanent in nervous activity. **Bulletin of Mathematical Biophysics**, v. 5, n. 4, p. 115–133, 1943.

NOCEDAL, J. Updating quasi-Newton matrices with limited storage. **Mathematics of Computation**, v. 35, n. 151, p. 773–782, 1980.

PARIKH, N.; BOYD, S. Proximal Algorithms. **Foundations and Trends in Optimization**, v. 1, n. 3, p. 123–231, 2013. DOI: 10.1561/2400000003.

PASZKE, A. et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In: ANNUAL Conference on Neural Information Processing Systems (NeurIPS). Vancouver: [s.n.], 2019. p. 8024–8035. Available from: <https://arxiv.org/abs/1912.01703>.

PATEL, A. et al. A Neural Operator-Based Emulator for Regional Shallow Water Dynamics. **arXiv preprint**, 2025. Available from: <https://arxiv.org/abs/2502.14782>.

PEI, K. et al. Data-driven soliton mappings for integrable fractional nonlinear wave equations via deep learning with Fourier neural operator. **Chaos, Solitons & Fractals**, v. 165, 2022. DOI: 10.1016/j.chaos.2022.112848.

PEREGRINE, D. H. Long waves on a beach. **Journal of Fluid Mechanics**, v. 27, n. 4, p. 815–827, 1967. DOI: 10.1017/S0022112067002605.

RACKAUCKAS, C. et al. Universal Differential Equations for Scientific Machine Learning. **arXiv preprint arXiv:2001.04385**, 2020.

RAHAMAN, N. et al. On the spectral bias of neural networks. In: PMLR. INTERNATIONAL Conference on Machine Learning (ICML). Long Beach: [s.n.], 2019. p. 5301–5310.

RAISSI, M.; PERDIKARIS, P.; KARNIADAKIS, G. E. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. **Journal of Computational Physics**, v. 378, p. 686–707, 2019.

ROSENBLATT, F. The perceptron: a probabilistic model for information storage and organization in the brain. **Psychological Review**, v. 65, n. 6, p. 386–408, 1958.

ROSOFSKY, S. et al. Applications of Physics-Informed Neural Operators (PINOs): Shallow Water Equations and Beyond. **arXiv preprint**, 2023. Available from: <https://github.com/shawnrosofsky/PINO_Applications>.

RUMELHART, D. E.; HINTON, G. E.; WILLIAMS, R. J. Learning representations by back-propagating errors. **Nature**, v. 323, n. 6088, p. 533–536, 1986.

STEINMOELLER, D. T.; STASTNA, M.; LAMB, K. G. Fourier pseudospectral methods for 2D Boussinesq-type equations. **Ocean Modelling**, v. 52–53, p. 76–89, 2012. DOI: 10.1016/j.ocemod.2012.05.003.

STRANG, G. **Linear Algebra and Its Applications**. 4. ed. Belmont: Thomson Brooks/Cole, 2006.

STROGATZ, S. H. **Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry, and Engineering**. 2. ed. Boca Raton: CRC Press, 2018.

SÜLI, E.; MAYERS, D. F. **An Introduction to Numerical Analysis**. Cambridge: Cambridge University Press, 2003.

TODO. A Review of Physics-Informed Machine Learning in Fluid Mechanics. **Energies**, v. 16, n. 5, 2023. DOI: 10.3390/en16052343.

_____. Modelo Numérico de Boussinesq Adaptado para Ondas de Embarcação. **Revista Brasileira de Recursos Hídricos**, v. 15, n. 4, p. 5–15, 2010. DOI: 10.21168/rbrh.v15n4.p5-15.

TREFETHEN, L. N. **Spectral Methods in MATLAB**. Philadelphia: Society for Industrial and Applied Mathematics, 2000. DOI: 10.1137/1.9780898719598.

UCHIDA, T. et al. Ocean Emulation With Fourier Neural Operators: Double Gyre. **Journal of Advances in Modeling Earth Systems**, v. 17, n. 1, 2025. DOI: 10.1029/2023MS004137.

WANG, L. et al. Explorations of nonlinear waves of the Boussinesq and Camassa-Holm equations using PINNs. **Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences**, v. 480, n. 2286, 2024. DOI: 10.1098/rspa.2023.0580.

WANG, S.; TENG, Y.; PERDIKARIS, P. Understanding and Mitigating Gradient Flow Pathologies in Physics-Informed Neural Networks. **SIAM Journal on Scientific Computing**, v. 43, n. 5, a3055–a3081, 2021. DOI: 10.1137/20M1318043.

WANG, S.; YU, X.; PERDIKARIS, P. When and Why PINNs Fail to Train: A Neural Tangent Kernel Perspective. **Journal of Computational Physics**, v. 449, p. 110768, 2022. DOI: 10.1016/j.jcp.2021.110768.

WHITHAM, G. B. **Linear and Nonlinear Waves**. New York: Wiley-Interscience, 1974.

XU, Z.-Q. J. et al. Frequency Principle: Fourier Analysis Sheds Light on Deep Neural Networks. In: 5. COMMUNICATIONS in Computational Physics. [S.l.: s.n.], 2020. v. 28, p. 1746–1767. DOI: 10.4208/cicp.OA-2020-0085.

YU, J. et al. Spherical multigrid neural operator for improving autoregressive global weather forecasting. **Scientific Reports**, v. 15, n. 1, 2025. DOI: 10.1038/s41598-025-96208-y.

ZHANG, Y. et al. Influence of layer and neuron configurations on Boussinesq equation solutions via a bilinear neural network. **Nonlinear Dynamics**, v. 112, p. 12315–12333, 2024. DOI: 10.1007/s11071-024-09708-3.

ZHONG, M.; YAN, Z.; TIAN, S. Data-driven parametric soliton-rogon state transitions for nonlinear wave equations using deep learning with Fourier neural operator. **Communications in Theoretical Physics**, v. 75, n. 2, p. 025001, 2023. DOI: 10.1088/1572-9494/acab55.